JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
ANANTAPURB.Tech (CSE)– III-I Sem
(20A05502T) ARTIFICIAL INTELLIGENCE

Unit – I: Introduction: What is AI, Foundations of AI, History of AI, The State of Art.
Intelligent Agents: Agents and Environments, Good Behaviour: The Concept of Rationality, The Nature of Environments, The Structure of Agents.

Unit – II: Solving Problems by searching:
Problem Solving Agents, Example problems, Searching for Solutions, Uninformed Search Strategies, Informed search strategies, Heuristic Functions.
Beyond Classical Search: Local Search Algorithms and Optimization Problems, Local Search in Continues Spaces, Searching with Nondeterministic Actions, Searching with partial observations, online search agents and unknown environments.

Unit – III: Reinforcement Learning:
Introduction, Passive Reinforcement Learning, Active Reinforcement Learning, Generalization in Reinforcement Learning, Policy Search, applications of RL.
Natural Language Processing: Language Models, Text Classification, Information Retrieval, Information Extraction.

Unit-IV: Natural Language for Communication:
Phrase structure grammars, Syntactic Analysis, Augmented Grammars and semantic Interpretation, Machine Translation, Speech Recognition
Perception: Image Formation, Early Image Processing Operations, Object Recognition by appearance, Reconstructing the 3D World, Object Recognition from Structural information, Using Vision.

Unit-V: Robotics:
Introduction, Robot Hardware, Robotic Perception, Planning to move, planning uncertain movements, Moving, Robotic software architectures, application domains

Philosophical foundations:
Weak AI, Strong AI, Ethics and Risks of AI, Agent Components, Agent Architectures, Are we going in the right direction, What if AI does succeed.

Textbook: 1. Stuart J.Russell, Peter Norvig, "Artificial Intelligence A Modern Approach", 3rd Edition, Pearson Education, 2019.
References: 1. Nilsson, Nils J., and Nils Johan Nilsson. Artificial intelligence: a new synthesis. Morgan Kaufmann, 1998.

UNIT-I

Unit – I: Introduction: What is AI, Foundations of AI, History of AI, The State of Art.

Intelligent Agents: Agents and Environments, Good Behaviour: The Concept of Rationality, The Nature of Environments, The Structure of Agents.

Since the invention of computers or machines, their capability to perform various tasks went on growing exponentially. Humans have developed the power of computer systems in terms of their diverse working domains, their increasing speed, and reducing size with respect to time.

A branch of Computer Science named *Artificial Intelligence* pursues creating the computers or machines as intelligent as human beings.

It is difficult to give a precise definition of artificial intelligence. Some recent artificial intelligence scientists have attempted to define artificial intelligence (in short AI) in various ways.

According to Haugeland, artificial intelligence is, "the exciting new effort to make computers think … machines with minds, in the full and literal sense

For Bellman, it is "the automation of activities that we associates with human thinking, activities such as decision making, problem solving".

Artificial Intelligence

According to the father of Artificial Intelligence, John McCarthy, it is *"The science and engineering of making intelligent machines, especially intelligent computer programs".*

Artificial Intelligence is a way of **making a computer, a computer-controlled robot, or a software think intelligently**, in the similar manner the intelligent humans think.

AI is accomplished by studying how human brain thinks, and how humans learn, decide, and work while trying to solve a problem, and then using the outcomes of this study as a basis of developing intelligent software and systems.

Philosophy of AI

While exploiting the power of the computer systems, the curiosity of human, lead him to wonder, *"Can a machine think and behave like humans do?"*

Thus, the development of AI started with the intention of creating similar intelligence in machines that we find and regard high in humans.

Goals of AI

- **To Create Expert Systems** − The systems which exhibit intelligent behavior, learn, demonstrate,explain, and advice its users.
- **To Implement Human Intelligence in Machines** − Creating systems that understand, think, learn and behave like humans.

What Contributes to AI?

Artificial intelligence is a science and technology based on disciplines such as Computer Science, Biology, Psychology, Linguistics, Mathematics, and Engineering. A major thrust of AI is in the development of computer functions associated with human intelligence, such as reasoning, learning, and problem solving.

Out of the following areas, one or multiple areas can contribute to build an intelligent system.

AI is used in the fields Psychology, neuron science, biology, maths, sociology, philosophy and computer science.

Programming Without and With AI

The programming without and with AI is different in following ways –

Programming Without AI	**Programming With AI**
A computer program without AI can answer the **specific** questions it is meant to solve.	A computer program with AI can answer the **generic** questions it is meant to solve.
Modification in the program leads to change in its structure.	AI programs can absorb new modifications by putting highly independent pieces of information together. Hence you can modify even a minute piece of information of program without affectingits structure.
Modification is not quick and easy. It may lead to affecting the program adversely.	Quick and Easy program modification.

What is AI Technique?

In the real world, the knowledge has some unwelcomed properties −

- Its volume is huge, next to unimaginable.
- It is not well-organized or well-formatted.
- It keeps changing constantly.

AI Technique is a manner to organize and use the knowledge efficiently in such a way that −

- It should be perceivable by the people who provide it.
- It should be easily modifiable to correct errors.
- It should be useful in many situations though it is incomplete or inaccurate.

AI techniques elevate the speed of execution of the complex program it is equipped with.

Applications of AI

AI has been dominant in various fields such as −

- **Gaming** − AI plays crucial role in strategic games such as chess, poker, tic-tac-toe, etc., where machine can think of large number of possible positions based on heuristic knowledge.
- **Natural Language Processing** − It is possible to interact with the computer that understands natural language spoken by humans.

- **Expert Systems** − There are some applications which integrate machine, software, and special information to impart reasoning and advising. They provide explanation and advice to the users.
- **Vision Systems** − These systems understand, interpret, and comprehend visual input on the computer. For example,
  - A spying aeroplane takes photographs, which are used to figure out spatial information or map of the areas.
  - Doctors use clinical expert system to diagnose the patient.
  - Police use computer software that can recognize the face of criminal with the stored portrait made by forensic artist.
- **Speech Recognition** − Some intelligent systems are capable of hearing and comprehending the language in terms of sentences and their meanings while a human talks to it. It can handle different accents, slang words, noise in the background, change in human's noise due to cold, etc.
- **Handwriting Recognition** − The handwriting recognition software reads the text written on paper by a pen or on screen by a stylus. It can recognize the shapes of the letters and convert it into editable text.
- **Intelligent Robots** − Robots are able to perform the tasks given by a human. They have sensors to detect physical data from the real world such as light, heat, temperature, movement, sound, bump, and pressure. They have efficient processors, multiple sensors and huge memory, to exhibit intelligence. In addition, they are capable of learning from their mistakes and they can adapt to the new environment.

All the definitions of AI can be organized into four categories. They are as follows

(i) Systems that think like humans.
(ii) Systems that act like humans.
(iii) Systems that think rationally.
(iv) Systems that act rationally

Building AI Systems:

The definitions of AI: Artificial Intelligence "The exciting new effort to make computers think . . . machines withminds, in the full and literal sense" (Haugeland, 1985)

"The automation of activities that we associate with human thinking, activities such as decision-making, problemsolving, learning..."(Bellman, 1978)

b) "The study of mental faculties through the use of computational models" (Charniak and

McDermott, 1985)"The study of the computations that make it possible to perceive, reason, and act"

(Winston, 1992)

c) "The art of creating machines that perform functions that require intelligence when performed by people"(Kurzweil, 1990)

"The study of how to make computers do things at which, at the moment, people are better" (Rich and Knight, 1 991)

:d) "A field of study that seeks to explain and emulate intelligent behavior in terms of computational processes"(Schalkoff, 1 990)

"The branch of computer science that is concerned with the automation of intelligent behavior" (Luger andStubblefield, 1993)

The definitions on the top, (a) and (b) are concerned with reasoning, whereas those on the bottom, (c) and (d) address behavior. The definitions on the left, (a) and (c) measure success in terms of human performance, and thoseon the right, (b) and (d) measure the ideal concept of intelligence called rationality.

History of AI: <u>Historical Developments of AI</u>

Artificial intelligence is a new research area of growing interdisciplinary interest and practical importance. People with widely varying backgrounds and professional knowledge are contributing new ideas and introducing new tools into this discipline. Cognitive minded psychologists have developed new models of the mind based on the fundamental concepts of artificial intelligence, symbols systems and information processing. Linguists are also interested in these basic notions while developing different models in computational linguistics. And philosophers, in considering the progress, problems and potential of this work towards nonhuman intelligence, have sometimes found solution to the age-old problems of the nature of mind and knowledge.

Artificial intelligence is a part of computer science in which are designed intelligent systems that exhibit the characteristics we associate with intelligence in human behaviour, understanding language learning, reasoning, problem solving, and so on. It is believed that insights into the nature of the mind can be gained by studying the operation of such systems. Artificial intelligence researchers have invented dozens of programming techniques that support some sort of intelligent behavior.

The history of AI during 20th century

Year	Innovation
1923	Karel Čapek play named “Rossum's Universal Robots” (RUR) opens in London, first use of the word "robot" in English.
1943	Foundations for neural networks laid.
1945	Isaac Asimov, a Columbia University alumni, coined the term *Robotics*
1950	Alan Turing introduced Turing Test for evaluation of intelligence and published *Computing Machinery and Intelligence*. Claude Shannon published *Detailed Analysis of Chess Playing* as a search.
1956	John McCarthy coined the term *Artificial Intelligence*. Demonstration of the first running AIprogram at Carnegie Mellon University.
1958	John McCarthy invents LISP programming language for AI.
1964	Danny Bobrow's dissertation at MIT showed that computers can understand natural language wellenough to solve algebra word problems correctly.
1965	Joseph Weizenbaum at MIT built *ELIZA*, an interactive problem that carries on a dialogue in English
1969	Scientists at Stanford Research Institute Developed *Shakey*, a robot, equipped with locomotion,perception, and problem solving.
1973	The Assembly Robotics group at Edinburgh University built *Freddy*, the Famous Scottish Robot,capable of using vision to locate and assemble

	models.
1979	The first computer-controlled autonomous vehicle, Stanford Cart, was built.
1985	Harold Cohen created and demonstrated the drawing program, *Aaron*.
1990	Major advances in all areas of AI such as Significant demonstrations in machine learning, Case-based reasoning, Games, Virtual Reality and NLP etc
1997	The Deep Blue Chess Program beats the then world chess champion, Garry Kasparov.
2000	Interactive robot pets become commercially available. MIT displays *Kismet*, a robot with a face thatexpresses emotions. The robot *Nomad* explores remote regions of Antarctica and locatesmeteorites.

(Another Definition)Artificial Intelligence:

The ability of a system to calculate, reason, perceive relationships and analogies, learn from experience, store and retrieve information from memory, solve problems, comprehend complex ideas, use natural language fluently, classify, generalize, and adapt new situations.

Types of Artificial Intelligence

In order to design intelligent systems, it is important to categorize them into four categories

1. Systems that think like humans
2. Systems that think rationally
3. Systems that behave like humans
4. Systems that behave rationally

Intelligence	**Description**	**Example**
Linguistic intelligence	The ability to speak, recognize, and use mechanisms of phonology (speech sounds), syntax (grammar),and semantics (meaning).	Narrators, Orators
Musical intelligence	The ability to create, communicate with, and understand meanings madeof sound, understanding of pitch, rhythm.	Musicians, Singers, Composers
Logical-mathematical intelligence	The ability of use and understand relationships in the absence of actionor objects. Understanding complex and abstract ideas.	Mathematicians,Scientists
Spatial intelligence	The ability to perceive visual or spatial information, change it, and re-create visual images without reference to theobjects, construct 3D images, and to move and rotate them	Map readers,Astronauts, Physicists

A machine or a system is **artificially intelligent** when it is equipped with at least one and at most all intelligences in it.

Intelligence Composed of

The intelligence is intangible. It is composed of −

- Reasoning
- Learning
- Problem Solving
- Perception
- Linguistic Intelligence

Let us go through all the components briefly −

- **Reasoning** − It is the set of processes that enables us to provide basis for judgment, making decisions, and prediction. There are broadly two types –

Inductive Reasoning	**Deductive Reasoning**
It conducts specific observations tomake broad general statements	It starts with a general statement and examines the possibilities to reach a specific, logical conclusion.
Even if all of the premises are true in a statement, inductive reasoning allows for the conclusion to be false.	If something is true of a class of things in general, it is also true for all members of that class.

- **Learning** − It is the activity of gaining knowledge or skill by studying, practising, being taught, or experiencing something. Learning enhances the awareness of the subjects of the study.

  The ability of learning is possessed by humans, some animals, and AI-enabled systems.Learning is categorized as −

  - **Auditory Learning** − It is learning by listening and hearing. For example, studentslistening to recorded audio lectures.
  - **Episodic Learning** − To learn by remembering sequences of events that one haswitnessed or experienced. This is linear and orderly.
  - **Motor Learning** − It is learning by precise movement of muscles. For example,picking objects, Writing, etc.
  - **Observational Learning** − To learn by watching and imitating others. For example,child tries to learn by mimicking her parent.
  - **Perceptual Learning** − It is learning to recognize stimuli that one has seen before. For example, identifying and classifying objects and situations.
  - **Relational Learning** − It involves learning to differentiate among various stimuli on the basis of relational properties, rather than absolute properties. For Example, Adding 'little less' salt at the time of cooking potatoes that came up salty last time, when cooked with adding say a tablespoon of salt.
  - **Spatial Learning** − It is learning through visual stimuli such as images, colors, maps, etc. For Example, A person can create roadmap in mind before actually following the road.
  - **Stimulus-Response Learning** − It is learning to perform a particular behavior whena certain stimulus is present. For example, a dog raises its ear on hearing doorbell.

- **Problem Solving** − It is the process in which one perceives and tries to arrive at a desiredsolution from a present situation by taking some path, which is blocked by known or unknown hurdles.

:

- Problem solving also includes **decision making**, which is the process of selecting the bestsuitable alternative out of multiple alternatives to reach the desired goal are available.
- **Perception** − It is the process of acquiring, interpreting, selecting, and organizing sensory information.

  Perception presumes **sensing**. In humans, perception is aided by sensory organs. In the domain of AI, perception mechanism puts the data acquired by the sensors together in a meaningful manner.
- **Linguistic Intelligence** − It is one's ability to use, comprehend, speak, and write the verbal and written language. It is important in interpersonal communication.

Difference between Human and Machine Intelligence

- Humans perceive by patterns whereas the machines perceive by set of rules and data.
- Humans store and recall information by patterns, machines do it by searching algorithms. For example, the number 40404040 is easy to remember, store, and recall as its pattern is simple.
- Humans can figure out the complete object even if some part of it is missing or distorted; whereas the machines cannot do it correctly.

Example-1: Tic-Tac-Toe

The first approach (simple) The Tic-Tac-Toe game consists of a nine element vector called BOARD; it represents the numbers 1 to 9 in three rows. An element contains the value 0 for blank, 1 for X and 2 for O. A MOVETABLE vector consists of 19,683 elements and is needed where each element is a nine element vector. The contents of the vector are especially chosen to help the algorithm.

1	2	3
4	5	6
7	8	9

The algorithm makes moves by pursuing the following: 1. View the vector as a ternary number. Convert itto a decimal number. 2. Use the decimal number as an index in MOVETABLE and access the vector. 3. Set BOARD to this vector indicating how the board looks after the move. This approach is capable in time but it has several disadvantages. It takes more space and requires stunning 8 effort to calculate the decimal numbers. This method is specific to this game and cannot be completed.

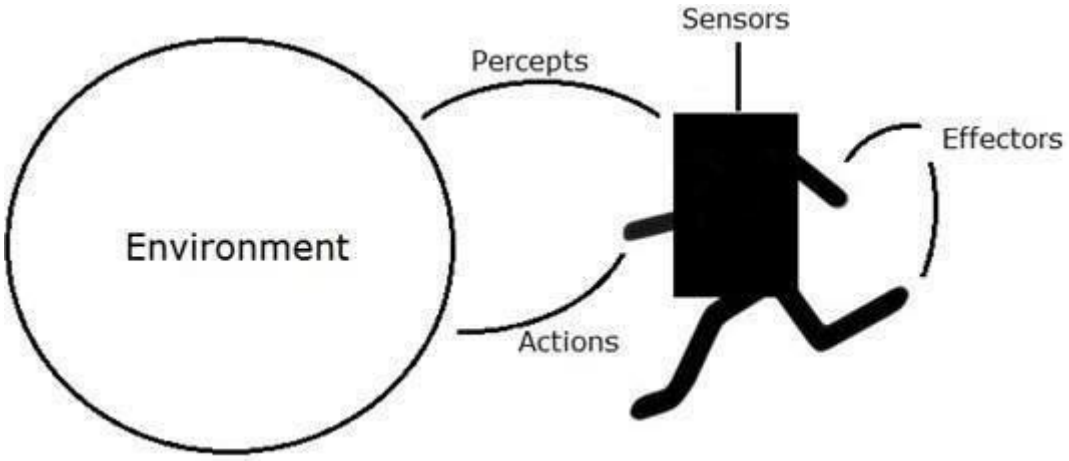
AI - Agents & Environments

An AI system is composed of an agent and its environment. The agents act in their environment.The environment may contain other agents.

Agent and Environment:

An **agent** is anything that can perceive its environment through **sensors** and acts upon that environment through **effectors.**

- A **human agent** has sensory organs such as eyes, ears, nose, tongue and skin parallel tothe sensors, and other organs such as hands, legs, mouth, for effectors.
- A **robotic agent** replaces cameras and infrared range finders for the sensors, and variousmotors and actuators for effectors.
- A **software agent** has encoded bit strings as its programs and actions.



Agent Terminology:

- **Performance Measure of Agent** − It is the criteria, which determines how successful anagent is.
- **Behavior of Agent** − It is the action that agent performs after any given sequence ofpercepts.
- **Percept** − It is agent's perceptual inputs at a given instance.
- **Percept Sequence** − It is the history of all that an agent has perceived till date.
- **Agent Function** − It is a map from the precept sequence to an action.

Rationality (INTELLIGENT AGENT)

Rationality is nothing but status of being reasonable, sensible, and having good sense of judgment.

Rationality is concerned with expected actions and results depending upon what the agent has perceived. Performing actions with the aim of obtaining useful information is an important part of rationality.

Ideal Rational Agent:

An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of −

- Its percept sequence
- Its built-in knowledge base

Rationality of an agent depends on the following −

- The **performance measures**, which determine the degree of success.
- Agent's **Percept Sequence** till now.

- The agent's **prior knowledge about the environment**.
- The **actions** that the agent can carry out.

A rational agent always performs right action, where the right action means the action that causesthe agent to be most successful in the given percept sequence. The problem the agent solves is characterized by Performance Measure, Environment, Actuators, and Sensors (PEAS).

The Structure of Intelligent Agents

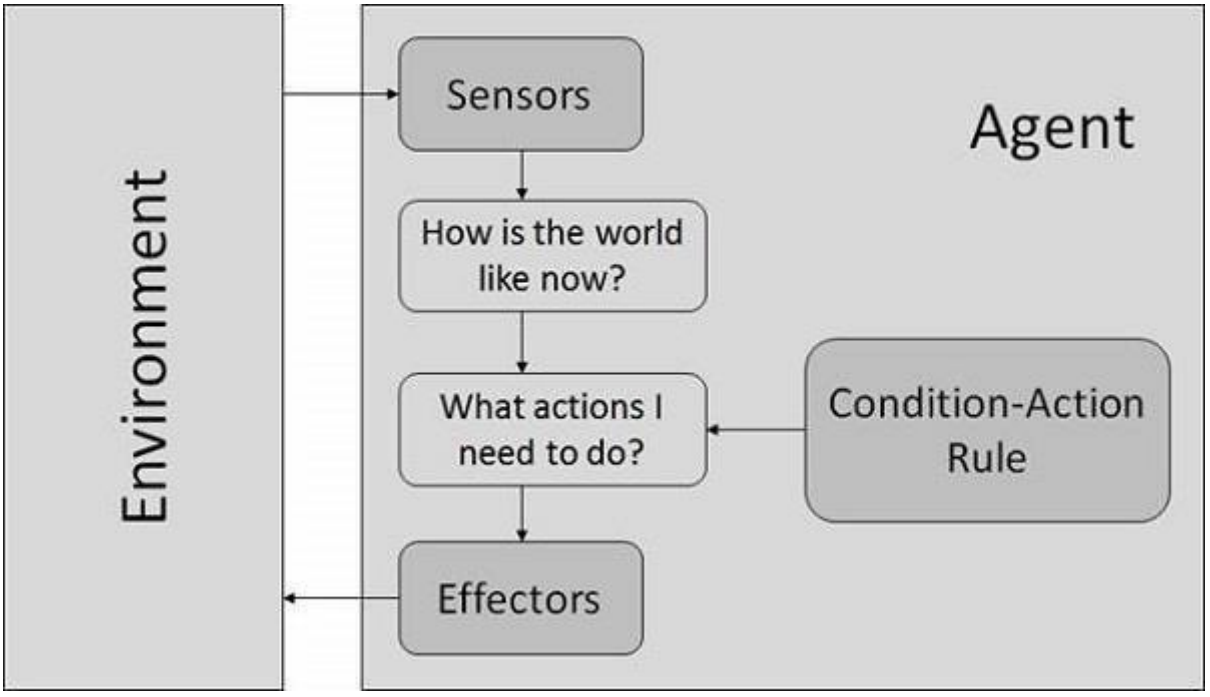
Agent's structure can be viewed as −

- Agent = Architecture + Agent Program
- Architecture = the machinery that an agent executes on.
- Agent Program = an implementation of an agent function.

Simple Reflex Agents

- They choose actions only based on the current percept.
- They are rational only if a correct decision is made only on the basis of current precept.
- Their environment is completely observable.

Condition-Action Rule − It is a rule that maps a state (condition) to an action.



Model Based Reflex Agents

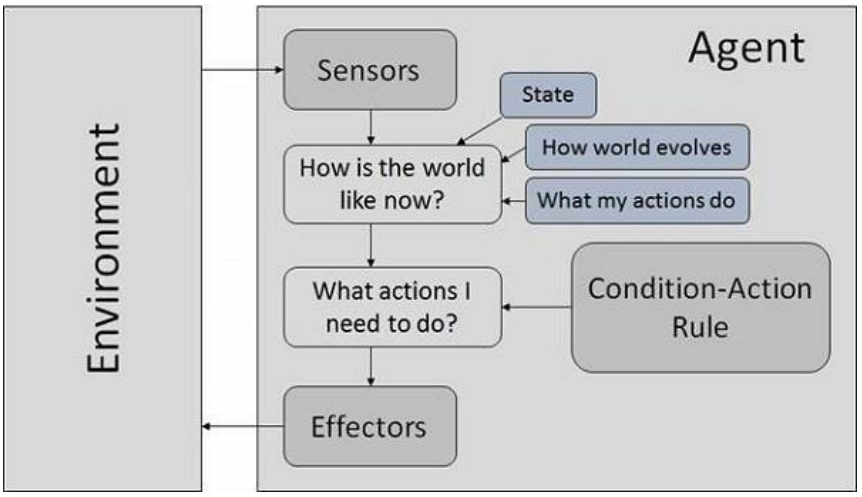
They use a model of the world to choose their actions. They maintain an internal state.

Model − knowledge about "how the things happen in the world".

Internal State − It is a representation of unobserved aspects of current state depending onpercept history.

Updating the state requires the information about −

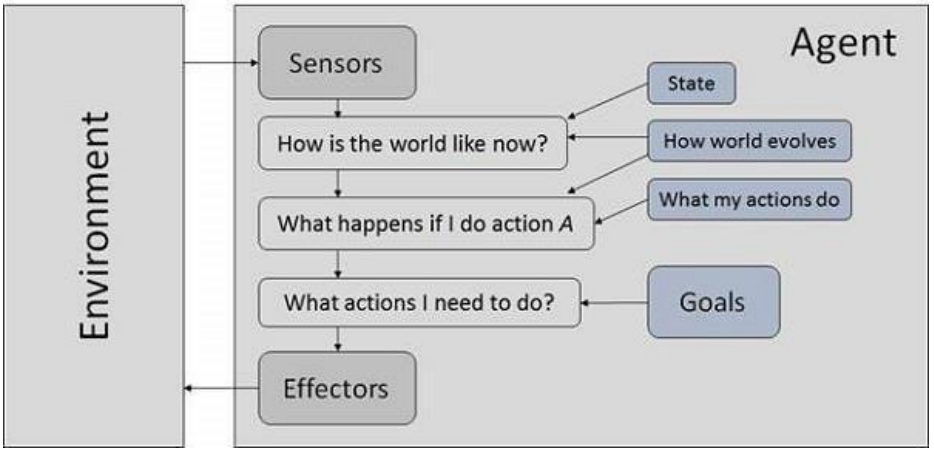
- How the world evolves.
- How the agent's actions affect the world



Goal Based Agents

They choose their actions in order to achieve goals. Goal-based approach is more flexible than reflex agent since the knowledge supporting a decision is explicitly modeled, thereby allowing for modifications.

Goal − It is the description of desirable situations.



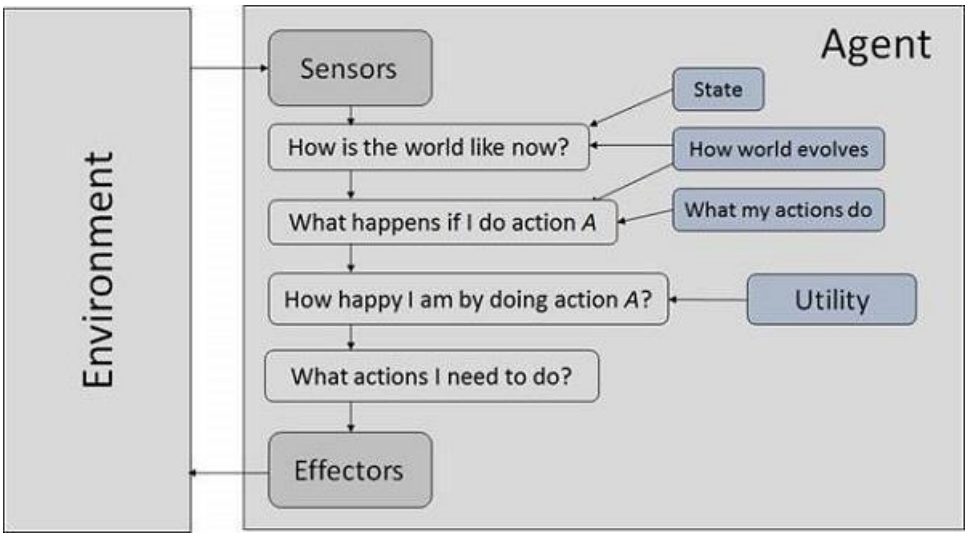
Utility Based Agents

They choose actions based on a preference (utility) for each state.Goals

are inadequate when −

- There are conflicting goals, out of which only few can be achieved.
- Goals have some uncertainty of being achieved and you need to weigh likelihood ofsuccess against the importance of a goal.

:



The Nature of Environments

Some programs operate in the entirely **artificial environment** confined to keyboard input, database, computer file systems and character output on a screen.

In contrast, some software agents (software robots or softbots) exist in rich, unlimited softbots domains. The simulator has a **very detailed, complex environment**. The software agent needs to choose from a long array of actions in real time. A softbot designed to scan the online preferences of the customer and show interesting items to the customer works in the **real** as well as an **artificial** environment.

The most famous **artificial environment** is the **Turing Test environment**, in which one real and other artificial agents are tested on equal ground. This is a very challenging environment as it is highly difficult for a software agent to perform as well as a human.

Turing Test

The success of an intelligent behavior of a system can be measured with Turing Test.

Two persons and a machine to be evaluated participate in the test. Out of the two persons, one plays the role of the tester. Each of them sits in different rooms. The tester is unaware of who is machine and who is a human. He interrogates the questions by typing and sending them to both intelligences, to which he receives typed responses.

This test aims at fooling the tester. If the tester fails to determine machine's response from the human response, then the machine is said to be intelligent.

Properties of Environment

The environment has multifold properties −

- **Discrete / Continuous** − If there are a limited number of distinct, clearly defined, states of the environment, the environment is discrete (For example, chess); otherwise it is continuous (For example, driving).
- **Observable / Partially Observable** − If it is possible to determine the complete state of the environment at each time point from the percepts it is observable; otherwise it is only partially observable.

- **Static / Dynamic** − If the environment does not change while an agent is acting, then it is static; otherwise it is dynamic.
- **Single agent / Multiple agents** − The environment may contain other agents which may be of the same or different kind as that of the agent.
- **Accessible / Inaccessible** − If the agent's sensory apparatus can have access to the complete state of the environment, then the environment is accessible to that agent.
- **Deterministic / Non-deterministic** − If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
- **Episodic / Non-episodic** − In an episodic environment, each episode consists of the agentperceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions in the previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

:Unit – II: Solving Problems by searching: Problem Solving Agents, Example problems, Searching for Solutions, Uninformed Search Strategies, Informed search strategies, Heuristic Functions, Beyond Classical Search: Local Search Algorithms and Optimization Problems, Local Search in Continues Spaces, Searching with Nondeterministic Actions, Searching with partial observations, online search agents and unknown environments.

Problem-solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation.

Search Algorithm Terminologies:

- **Search:** Searching is a step by step procedure to solve a search-problem in a given searchspace. A search problem can have three main factors:
  a. **Search Space:** Search space represents a set of possible solutions, which a systemmay have.
  b. **Start State:** It is a state from where agent begins **the search**.
  c. **Goal test:** It is a function which observe the current state and returns whether thegoal state is achieved or not.

Search tree: A tree representation of search problem is called Search tree. The root of the searchtree is the root node which is corresponding to the initial state.

Actions: It gives the description of all the available actions to the agent.

Transition model: A description of what each action do, can be represented as a transition model.

Path Cost: It is a function which assigns a numeric cost to each path.

Solution: It is an action sequence which leads from the start node to the goal node.

Optimal Solution: If a solution has the lowest cost among all solutions.

Problem Space − It is the environment in which the search takes place. (A set of states and setof operators to change those states)

Problem Instance − It is Initial state + Goal state.

Problem Space Graph − It represents problem state. States are shown by nodes and operatorsare shown by edges.

Depth of a problem − Length of a shortest path or shortest sequence of operators from InitialState to goal state.

Space Complexity − The maximum number of nodes that are stored in memory.

Time Complexity − The maximum number of nodes that are created.

Admissibility − A property of an algorithm to always find an optimal solution. **Branching**

Factor − The average number of child nodes in the problem space graph.

Depth − Length of the shortest path from initial state to goal state.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

***Completeness*: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.**

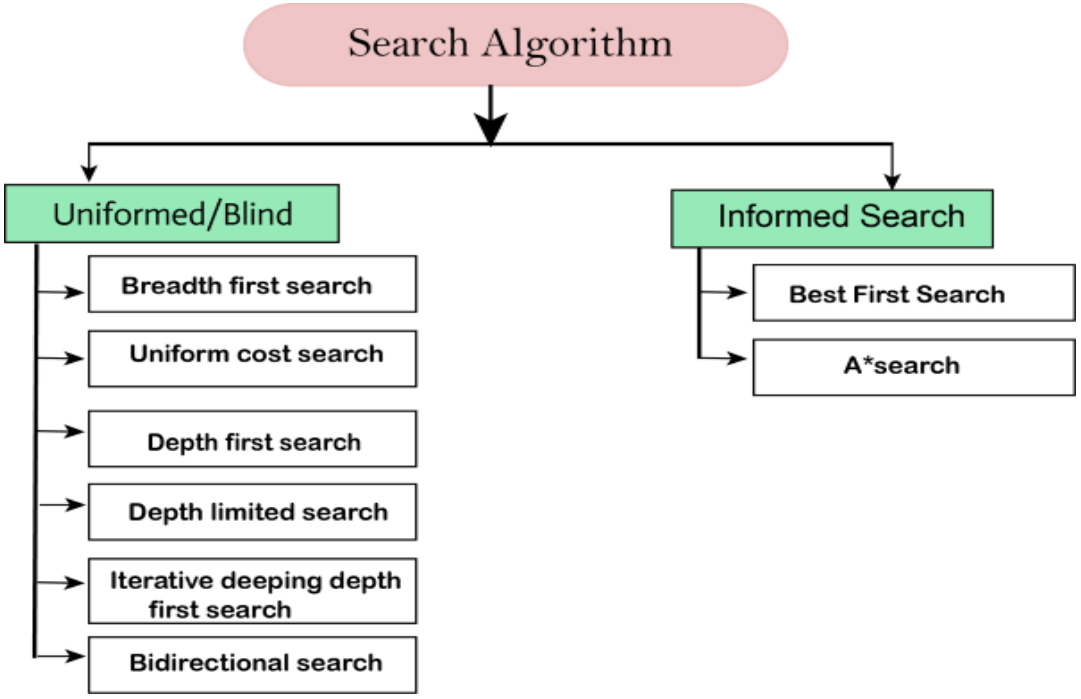
***Optimality:* If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be anoptimal solution.**

***Time Complexity*: Time complexity is a measure of time for an algorithm to complete itstask.**

***Space Complexity*: It is the maximum storage space required at any point during the search, as the complexity of the problem.**

Types of search algorithms:

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Searching is the universal technique of problem solving in AI. There are some single-player games such as tile games, Sudoku, crossword, etc. The search algorithms help you to search for a particular position in such games.

Single Agent Path finding Problems

The games such as 3X3 eight-tile, 4X4 fifteen-tile, and 5X5 twenty four tile puzzles are single- agent-path-finding challenges. They consist of a matrix of tiles with a blank tile. The player is required to arrange the

tiles by sliding a tile either vertically or horizontally into a blank space with the aim of accomplishing some objective.

The other examples of single agent path finding problems are Travelling Salesman Problem, Rubik's Cube, and Theorem Proving.

Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which searchtree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve much complex problem which could not be solved in another way.An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

Uninformed Search Algorithms:

Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

Following are the various types of uninformed search algorithms:

1. **Breadth-first Search**
2. **Depth-first Search**
3. **Depth-limited Search**
4. **Iterative deepening depth-first search**

5. **Uniform cost search**
6. **Bidirectional Search**

1. Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth-wise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

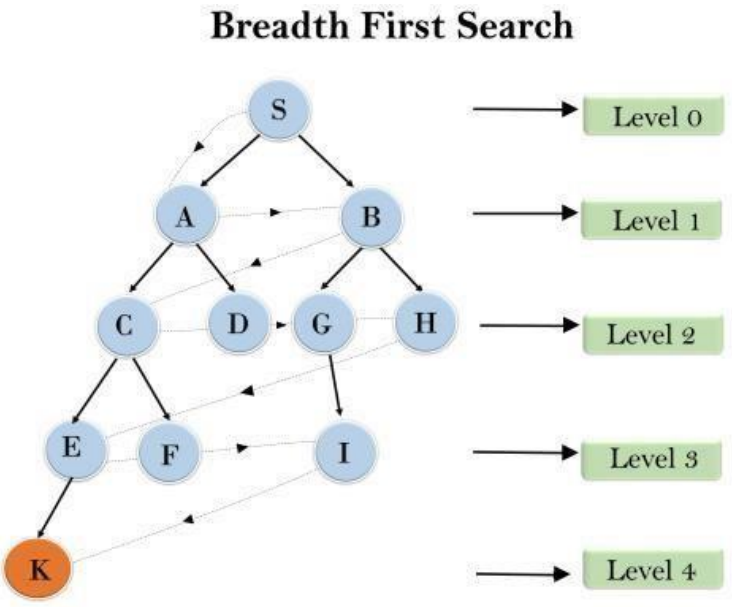
Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I------>K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$$T(b) = 1+b^2+b^3+. \ldots\ldots + b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth,then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Note: *Backtracking is an algorithm technique for finding all possible solutions using recursion.*

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

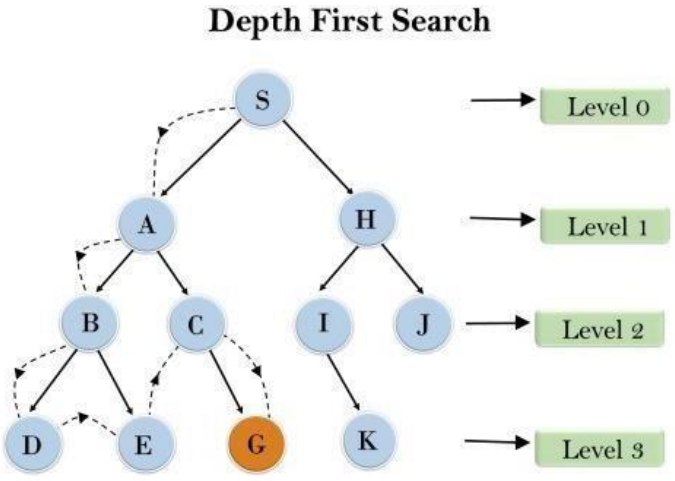
Disadvantage:

There is the possibility that many states keep re-occurring, and there is no guarantee of finding thesolution.

- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:
Root node--->Left node > right node.



It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will

traverse node C and then G, and here it will terminate as it found goal node.

:

Completeness: DFS search algorithm is complete within finite state space as it will expand everynode within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowestsolution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hencespace complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or highcost to reach to the goal node.

3. Depth-Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

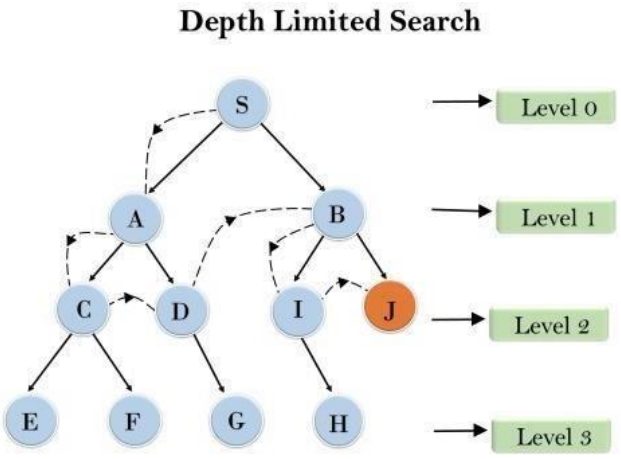
Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Example:



Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is **$O(b^{\ell})$**. **Space Complexity:** Space complexity of DLS algorithm is **$O(b\times\ell)$**.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

4. Uniform-cost Search Algorithm:

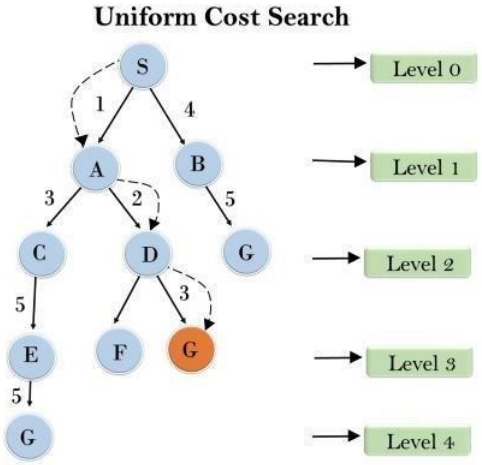
:Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs from the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Advantages:

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.



Completeness:

Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity:

Let C* **is Cost of the optimal solution**, and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is $\mathbf{O(b^{1 + \lceil C^*/\varepsilon \rceil})}$/.

5. Iterative deepening depth-first Search:

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This searchalgorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Advantages:

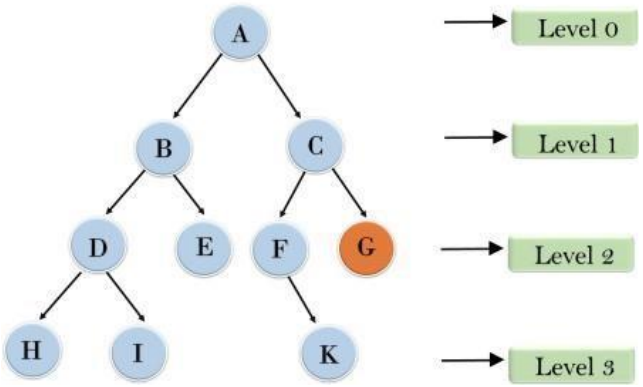
- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memoryefficiency.

Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:

Iterative deepening depth first search



1'st Iteration ------ > A
2'nd Iteration ----- > A, B, C
3'rd Iteration -------- >A, B, D, E, C, F, G
4'th Iteration -------- >A, B, D, H, I, E, C, F, K, G
In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete is if the branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexityis **$O(b^d)$**.

Space Complexity:

The space complexity of IDDFS will be **O(bd)**.

Optimal:

ID-DFS algorithm is optimal if path cost is a non- decreasing function of the depth of the node.

6. Bidirectional Search Algorithm:

Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

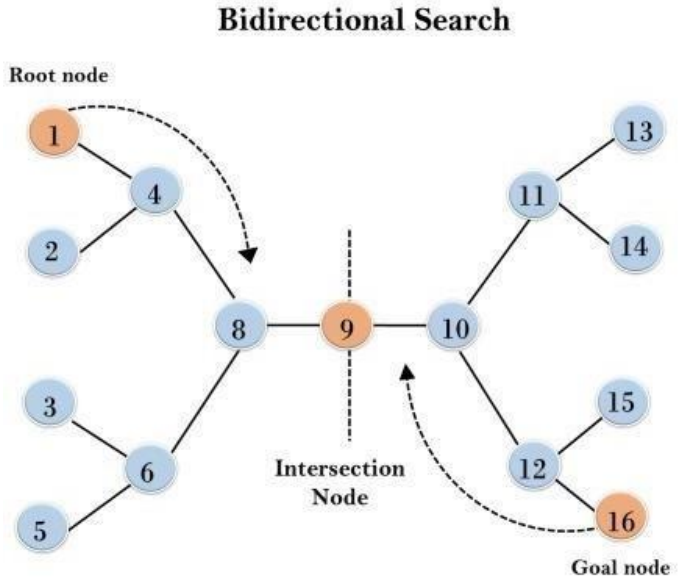
Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- **In bidirectional search, one should know the goal state in advance.**

Example:

In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.



Completeness: Bidirectional Search is complete if we use BFS in both searches.**Time Complexity:** Time complexity of bidirectional search using BFS is **O(b^d)**. **Space Complexity:** Space complexity of bidirectional search is **O(b^d)**.

Optimal: Bidirectional search is Optimal.

Informed Search Algorithms

The uninformed search algorithms which looked through search space for all possible solutions of the problem without having any additional knowledge about search space. But informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc. This knowledge helps agents to explore less to the search space and find more efficiently the goal node.

The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by h(n), and it calculates the cost of an optimalpath between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as:

1. h(n) <= h*(n)

   Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should beless than or equal to the estimated cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet notbeen expanded. On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

1.) Best-first Search Algorithm (Greedy Search):

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

1. f(n)= g(n).

Were, h(n)= estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it inthe CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Advantages:

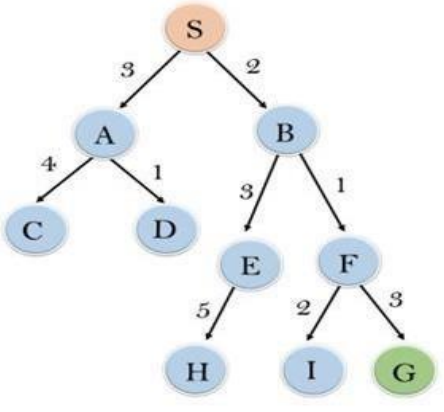
- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

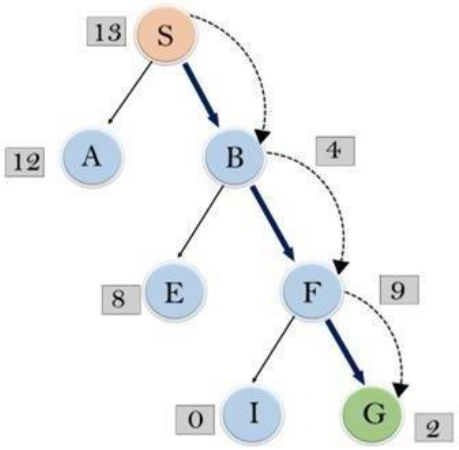
Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are theiteration for traversing the above example.



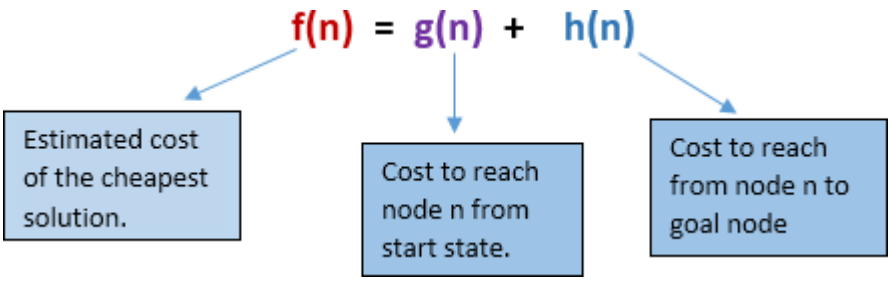
Expand the nodes of S and put in the CLOSED list
Initialization: Open [A, B], Closed [S]
Iteration 1: Open [A], Closed [S, B]
Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]
Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S----> B----->F----------> G**
Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, mis the maximum depth of the search space.
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.
Optimal: Greedy best first search algorithm is not optimal.

2.) **A* Search Algorithm:**
A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state g(n). It has combined features of UCS and greedybest-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we cancombine both costs as following, and this sum is called as a **fitness number**.



At each point in the search space, only those node is expanded which have the lowest value of f(n), and the algorithm terminates when the goal node is found.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then computeevaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to **Step 2**.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

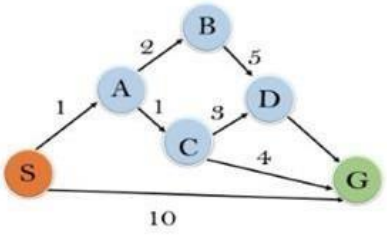
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

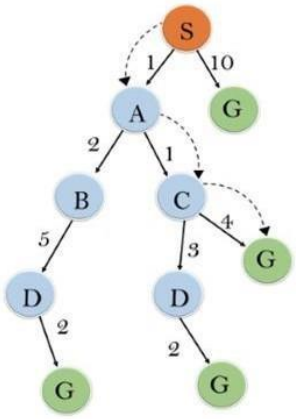
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), whereg(n) is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: {(S, 5)}
Iteration1: {(S--> A, 4), (S-->G, 10)}

Iteration2: {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}
Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}
Iteration 4 will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.
Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)<="" li="">

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.
Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity isO(b^d), where b is the branching factor.
Space Complexity: The space complexity of A* search algorithm is **O(b^d)**

Local Search Algorithms

Hill Climbing Algorithm in Artificial Intelligence

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.

- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

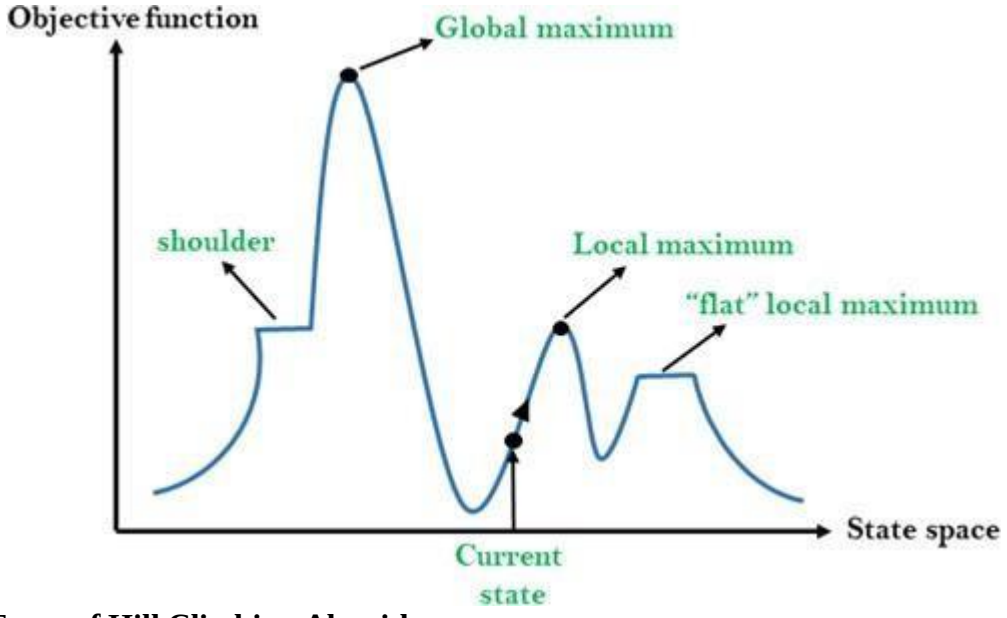
Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state**. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
    a. If it is goal state, then return success and quit.
    b. Else if it is better than the current state then assign new state as a current state.
    c. Else if not better than the current state, then return to step2.

Step 5: Exit.

2. Steepest-Ascent hill climbing:

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors

Algorithm for Steepest-Ascent hill climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
- **Step 2:** Loop until a solution is found or the current state does not change.
    a. Let SUCC be a state such that any successor of the current state will be better than it.
    b. For each operator that applies to the current state:
        a. Apply the new operator and generate a new state.
        b. Evaluate the new state.
        c. If it is goal state, then return it and quit, else compare it to the SUCC.
        d. If it is better than SUCC, then set new state as SUCC.
        e. If the SUCC is better than the current state, then set current state to SUCC.

Step 5: Exit.

3. Stochastic hill climbing:

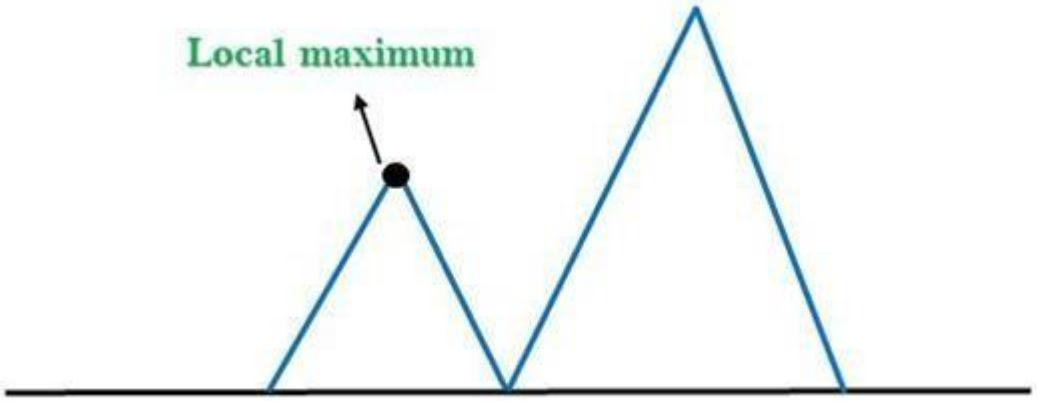
Stochastic hill climbing does not examine for all its neighbor before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Problems in Hill Climbing Algorithm:

1. Local Maximum: A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

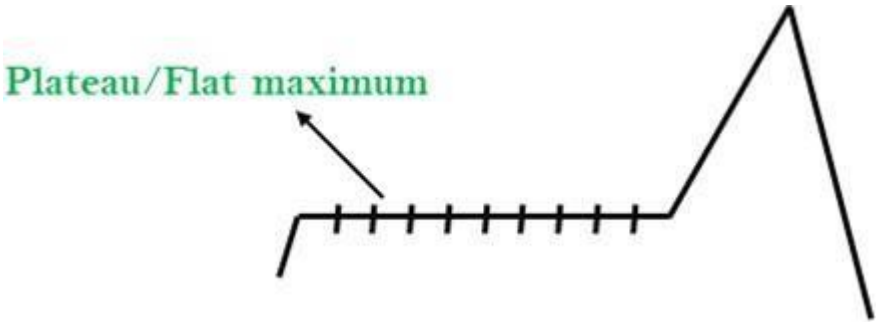
Solution: Backtracking technique can be a solution of the local maximum in state space landscape.Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as

well.



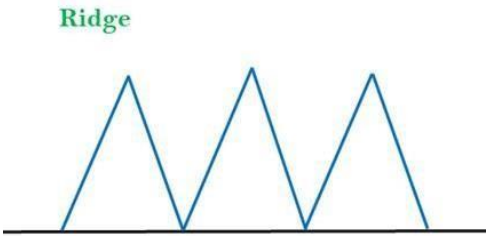
2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction tomove. A hill-climbing search might be lost in the plateau area.

Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possiblethat the algorithm could find non-plateau region.



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.



Simulated Annealing:

A hill-climbing algorithm which never makes a move towards a lower value guaranteed to be incomplete because it can get stuck on a local maximum. And if algorithm applies a random walk, by moving a successor, then it may complete but not efficient. **Simulated Annealing** is an algorithm which yields both efficiency and completeness.

In mechanical term **Annealing** is a process of hardening a metal or glass to a high temperature then cooling gradually, so this allows the metal to reach a low-energy crystalline state. The same process is used in simulated annealing in which the algorithm picks a random move, instead of picking the best move. If the random move improves the state, then it follows the same path. Otherwise, the algorithm follows the path which has a probability of less than 1 or it moves downhill and chooses another path.

They start from a prospective solution and then move to a neighboring solution. They can return a valid solution even if it is interrupted at any time before they end.

Hill-Climbing Search

It is an iterative algorithm that starts with an arbitrary solution to a problem and attempts to find a better solution by changing a single element of the solution incrementally. If the change produces a better solution, an incremental change is taken as a new solution. This process is repeated untilthere are no further improvements.

function Hill-Climbing (problem), returns a state that is a local maximum.

inputs: problem, a problem
local variables: current, a node
neighbor, a node
current <-Make_Node(Initial-State[problem])loop
do neighbor <- a highest_valued successor of *current*
if Value[neighbor] ≤ Value[current] then
return State[current]current <-
neighbor

end

Disadvantage − This algorithm is neither complete, nor optimal.

Local Beam Search

In this algorithm, it holds k number of states at any given time. At the start, these states are generated randomly. The successors of these k states are computed with the help of objective function. If any of these successors is the maximum value of the objective function, then the algorithm stops.

Otherwise the (initial k states and k number of successors of the states = 2k) states are placed in a pool. The pool is then sorted numerically. The highest k states are selected as new initial states. This process continues until a maximum value is reached.

function BeamSearch(*problem, k*), returns a solution state.

start with k randomly generated statesloop
generate all successors of all k states
if any of the states = solution, then return the stateelse select the k best successors
end

Simulated Annealing

Annealing is the process of heating and cooling a metal to change its internal structure for modifying its physical properties. When the metal cools, its new structure is seized, and the metal retains its newly obtained properties. In simulated annealing process, the temperature is kept variable.

We initially set the temperature high and then allow it to 'cool' slowly as the algorithm proceeds. When the temperature is high, the algorithm is allowed to accept worse solutions with high frequency.

Start

- Initialize k = 0; L = integer number of variables;
- From i → j, search the performance difference Δ.
- If Δ <= 0 then accept else if exp(-Δ/T(k)) > random(0,1) then accept;
- Repeat steps 1 and 2 for L(k) steps.
- k = k + 1;

Repeat steps 1 through 4 till the criteria is met.End

Differentiate between Dynamic Programming and Greedy Method

Dynamic Programming	**Greedy Method**
Dynamic Programming is used to obtain the optimal solution.	Greedy Method is also used to get the optimal solution.
In Dynamic Programming, we choose at each step, but the choice may depend on the solution to sub-problems	In a greedy Algorithm, we make whatever choice seems best at the moment and then solve the sub-problems arising after the choice is made.
Less efficient as compared to a greedyapproach	More efficient as compared to a greedy approach
Example: 0/1 Knapsack	Example: Fractional Knapsack
It is guaranteed that dynamic Programming generate an optimal solution using Principle of Optimality	In Greedy Method, there is no such guarantee of getting Optimal Solution.

Backtracking

Backtracking is one of the techniques that can be used to solve the problem. We can write the algorithm using this strategy. It uses the Brute force search to solve the problem, and the brute force search says that for the given problem, we try to make all the possible solutions and pick out the best solution from all the desired solutions.

Backtracking name itself suggests that we are going back and coming forward; if it satisfies the condition, then return success, else we go back again. It is used to solve a problem in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criteria.

When to use a Backtracking algorithm?

When we have multiple choices, then we make the decisions from the available choices. In thefollowing cases, we need to use the backtracking algorithm:

- A piece of sufficient information is not available to make the best choice, so we use the backtracking strategy to try out all the possible solutions.
- Each decision leads to a new set of choices. Then again, we backtrack to make new decisions. In this case, we need to use the backtracking strategy.

Applications of Backtracking

- N-queen problem
- Sum of subset problem
- Graph coloring
- Hamiliton cycle

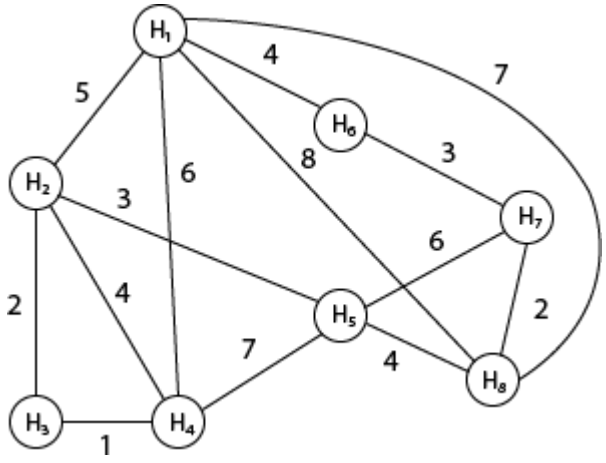
Travelling Sales Person Problem

The traveling salesman problems abide by a salesman and a set of cities. The salesman has to visit every one of the cities starting from a certain one (e.g., the hometown) and to return to the same city. The challenge of the problem is that the traveling salesman needs to minimize the total length of the trip.

Suppose the cities are x_1 x_2 x_n where cost c_{ij} denotes the cost of travelling from city x_i to x_j. The travelling salesperson problem is to find a route starting and ending at x_1 that will take in all cities with the minimum cost.

Example: A newspaper agent daily drops the newspaper to the area assigned in such a manner that he has to cover all the houses in the respective area with minimum travel cost. Compute the minimum travel cost.

The area assigned to the agent where he has to drop the newspaper is shown in fig:



Solution: The cost- adjacency matrix of graph G is as follows:

costij =

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
H_1	0	5	0	6	0	4	0	7
H_2	5	0	2	4	3	0	0	0
H_3	0	2	0	1	0	0	0	0
H_4	6	4	1	0	7	0	0	0
H_5	0	3	0	7	0	0	6	4
H_6	4	0	0	0	0	0	3	0
H_7	0	0	0	0	6	3	0	2
H_8	7	0	0	0	4	0	2	0

The tour starts from area H1 and then select the minimum cost area reachable from H1.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
(H1)	0	5	0	6	0	(4)	0	7
H_2	5	0	2	4	3	0	0	0
H_3	0	2	0	1	0	0	0	0
H_4	6	4	1	0	7	0	0	0
H_5	0	3	0	7	0	0	6	4
H_6	4	0	0	0	0	0	3	0
H_7	0	0	0	0	6	3	0	2
H_8	7	0	0	0	4	0	2	0

Mark area H6 because it is the minimum cost area reachable from H1 and then select minimumcost area reachable from H6.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
(H1)	0	5	0	6	0	(4)	0	7
H2	5	0	2	4	3	0	0	0
H3	0	2	0	1	0	0	0	0
H4	6	4	1	0	7	0	0	0
H5	0	3	0	7	0	0	6	4
(H6)	4	0	0	0	0	0	(3)	0
H7	0	0	0	0	6	3	0	2
H8	7	0	0	0	4	0	2	0

Mark area H7 because it is the minimum cost area reachable from H6 and then select minimumcost area reachable from H7.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
(H1)	0	5	0	6	0	(4)	0	7
H2	5	0	2	4	3	0	0	0
H3	0	2	0	1	0	0	0	0
H4	6	4	1	0	7	0	0	0
H5	0	3	0	7	0	0	6	4
(H6)	4	0	0	0	0	0	(3)	0
(H7)	0	0	0	0	6	3	0	(2)
H8	7	0	0	0	4	0	2	0

Mark area H8 because it is the minimum cost area reachable from H8.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
(H1)	0	5	0	6	0	(4)	0	7
H2	5	0	2	4	3	0	0	0
H3	0	2	0	1	0	0	0	0
H4	6	4	1	0	7	0	0	0
H5	0	3	0	7	0	0	6	4
(H6)	4	0	0	0	0	0	(3)	0
(H7)	0	0	0	0	6	3	0	(2)
(H8)	7	0	0	0	(4)	0	2	0

Mark area H5 because it is the minimum cost area reachable from H5.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
H_1	0	5	0	6	0	(4)	0	7
H_2	5	0	2	4	3	0	0	0
H_3	0	2	0	1	0	0	0	0
H_4	6	4	1	0	7	0	0	0
H_5	0	(3)	0	7	0	0	6	4
H_6	4	0	0	0	0	0	(3)	0
H_7	0	0	0	0	6	3	0	(2)
H_8	7	0	0	0	(4)	0	2	0

Mark area H2 because it is the minimum cost area reachable from H2.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
H_1	0	5	0	6	0	(4)	0	7
H_2	5	0	(2)	4	3	0	0	0
H_3	0	2	0	1	0	0	0	0
H_4	6	4	1	0	7	0	0	0
H_5	0	(3)	0	7	0	0	6	4
H_6	4	0	0	0	0	0	(3)	0
H_7	0	0	0	0	6	3	0	(2)
H_8	7	0	0	0	(4)	0	2	0

Mark area H3 because it is the minimum cost area reachable from H3.

	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
H_1	0	5	0	6	0	(4)	0	7
H_2	5	0	(2)	4	3	0	0	0
H_3	0	2	0	(1)	0	0	0	0
H_4	6	4	1	0	7	0	0	0
H_5	0	(3)	0	7	0	0	6	4
H_6	4	0	0	0	0	0	(3)	0
H_7	0	0	0	0	6	3	0	(2)
H_8	7	0	0	0	(4)	0	2	0

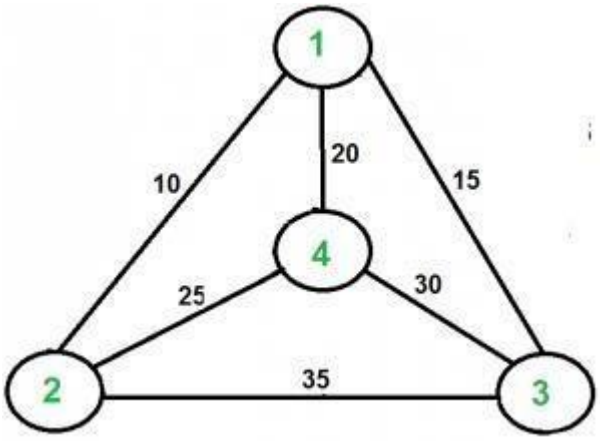
Mark area H4 and then select the minimum cost area reachable from H4 it is H1.So, using thegreedy strategy, we get the following.

4 3 2 4 3 2 1 6

$H_1 \rightarrow H_6 \rightarrow H_7 \rightarrow H_8 \rightarrow H_5 \rightarrow H_2 \rightarrow H_3 \rightarrow H_4 \rightarrow {}_{H1}$.

Thus the minimum travel cost = 4 + 3 + 2 + 4 + 3 + 2 + 1 + 6 = 25

```
# Python3 program to implement traveling salesman
from sys import maxsize
from itertools import permutationsV = 4
def travellingSalesmanProblem(graph, s):vertex =
    []
    for i in range(V):if i
        != s:
            vertex.append(i)
    min_path = maxsize
    next_permutation=permutations(vertex)for i
    in next_permutation:
        current_pathweight = 0k
        = s
        for j in i:
            current_pathweight += graph[k][j]k =
            j
        current_pathweight += graph[k][s]
        min_path = min(min_path, current_pathweight)
    return min_path
if __name__== "__main__":
    graph = [[0, 10, 15, 20], [10, 0, 35, 25],
             [15, 35, 0, 30], [20, 25, 30, 0]]
    s = 0 print(travellingSalesmanProblem(graph, s))
```



Output of Given Graph:

minimum weight Hamiltonian Cycle :10 + 25 +30 + 15 = 80

LOCAL SEARCH IN CONTINUES SPACES

LOCAL SEARCH IN CONTINUOUS SPACES

- We have considered algorithms that work only in discrete environments, but real-world environment arecontinuous.
- Local search amounts to maximizing a continuous objective function in a multi-dimensional
- vector space.This is hard to do in general, Can immediately retreat
- Apply a discrete local search strategy (e.g., stochastic hill climbing, simulated
- annealing)Amounts to greedy hill climbing in discretized state space
- Can employ Newton-Raphson Method to find maxima.
- Continuous problems have similar problems: plateaus, ridges, local maxima, etc.

Defining the problem as State Space Search:

The state space representation forms the basis of most of the AI methods.

Formulate a problem as a **state space search** by showing the legal problem states, the legal operators, and theinitial and goal states.

A **state** is defined by the specification of the values of all attributes of interest in the world

An **operator** changes one state into the other; it has a precondition which is the value of certain attributes prior tothe application of the operator, and a set of effects, which are the attributes altered by the operator

The **initial state** is where you start

The **goal state** is the partial description of the solution

Formal Description of the problem:

1. Define a state space that contains all the possible configurations of the relevant objects.

2. Specify one or more states within that space that describe possible situations from which the problem solvingprocess may start **(initial state)**

3. Specify one or more states that would be acceptable as solutions to the problem**. (goal states)**

Specify a set of rules that describe the actions **(operations)** available

State-Space Problem Formulation:

Example: A problem is defined by four items:

1. **initial state** e.g., "at Arad**—**

2. **actions or successor function** : *S(x)* = set of action–state pairs e.g., *S(Arad)* = {<*Arad* □ *Zerind*, *Zerind*>, … }

3. goal test (or set of goal states)

e.g., *x* = "at Bucharest‖, *Checkmate(x)*

4. path cost (additive)

e.g., sum of distances, number of actions executed, etc.

c(x,a,y) is the step cost, assumed to be ≥ 0

A solution is a sequence of actions leading from the initial state to a goal state

SEARCHING WITH NONDETERMINISTIC ACTIONS

Can't be sure of the outcome of actions – Percepts help narrow down possible resultant states

• Need a contingency plan (or strategy) that specifies what to do depending on what percepts are received

SEARCHING WITH PARTIAL OBSERVATIONS

Introduce belief state: represents agent's current belief about possible physical states it could be in, given sequenceof actions and percepts up to that point

• 3 scenarios: –

- **Searching with no observation –**
- **Searching with observations –**
- **Solving partially observable**

problemsSearching with no observations: Sensorless

Example: Vacuum world with no sensors: – Agent knows geography of environment – Agent doesn't know its location or distribution of dirt – Initial state: {1, 2, 3, 4, 5, 6, 7, 8} – Action outcomes: • [Right]: possible successor states: {2, 4, 6, 8} • [Right, Suck]: {4, 8} • [Right, Suck, Left Suck]: {7}

Searching with observations

Define PERCEPT(s) that returns the agent's percept, given the state s • E.g., In Vacuum world, PERCEPT(state1)=[A,Dirty] • Special cases: – Fully observable problems: PERCEPT(s) = s – Sensorless problems:PERCEPT(s) = Null

Searching with Partial observations

Prediction: given action a in belief state b, predict the resulting belief state: $b\hat{}$ = Predict(b, a)

• Observation prediction: determine set of percepts o that could be observed in the predicted belief state:POSSIBLE_PERCEPTS($b\hat{}$)= {o: o = PERCEPT s and $s \in b\hat{}$} •

Update: determine belief state that results from each possible percept (i.e., which set of states in $b\hat{}$ could have produced the percept) b0=UPDATE ($b\hat{}$,o)= {s: o =PERCEPT s and $s \in b\hat{}$} • Then, obtain possible belief states resulting from action and subsequent possible percepts: RESULTS $b, a = b$0: b0= UPDATE(PREDICT(b, a , o) and $o \in$POSSIBLE−PERCEPTS (PREDICT (b, a))}

Can use AND-OR search algorithm to solve

Online search: – Interleaves computation and action • "Solved" by an agent executing actions – Useful for dynamicenvironments – Useful for nondeterministic environments, since it allows agent to focus on contingencies that actually arise – not just those that might arise – Necessary for unknown environments

Agent only knows: – Actions(s) – list of actions allowed in state s – Step-cost function c(s, a, s') – can only be determined after s' discovered – Goal-Test(s) • Agent might know: admissible heuristic to determine distance to goalstate • Cost: total path cost of the path the agent actually travels • Competitive ratio: ratio of actual cost to optimal cost (i.e., best case if the agent knew the search space in advance) – "1" is optimal actual cost

Agent can only explore from current state – Can't bounce around to other search paths, like offline A* – DFS can beused, as long as actions are reversible – Also online versions of iterative deepening will work – Random walk is possible • Select at random an available action from current state

• Preference to previously unexplored actions • Will eventually find goal, if space is finite • But slow – exponential search in worst case:

Unit – III: Reinforcement Learning: Introduction, Passive Reinforcement Learning, Active Reinforcement Learning, Generalization in Reinforcement Learning, Policy Search, applications of RL.

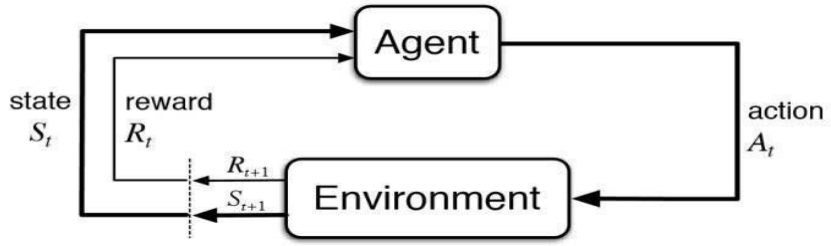
Natural Language Processing: Language Models, Text Classification, Information Retrieval, Information Extraction.

Reinforcement Learning

Reinforcement Learning is a part of machine learning.
Reinforcement learning (RL) can be viewed as an approach which falls between *supervised* and *unsupervised* learning. It is not strictly supervised as it does not rely only on a set of labelled training data but is not unsupervised learning because we have a reward which we want our agent to maximise. The agent needs to find the "right" actions to take in different situations to achieve its overall goal.

Reinforcement learning is the science of decision making.
Reinforcement learning involves no supervisor and only a *reward signal* is used for an agent to determine if they are doing well or not. *Time* is a key component in RL where the process is *sequential* with *delayed feedback*. Each *action* the agent makes affects the next data it receives.

Here, agents are self-trained on reward and punishment mechanisms. It's about taking the best possible action or path to gain maximum rewards and minimum punishment through observations in a specific situation. It acts as a signal to positive and negative behaviors. Essentially an agent (or several) is built that can perceive and interpret the environment in whichis placed, furthermore, it can take actions and interact with it.

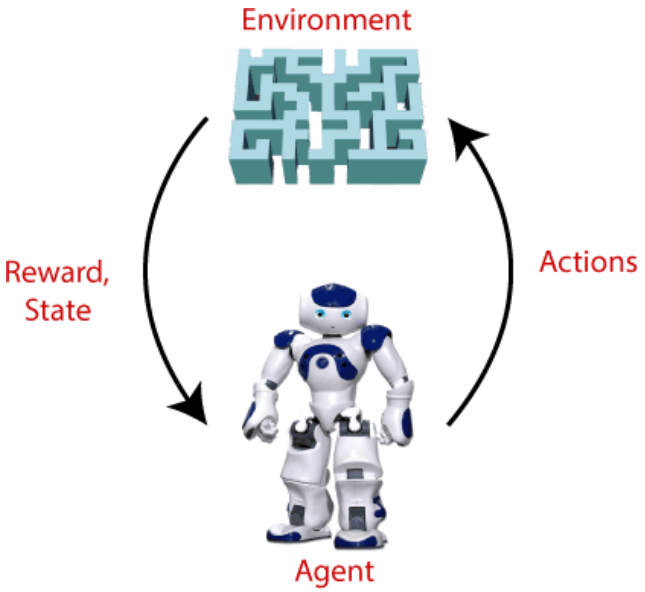


Basic Diagram of Reinforcement Learning

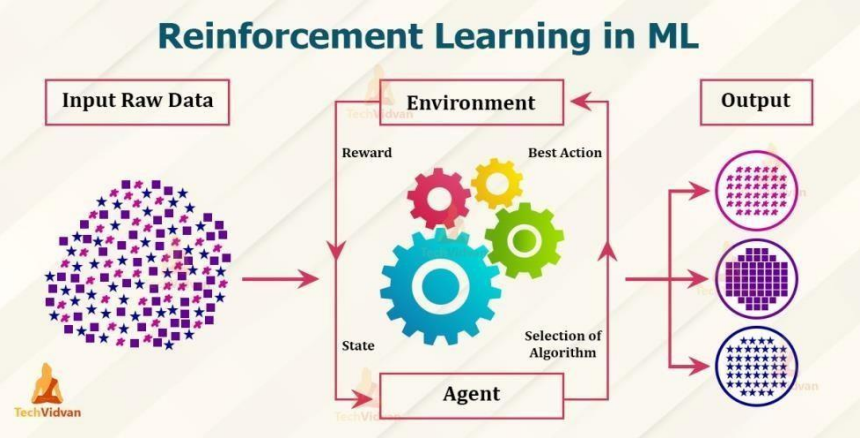
It is a type of machine learning technique where a computer agent learns to perform a task through repeated trial and error interactions with a dynamic environment. This learning approach enables the agent to make a series of decisions that maximize a reward metric for the task without human intervention and without being explicitly programmed to achieve the task.

Reinforcement Learning is a feedback-based Machine learning technique in which an agent learns tobehave in an environment by performing the actions and seeing the results of actions. For eachgood action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.

- In Reinforcement Learning, the agent learns automatically using feedbacks without any labeled data, unlike supervised learning.
- Since there is no labeled data, so the agent is bound to learn by its experience only.
- RL solves a specific type of problem where decision making is sequential, and the goal is long-term, such as **game-playing, robotics**, etc.
- The agent interacts with the environment and explores it by itself. The primary goal of an agent in reinforcement learning is to improve the performance by getting the maximum positive rewards.
- The agent learns with the process of hit and trial, and based on the experience, it learns to perform the task in a better way. Hence, we can say that ***"Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that."*** How a Robotic dog learns the movement of his arms is an example of Reinforcement learning.
- It is a core part of Artificial intelligence, and all AI agent works on the concept of reinforcement learning. Here we do not need to pre-program the agent, as it learns from its own experience without any human intervention.
- **Example:** Suppose there is an AI agent present within a maze environment, and his goal is to find the diamond. The agent interacts with the environment by performing some actions, and based on those actions, the state of the agent gets changed, and it also receives a reward or penalty as feedback.
- The agent continues doing these three things (**take action, change state/remain in the same state, and get feedback**), and by doing these actions, he learns and explores the environment.
- The agent learns that what actions lead to positive feedback or rewards and what actions lead to negative feedback penalty. As a positive reward, the agent gets a positive point, and as a penalty, it gets a negative point.



Terminologies used in Reinforcement Learning



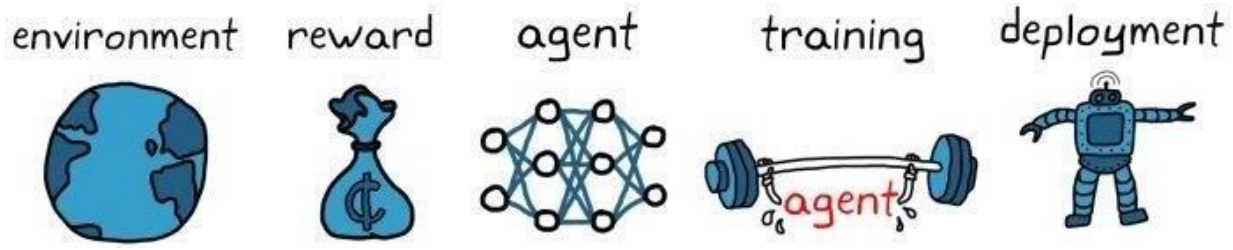
Terms used in Reinforcement Learning

- **Agent():** An entity that can perceive/explore the environment and act upon it. Is the sole decision-maker and learner
- **Environment():** A situation in which an agent is present or surrounded by. In RL, we assume thestochastic environment, which means it is random in nature.
- **Action():** Actions are the moves taken by an agent within the environment. a list of action which anagent can perform
- **State():** State is a situation returned by the environment after each action taken by the agent. It is the current situation of the agent in the environment.
- **Reward():** A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy():** Policy is a strategy applied by the agent for the next action based on the current state. – the agent prepares strategy(decision-making) to map situations to actions
- **Value():** It is expected long-term retuned with the discount factor and opposite to the short-term reward. The value of state shows up the reward achieved starting from the state until the policy is executed.
- **Q-value():** It is mostly similar to the value, but it takes one additional parameter as a current action (a).

Key Features of Reinforcement Learning

- In RL, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

Reinforcement Learning Workflow



Reinforcement Learning Workflow:
– Create the Environment
– Define the reward
– Create the agent
– Train and validate the agent
– Deploy the policy

Approaches to implement Reinforcement Learning

There are mainly three ways to implement reinforcement-learning in ML, which are:

1. **Value-based:**
   The value-based approach is about to find the optimal value function, which is the maximum value at a state under any policy. Therefore, the agent expects the long-term return at any state(s) under policy π.
2. **Policy-based:**
   Policy-based approach is to find the optimal policy for the maximum future rewards without using the value function. In this approach, the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward. The policy-based approach has mainly two types of policy:
   - **Deterministic:** The same action is produced by the policy (π) at any state.
   - **Stochastic:** In this policy, probability determines the produced action.
3. **Model-based:** In the model-based approach, a virtual model is created for the environment, and the agent explores that environment to learn it. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

Elements of Reinforcement Learning

There are four main elements of Reinforcement Learning, which are given below:

1. Policy
2. Reward Signal
3. Value Function
4. Model of the environment

1) Policy: A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states. A policy is the core element of the RL as it alone can define the behavior of the agent. In some cases, it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation asa search process. It could be deterministic or a stochastic policy:

For deterministic policy: a = π(s)
For stochastic policy: π(a | s) = P[At =a | St = s]

2) Reward Signal: The goal of reinforcement learning is defined by the reward signal. At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a **reward signal**. These rewards are given according to the good and bad actions taken by the agent. The agent's main objective is to maximize the total number of rewards for good actions. The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

3) Value Function: The value function gives information about how good the situation and action are and how much reward an agent can expect. A reward indicates the **immediate signal for each good and bad action**, whereas a value function specifies **the good state and action for the future**. The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

4) Model: The last element of reinforcement learning is the model, which mimics the behavior of the environment. With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state and reward.

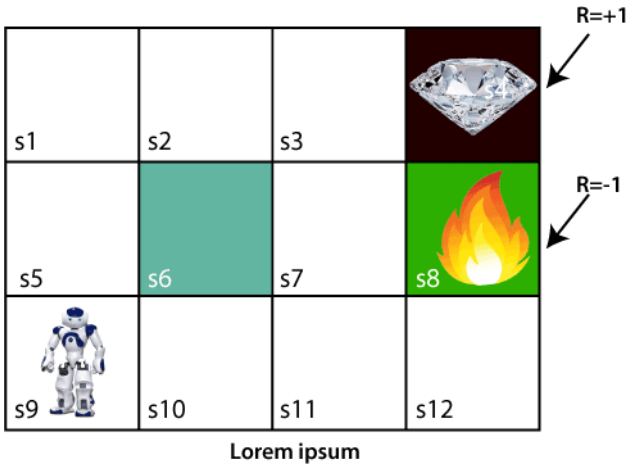
The model is used for planning, which means it provides a way to take a course of action by considering all future situations before actually experiencing those situations. The approaches for solving the RL problems **with the help of the model** are termed as the **model-based approach**. Comparatively, an approach **without using a model** is called a **model-free approach**.

Reinforcement Learning Working Nature

To understand the working process of the RL, we need to consider two main things:

- **Environment:** It can be anything such as a room, maze, football ground, etc.
- **Agent:** An intelligent agent such as AI robot.

Let's take an example of a maze environment that the agent needs to explore. Consider the belowimage:



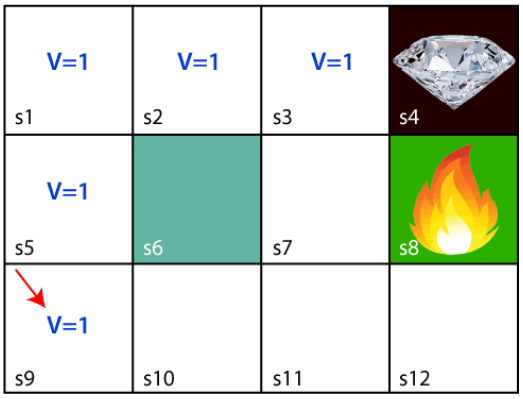
Lorem ipsum

In the above image, the agent is at the very first block of the maze. The maze is consisting of anS6 block, which is a **wall**, S8 a **fire pit**, and S4 a **diamond block.**

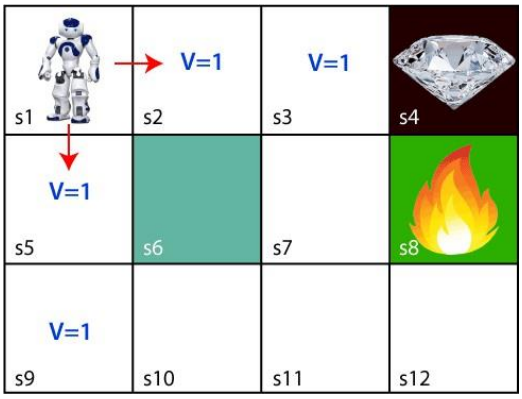
The agent cannot cross the S6 block, as it is a solid wall. If the agent reaches the S4 block, then get the **+1 reward;** if it reaches the fire pit, then gets **-1 reward point**. It can take four actions**: move up, move down, move left, and move right.**

The agent can take any path to reach to the final point, but he needs to make it in possible fewer steps. Suppose the agent considers the path **S9-S5-S1-S2-S3**, so he will get the +1-reward point.

The agent will try to remember the preceding steps that it has taken to reach the final step. To memorize the steps, it assigns 1 value to each previous step. Consider the below step:



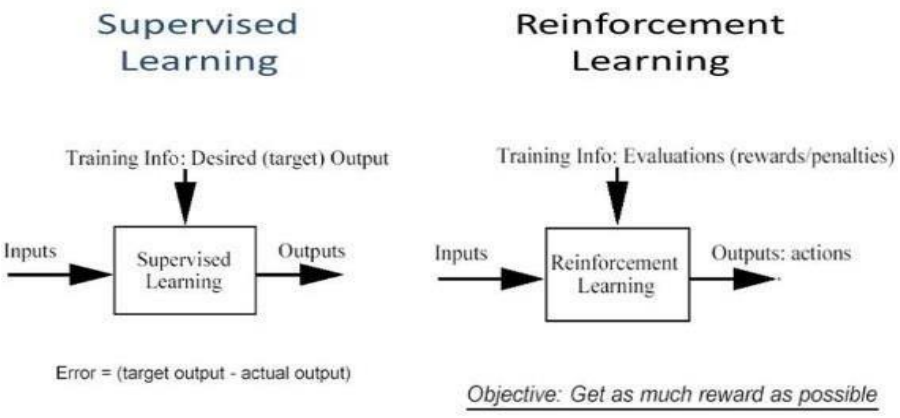
Now, the agent has successfully stored the previous steps assigning the 1 value to each previous block. But what will the agent do if he starts moving from the block, which has 1 value block on both sides? Consider the below diagram:



It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the above approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning.

Reinforcement learning different from supervised learning:

In supervised learning, the model is trained with a training dataset that has a correct answer key. The decision is doneon the initial input given as it has all the data that's required to train the machine. The decisions are independent of each other so each decision is represented through a label.
Example: Object Recognition

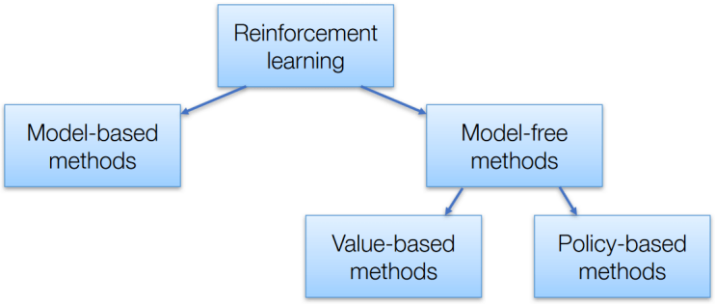


Difference between Supervised and Reinforcement Learning – pure study
In reinforcement learning, there isn't any answer and the reinforcement, agent decides what to be done to perform the required task. As the training dataset isn't available, the agent had to learn from its experience. It's all about compilingthe decisions in a sequential manner. To be said in simpler words, the output relies on the current input state and the next input relies on the output of the previous input. We give labels to the sequence of dependent decisions. Decisions are dependent. Example: Chess Game

Characteristics of Reinforcement Learning
– No supervision, only a real value or reward signal
– Decision making is sequential
– Time plays a major role in reinforcement problems
– Feedback isn't prompt but delayed

– The following data it receives is determined by the agent's actionsReinforcement Learning Algorithms

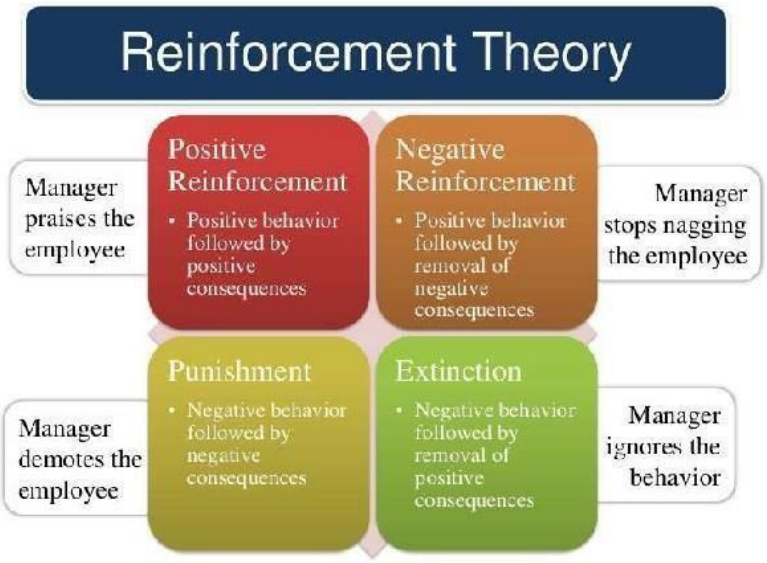
There are 3 approaches to implement reinforcement learning algorithms



Types of Reinforcement learning

There are mainly two types of reinforcement learning, which are:

- **Positive Reinforcement**
- **Negative Reinforcement**



Positive Reinforcement:
The positive reinforcement learning means adding something to increase the tendency that expected behavior would occur again. It impacts positively on the behavior of the agent and increases the strength of

the behavior.
This type of reinforcement can sustain the changes for a long time, but too much positivereinforcement may lead to an overload of states that can reduce the consequences.

Advantages

– Maximizes the performance of an action

– Sustain change for a longer period

Disadvantage

– Excess reinforcement can lead to an overload of states which would minimize the results.

Negative Reinforcement:

The negative reinforcement learning is opposite to the positive reinforcement as it increases thetendency that the specific behavior will occur again by avoiding the negative condition.
It can be more effective than the positive reinforcement depending on situation and behavior, butit provides reinforcement only to meet minimum behavior.

Advantages

– Maximized behavior

– Provide a decent to minimum standard of performance

Disadvantage

– It just limits itself enough to meet up a minimum behavior.

Active Vs Passive Reinforcement

Both active and passive reinforcement learning are types of RL. In case of passive RL, the agent's policy is fixed which means that it is told what to do. In contrast to this, in active RL, an agent needs to decide what to do as there's no fixed policy that it can act on. Therefore, the goal of a passive RL agent is to execute a fixed policy (sequence of actions) and evaluate it while that of an active RL agent is to act and learn an optimal policy.

Passive Learning

As the goal of the agent is to evaluate how good an optimal policy is, the agent needs to learn the expected utility $U\pi(s)$ for each states. This can be done in three ways.

Direct Utility Estimation

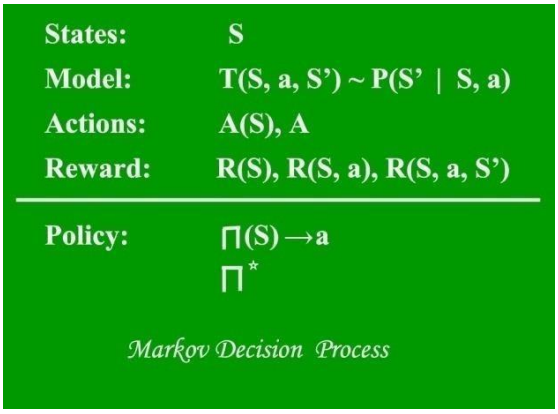
In this method, the agent executes a sequence of trials or runs (sequences of states-actions transitions that continue until the agent reaches the terminal state). Each trial gives a sample value and the agent estimates the utility based on the sample values. Can be calculated as running averages of sample values.

	Fixed Policy(Active)	**Policy not fixed(Passive)**
Model-free (real world)	Temporal Difference Learning (TD)	Q-learning
Model-based (simulation)	Adaptive Dynamic Programming(ADP)	ADP with proper exploration function

Passive Learning	Active Learning
Uses a large set of pre-labeled data to train the algorithm	Starts with a small set of labeled data and requests additional data from the user
The algorithm does not interact with the user	The algorithm interacts with the user to acquire additional data
It does not require user input after training is complete	May continue to request additional data until a satisfactory level of accuracy is achieved
Suitable for applications where a large dataset is available	Suitable for applications where labeled data is scarce or expensive to acquire

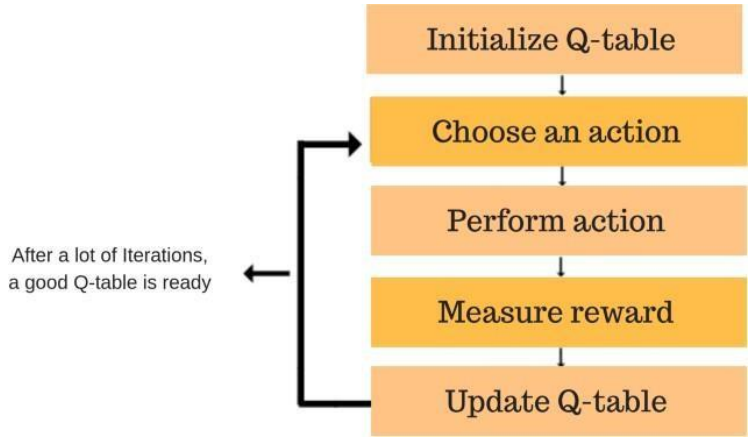
Widely used models for reinforcement learning

1. **Markov Decision Process (MDP's)** – are mathematical frameworks for mapping solutions in RL. The set of parameters that include Set of finite states – S, Set of possible Actions in each state – A, Reward – R, Model – T, Policy – π. The outcome of deploying an action to a state doesn't depend on previous actions or states but on currentaction and state.



Markov Decision Process

2. **Q Learning** – it's a value-based model free approach for supplying information to intimate which action an agent should perform. It revolves around the notion of updating Q values which shows the value of doing action A in state S.Value update rule is the main aspect of the Q-learning algorithm.



Q Learning

Practical Applications of reinforcement learning

– Robotics for Industrial Automation
– Text summarization engines, dialogue agents (text, speech), gameplays
– Autonomous Self Driving Cars
– Machine Learning and Data Processing
– Training system which would issue custom instructions and materials with respect to the requirements of students
– AI Toolkits, Manufacturing, Automotive, Healthcare, and Bots
– Aircraft Control and Robot Motion Control
– Building artificial intelligence for computer games

How to represent the agent state?

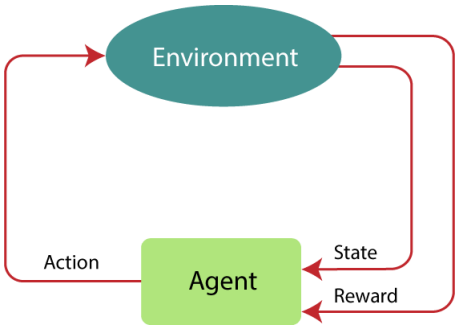
We can represent the agent state using the **Markov State** that contains all the required information from the history. The State St is Markov state if it follows the given condition:

$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, \ldots, S_t]$

The Markov state follows the **Markov property**, which says that the future is independent of the past and can only be defined with the present. The RL works on fully observable environments, where the agent can observe the environment and act for the new state. The complete process is known as Markov Decision process, which is explained below:

Markov Decision Process

Markov Decision Process or MDP, is used to **formalize the reinforcement learning problems**. If the environment is completely observable, then its dynamic can be modeled as a **Markov Process**.In MDP, the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.



MDP is used to describe the environment for the RL, and almost all the RL problem can beformalized using MDP.
MDP contains a tuple of four elements (S, A, P_a, R_a):

- A set of finite States S
- A set of finite Actions A
- Rewards received after transitioning from state S to state S', due to action a.
- Probability P_a.

MDP uses **Markov property**, and to better understand the MDP, we need to learn about it.

Markov Property:
It says that ***"If the agent is present in the current state S1, performs an action a1 and move to the state s2, then the state transition from s1 to s2 only depends on the current state and future action and states do not depend on past actions, rewards, or states."***
Or, in other words, as per Markov Property, the current state transition does not depend on any past action or state. Hence, MDP is an RL problem that satisfies the Markov property. Such as ina **Chess game, the players only focus on the current state and do not need to remember pastactions or states**.

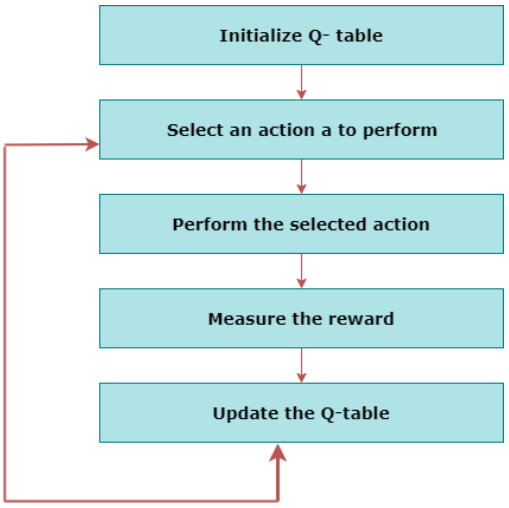
Finite MDP:
A finite MDP is when there are finite states, finite rewards, and finite actions. In RL, we consideronly the finite MDP.

Markov Process:
Markov Process is a memory less process with a sequence of random states S_1, S_2, ..., S_t that uses the Markov Property. Markov process is also known as Markov chain, which is a tuple (S, P) on state S and transition function P. These two components (S and P) can define the dynamics of the system.

Reinforcement Learning Algorithms

Reinforcement learning algorithms are mainly used in AI applications and gaming applications. The main used algorithms are:

- **Q-Learning:**
  - Q-learning is an **Off policy RL algorithm**, which is used for the temporal difference Learning. The temporal difference learning methods are the way of comparing temporally successive predictions.
  - It learns the value function Q (S, a), which means how good to take action "**a**" at a particular state "**s**."
  - The below flowchart explains the working of Q- learning:
  -



- **State Action Reward State action (SARSA):**
  - SARSA stands for **State Action Reward State action**, which is an **on-policy** temporal difference learning method. The on-policy control method selects the action for each state while learning using a specific policy.
  - The goal of SARSA is to calculate the **Q π (s, a) for the selected current policy π and all pairs of (s-a).**
  - The main difference between Q-learning and SARSA algorithms is that **unlike Q-learning, the maximum reward for the next state is not required for updating the Q-value in the table.**
  - In SARSA, new action and reward are selected using the same policy, which has determined the original action.
  - The SARSA is named because it uses the quintuple **Q(s, a, r, s', a').** Where,

    s: original state
    a: Original
    action
    r: reward observed while following the states
    s' and a': New state, action pair.
- **Deep Q Neural Network (DQN):**
  - As the name suggests, DQN is a **Q-learning using Neural networks**.
  - For a big state space environment, it will be a challenging and complex task to define and update a Q-table.
  - To solve such an issue, we can use a DQN algorithm. Where, instead of defining a Q-table, neural network approximates the Q-values for each action and state.

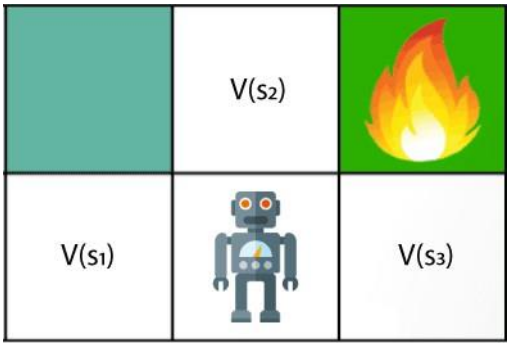
Now, we will expand the Q-learning.

Q-Learning Explanation:

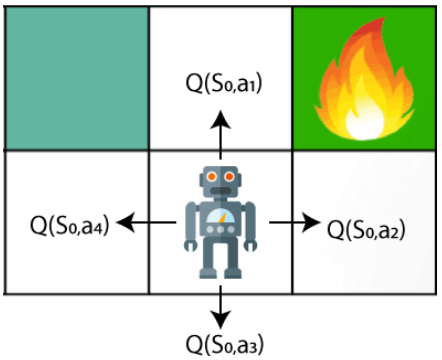
- Q-learning is a popular model-free reinforcement learning algorithm based on the Bellman equation.
- **The main objective of Q-learning is to learn the policy which can inform the agent that what actions should be taken for maximizing the reward under what circumstances.**
- It is an **off-policy RL** that attempts to find the best action to take at a current state.
- The goal of the agent in Q-learning is to maximize the value of Q.
- The value of Q-learning can be derived from the Bellman equation. Consider the Bellman equation given below:

$$V(s) = \max [R(s,a) + \gamma \sum_{s'} P(s,a,s') V(s`)]$$

In the equation, we have various components, including reward, discount factor (γ), probability, and end states s'. But there is no any Q-value is given so first consider the below image:



In the above image, we can see there is an agent who has three values options, V(s1), V(s2), V(s3).As this is MDP, so agent only cares for the current state and the future state. The agent can go to any direction (Up, Left, or Right), so he needs to decide where to go for the optimal path. Here agent will take a move as per probability bases and changes the state. But if we want some exact moves, so for this, we need to make some changes in terms of Q-value. Consider the below image:



Q- represents the quality of the actions at each state. So instead of using a value at each state, we will use a pair of state and action, i.e., Q(s, a). Q-value specifies that which action is more lubricativethan others, and according to the best Q-value, the agent takes his next move. The Bellman equation can be used for deriving the Q-value.

To perform any action, the agent will get a reward R(s, a), and also he will end up on a certainstate, so the Q -value equation will be:

$$Q(S, a)= R(s, a)+ \gamma\sum_{s'} P(s, a, s')V(s`)$$

Hence, we can say that, ***V(s) = max [Q(s, a)]***

$$Q(S, a)= R(s, a)+ \gamma\sum_{s'}(P(s, a, s')\max Q(s',a`))$$

The above formula is used to estimate the Q-values in Q-Learning.

What is 'Q' in Q-learning?

The Q stands for **quality** in **Q-learning**, which means it specifies the quality of an action taken bythe agent.

Q-table:

A Q-table or matrix is created while performing the Q-learning. The table follows the state and action pair, i.e., [s, a], and initializes the values to zero. After each action, the table is updated, and the q-values are stored within the table.

The RL agent uses this Q-table as a reference table to select the best action based on the q-values.

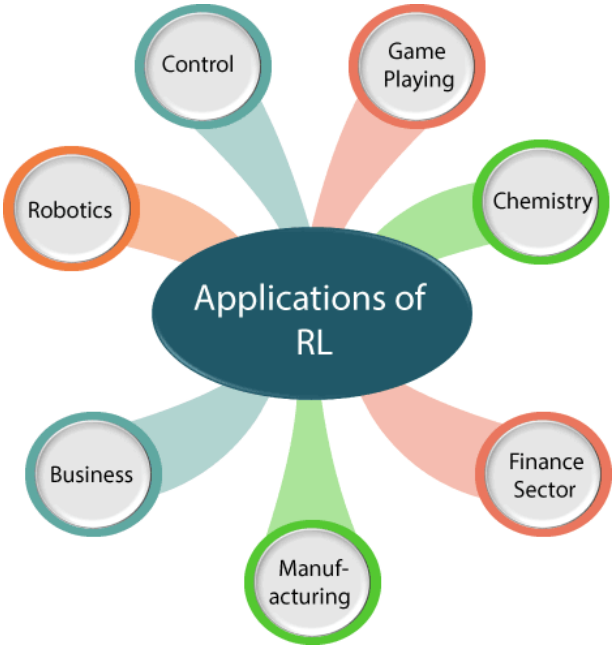
Difference between Reinforcement Learning and Supervised Learning

The Reinforcement Learning and Supervised Learning both are the part of machine learning, but both types of learnings are far opposite to each other. The RL agents interact with the environment, explore it, take action, and get rewarded. Whereas supervised learning algorithms learn from the labeled dataset and, on the basis of the training, predict the output.

The difference table between RL and Supervised learning is given below:

Reinforcement Learning	Supervised Learning
RL works by interacting with the environment.	Supervised learning works on the existing dataset.
The RL algorithm works like the human brain workswhen making some decisions.	Supervised Learning works as when a human learns things in the supervision of a guide.
There is no labeled dataset is present	The labeled dataset is present.
No previous training is provided to the learning agent.	Training is provided to the algorithm so that it can predict the output.
RL helps to take decisions sequentially.	In Supervised learning, decisions are made when input is given.

Reinforcement Learning Applications



1. **Robotics:**
   a. RL is used in **Robot navigation, Robo-soccer, walking, juggling**, etc.

Control :

. RL can be used for **adaptive control** such as Factory processes, admission control in telecommunication, and Helicopter pilot is an example of reinforcement learning.

Game Playing:

. RL can be used in **Game playing** such as tic-tac-toe, chess, etc.

Chemistry :

. RL can be used for optimizing the chemical reactions.

Business:

. RL is now used for business strategy planning.

Manufacturing:

. In various automobile manufacturing companies, the robots use deep reinforcement learning to pick goods and put them in some containers.

Finance Sector:

. The RL is currently used in the finance sector for evaluating trading strategies.

GENERALIZATION IN REINFORCEMENT LEARNING:

In Reinforcement learning, the generalization of the **agents is benchmarked on the environments they have been trained on**. In a supervised learning setting, this would mean testing the model using the training dataset.

APPLICATIONS OF RL NATURAL LANGUAGE PROCESSING:

Artificial intelligence has been improved tremendously without needing to change the underlying hardware infrastructure. Users can run an Artificial intelligence program in an old computer system. On the other hand, the beneficiary effect of machine learning is unlimited. Natural Language Processing is one of the branches of AI that gives the machines the ability to read, understand, and deliver meaning. NLP has been very successful in healthcare, media, finance, and human resource.

The most common form of unstructured data is texts and speeches. It's plenty but hard to extract useful information. Ifnot, it would take a long time to mine the information. Written text and speech contain rich information. It's because we, as intelligent beings, use writing and speaking as the primary form of communication. NLP can analyze these data for us and do the task like sentiment analysis, cognitive assistant, span filtering, identifying fake news, and real-time language translation.

NLP has widely used in cars, smartphones, speakers, computers, websites, etc. Google Translator usage machine translator which is the NLP system. Google Translator wrote and spoken natural language to desire language users want to translate. NLP helps google translator to understand the word in context, remove extra noises, and build CNN to understand native voice.

NLP is also popular in chatbots. Chatbots is very useful because it reduces the human work of asking what customer needs. NLP chatbot cans ask sequential questions like what the user problem is and where to find the solution. Apple and AMAZON have a robust chatbot in their system. When the user asks some questions, the chatbot converts them into understandable phrases in the internal system.

It's call toke. Then token goes into NLP to get the idea of what users are asking. NLP is used in information retrieval (IR). IR is a software program that deals with large storage, evaluation of information from large text documents from repositories. It will retrieve only relevant information. For example, it is used in google voice detection to trim unnecessary words.

Application of NLP

- Machine Translation i.e. Google Translator
- Information retrieval
- Question Answering i.e. ChatBot
- Summarization
- Sentiment Analysis
- Social Media Analysis
- Mining large data\

Words and Sequences

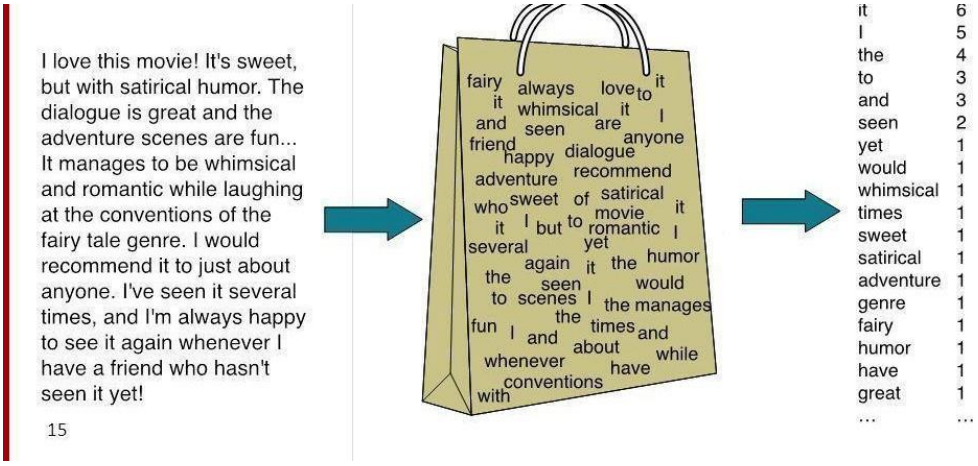
NLP system needs to understand text, sign, and semantic properly. Many methods help the NLP system to understand text and symbols. They are text classification, vector semantic, word embedding, probabilistic language model, sequence labeling, and speech reorganization.

1. *Text classification*

   Text clarification is the process of categorizing the text into a group of words. By using NLP, text classification can automatically analyze text and then assign a set of predefined tags or categories based on its context. NLP is used for sentiment analysis, topic detection, and language detection. There is mainly three textclassification approach-

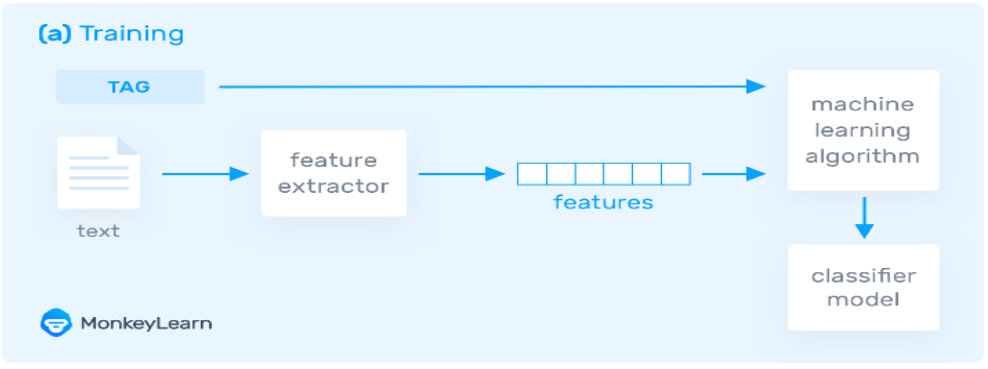
   - Rule-based System,
   - Machine System
   - Hybrid System.

   In the rule-based approach, texts are separated into an organized group using a set of handicraft linguistic rules. Those handicraft linguistic rules contain users to define a list of words that are characterized by groups. For example, words like Donald Trump and Boris Johnson would be categorized into politics. People like LeBron James and Ronaldo would be categorized into sports.
   Machine-based classifier learns to make a classification based on past observation from the data sets. User data is prelabeled as tarin and test data. It collects the classification strategy from the previous inputs and learns continuously. Machine-based classifier usage a bag of a word for feature extension.



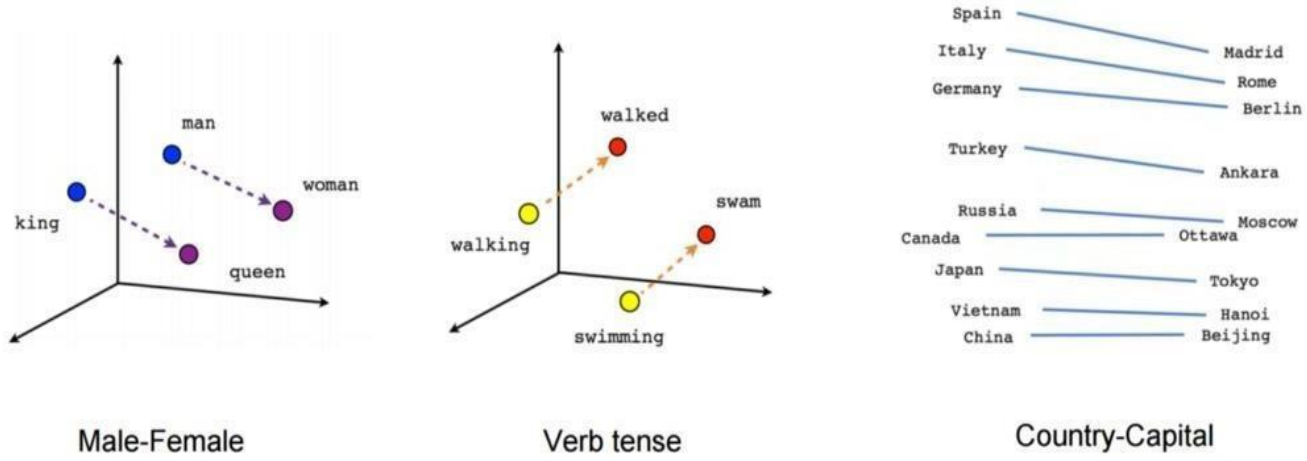
Source

In a bag of words, a vector represents the frequency of words in a predefined dictionary of a word list. We canperform NLP using the following machine learning algorithms: Naïve Bayer, SVM, and Deep Learning.



The third approach to text classification is the Hybrid Approach. Hybrid approach usage combines a rule- based and machine Based approach. Hybrid based approach usage of the rule-based system to create a tag and use machine learning to train the system and create a rule. Then the machine-based rule list is compared with the rule-based rule list. If something does not match on the tags, humans improve the list manually. It is the best method to implement text classification

2. *Vector Semantic*
   Vector Semantic is another way of word and sequence analysis. Vector semantic defines semantic and interprets words meaning to explain features such as similar words and opposite words. The main idea behind vector semantic is two words are alike if they have used in a similar context. Vector semantic divide the words in a multi-dimensional vector space. Vector semantic is useful in sentiment analysis.

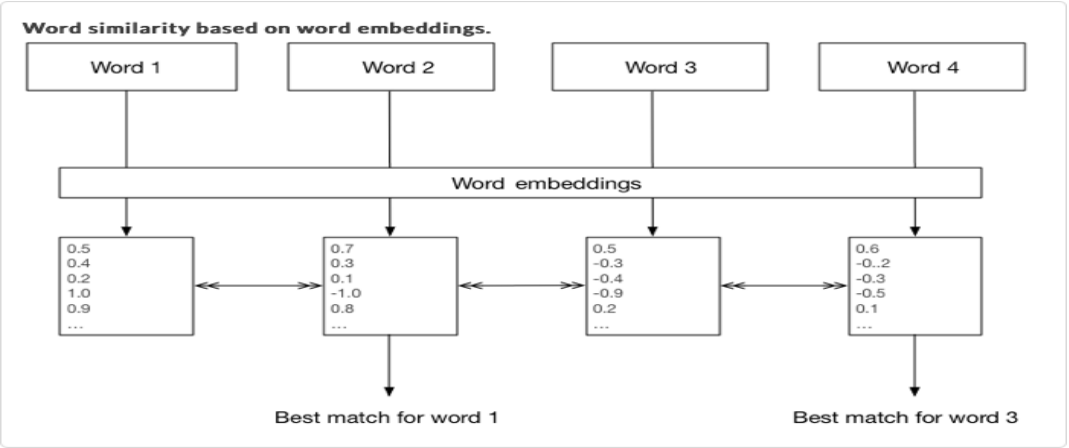


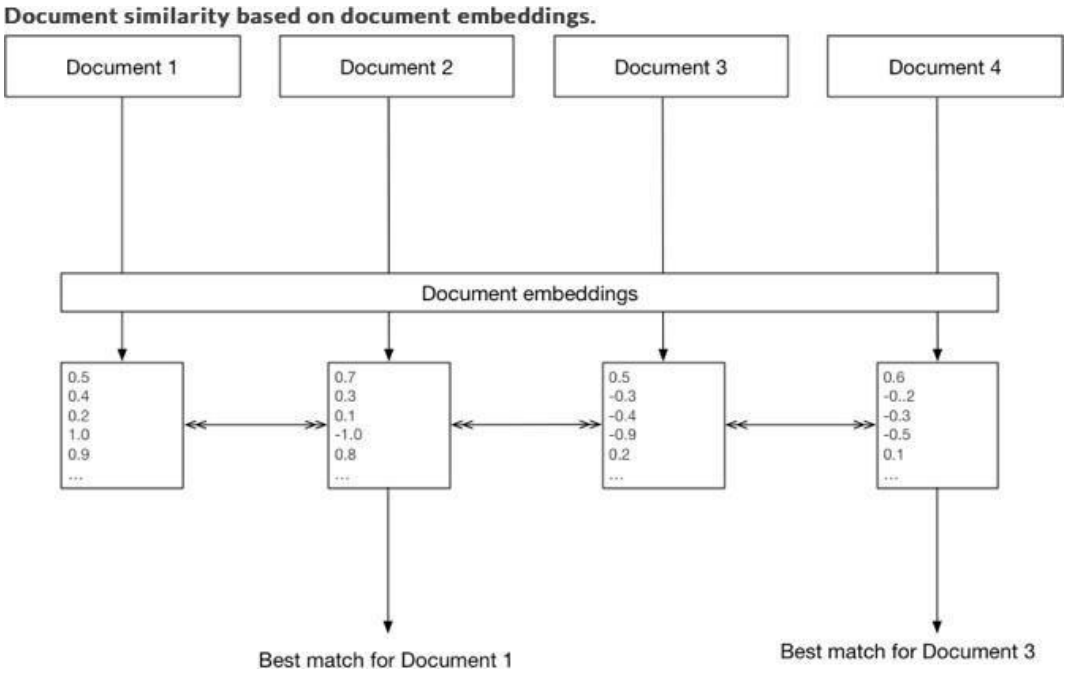
Source

3. *Word Embedding*
   Word embedding is another method of word and sequence analysis. Embedding translates spares vectors into a low-dimensional space that preserves semantic relationships. Word embedding is a type of word representation that allows words with similar meaning to have a similar representation. There are two types of word embedding-

- Word2vec
- Doc2Vec.

Word2Vec is a statistical method for effectively learning a standalone word embedding from a text corpus.





Doc2Vec is similar to Word2Vec, but it analyzes a group of text like pages.

4. *Probabilistic Language Model*

Another approach to word and sequence analysis is the probabilistic language model. The goal of the probabilistic language model is to calculate the probability of a sentence of a sequence of words. For example,the probability of the word “a” occurring in a given word “to” is 0.00013131 percent.

```
In [48]: word_1 = 'to'
         word_2 = 'a'

         prob_word_1 = word_list[word_list['words'] == word_1]['prob'].iloc[0]
         prob_word_2 = word_list[word_list['words'] == word_2]['prob'].iloc[0]

         unigram_prob = prob_word_1*prob_word_2

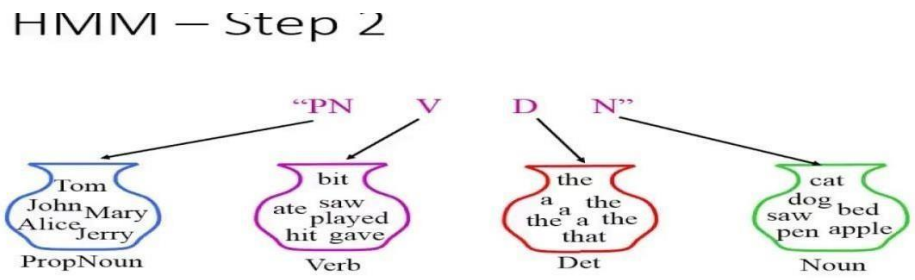
         print('The unigram probability of the word "a" occuring given the word "to" was
          the previous word is: ', np.round(unigram_prob,10))

         The unigram probability of the word "a" occuring given the word "to" was the pr
         evious word is:  0.0005955004
```

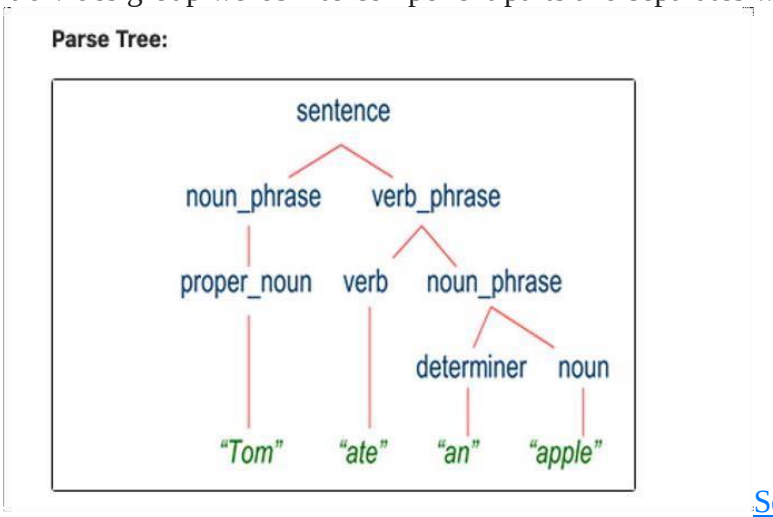
5. *Sequence Labeling*

Sequence labeling is a typical NLP task that assigns a class or label to each token in a given input sequence. If someone says “play the movie by tom hanks”. In sequence, labeling will be [play, movie, tom hanks]. Play determines an action. Movies are an instance of action. Tom Hanks goes for a search entity. It divides the input into multiple tokens and uses LSTM to analyze it. There are two forms of sequence labeling. They are token labeling and span labeling.

apple. The best example is Amazon Alexa. , noun ate, determiner tom, verb Parsing is a phase of NLP where the parser determines the syntactic structure of a text by analyzing itsconstituent words based on an underlying grammar. For example, “tom ate an apple” will be divided into proper noun



Parsing
Parser determines the syntactic structure of a text by analyzing its constituent words based on an underlying grammar. It divides group words into component parts and separates words.

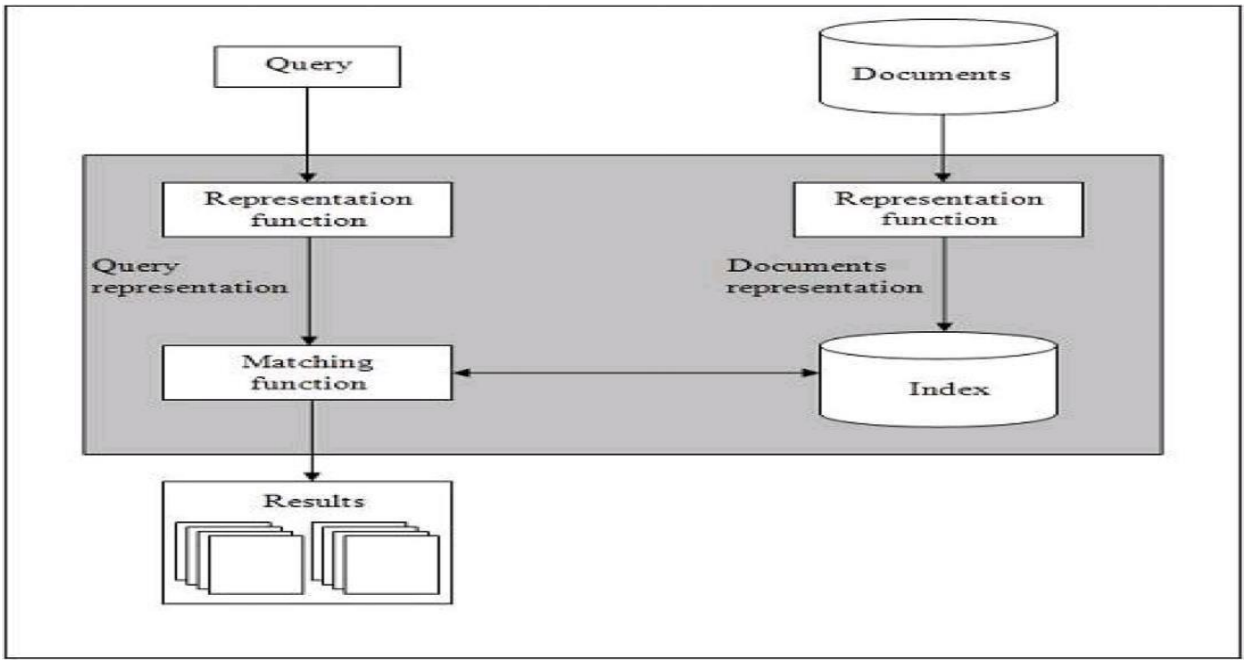


Source

Semantic
Text is at the heart of how we communicate. What is really difficult is understanding what is beingsaid in written or spoken conversation? Understanding lengthy articles and books are even more difficult.
Semantic is a process that seeks to understand linguistic meaning by constructing a model of the principle that the speaker uses to convey meaning. It's has been used in customer feedback analysis, article analysis, fake news detection, Semantic analysis, etc.

Information Retrieval, Information Extraction

Information retrieval is defined as the process of accessing and retrieving the most appropriate information from text based on a particular query given by the user, with the help of context-based indexing or metadata. **Google Search** is the most famous example of information retrieval.
Now let's discuss what are Information Retrieval Systems?
An information retrieval system searches a collection of natural language documents with the goal of retrieving exactly the set of documents that matches a user's question. They have their origin in library systems.
These systems assist users in finding the information they require but it does not attempt to deduce or generate answers. It tells about the existence and location of documents that might consist of the required information that is given to the user. The documents that satisfy the user's requirement are called relevant documents. If we have aperfect IR system, then it will retrieve only relevant documents.
Basics of IR Systems



The step by step procedure of these systems are as follows:

- Indexing the collection of documents.
- Transforming the query in the same way as the document content is represented.
- Comparing the description of each document with that of the query.
- Listing the results in order of relevancy.

Retrieval Systems consist of mainly two processes:

- Indexing
- Matching

Indexing

It is the process of selecting terms to represent a text.Indexing involves:

- Tokenization of string
- Removing frequent words
- Stemming

Two common Indexing Techniques:

- Boolean Model
- Vector space model

Matching

It is the process of finding a measure of similarity between two text representations.The relevance of a document is computed based on the following parameters:

1. TF: It stands for Term Frequency which is simply the number of times a given termappears in that document.

TF (i, j) = (count of ith term in jth document)/(total terms in jth document)

2. IDF: It stands for Inverse Document Frequency which is a measure of the generalimportance of the term.

IDF (i) = (total no. of documents)/(no. of documents containing ith term)

3. TF-IDF Score (i, j) = TF * IDF

Information Retrieval Models

Information retrieval models predict and explain what a user will find in relevance to thegiven query. These are basically a pattern that defines the above-mentioned aspects ofretrieval procedure that we discussed in ad-hoc retrieval and consists of the following:

- A model for documents.
- A model for queries.
- A matching function that compares queries to documents. Mathematically, a retrieval model consists of the following components:

- **D:** Representation for documents.
- **R:** Representation for queries.
- **F:** The modeling framework for D, Q along with the relationship between them.
- **R (q, di):** A ranking or similarity function that orders the documents with respect to thequery.

Types of IR Model

The following are three models that are classified for the Information model (IR) model:

Classical IR Models

These are the simplest and easy-to-implement IR models. These are based onmathematical knowledge that was easily recognized and understood as well.

Following are the examples of classical IR models:

- Boolean models,
- Vector models,
- Probabilistic models.

Non-Classical IR Models

These are completely opposite to the classical IR models. These are based onprinciples other than similarity, probability, Boolean operations.

Following are the examples of Non-classical IR models:

- Information logic models,
- Situation theory models,
- Interaction models.

Alternative IR Models

It is the enhancement of the classical IR model that makes use of some specifictechniques from some other fields.

Following are the examples of Alternative IR models:

- Cluster models,
- Fuzzy models,
- Latent Semantic Indexing (LSI) models.

Unit-IV: **Natural Language for Communication**: Phrase structure grammars, Syntactic Analysis, Augmented Grammars and semantic Interpretation, Machine Translation, Speech Recognition
Perception: Image Formation, Early Image Processing Operations, Object Recognition by appearance, Reconstructing the 3D World, Object Recognition from Structural information, Using Vision.

Computers don't speak languages the way humans do. They communicate in machine code or machine language, while we speak English, Dutch, French or some other human language. Most of us don't understand the millions of zeros and ones computers communicate in. And in turn, computers don't understand human language unless they are programmed to do so. That's where natural language processing (NLP) comes in.

What is natural language processing?
Natural language processing is a form of artificial intelligence (AI) that gives computers the ability to read, understand and interpret human language. It helps computers measure sentiment and determine which parts of human language are important. For computers, this is an extremely difficult thing to do because of the large amount of unstructured data, the lack of formal rules and the absence of real-world context or intent.

Language is a method of communication with the help of which we can speak, read and write. Natural Language Processing (NLP) is a subfield of Computer Science that deals with Artificial Intelligence (AI), which enables computers to understand and process human language.

Study of Human Languages

Language is a crucial component for human lives and also the most fundamental aspect of our behavior. We can experience it in mainly two forms - written and spoken. In the written form, it is a way to pass our knowledge from one generation to the next. In the spoken form, it is the primary medium for human beings to coordinate with each other in their day-to-day behavior. Language is studied in various academic disciplines. Each discipline comes with its own set of problems and a set of solution to address those.

Consider the following table to understand this −

Discipline	Problems	Tools
Linguists	How phrases and sentences can be formed with words?	Intuitions about well-formedness and meaning. Mathematical model of structure. For example,

	What curbs the possible meaningfor a sentence?	model theoretic semantics, formal language theory.
Psycholinguists	How human beings can identify the structure of sentences? How the meaning of words can be identified? When does understanding take place?	Experimental techniques mainly for measuring the performance of human beings. Statistical analysis of observations.
Philosophers	How do words and sentences acquire the meaning? How the objects are identified by the words? What is meaning?	Natural language argumentation by using intuition. Mathematical models like logic and model theory.
Computational Linguists	How can we identify the structure of a sentence How knowledge and reasoning can be modeled? How we can use language to accomplish specific tasks?	Algorithms Data structures Formal models of representation and reasoning. AI techniques like search & representation methods.

Ambiguity and Uncertainty in Language

Ambiguity, generally used in natural language processing, can be referred as the ability of being understood in more than one way. In simple terms, we can say that ambiguity is the capability of being understood in more than one way. Natural language is very ambiguous.

Components of Language

The language of study is divided into the interrelated components, which are conventional as well as arbitrary divisions of linguistic investigation. The explanation of these components is as follows −

Phonology

The very first component of language is phonology. It is the study of the speech sounds of a particular language. The origin of the word can be traced to Greek language, where ‘phone’ means sound or voice. Phonetics, a subdivision of phonology is the study of the speech sounds of human language from the perspective of their production, perception or their physical properties. IPA (International Phonetic Alphabet) is a tool that represents human sounds in a regular way while studying phonology. In IPA, every written symbol represents one and only one speech sound and vice-versa.

Phonemes

It may be defined as one of the units of sound that differentiate one word from other in a language. In linguistic, phonemes are written between slashes. For example, phoneme **/k/** occurs in the words such as kit, skit.

Morphology

It is the second component of language. It is the study of the structure and classification of the words in a particular language. The origin of the word is from Greek language, where the word 'morphe' means 'form'. Morphology considers the principles of formation of words in a language. In other words, how sounds combine into meaningful units like prefixes, suffixes and roots. It also considers how words can be grouped into parts of speech.

Lexeme

In linguistics, the abstract unit of morphological analysis that corresponds to a set of forms taken by a single word is called lexeme. The way in which a lexeme is used in a sentence is determined by its grammatical category. Lexeme can be individual word or multiword. For example, the word talk is an example of an individual word lexeme, which may have many grammatical variants like talks, talked and talking. Multiword lexeme can be made up of more than one orthographic word. For example, speak up, pull through, etc. are the examples of multiword lexemes.

Syntax

It is the third component of language. It is the study of the order and arrangement of the words into larger units. The word can be traced to Greek language, where the word suntassein means 'to put in order'. It studies the type of sentences and their structure, of clauses, of phrases.

Semantics

It is the fourth component of language. It is the study of how meaning is conveyed. The meaning can be related to the outside world or can be related to the grammar of the sentence. The word can be traced to Greek language, where the word semainein means means 'to signify', 'show', 'signal'.

Pragmatics:

It is the fifth component of language. It is the study of the functions of the language and its use in context. The origin of the word can be traced to Greek language where the word 'pragma' means 'deed', 'affair'

Spoken Language Syntax

The written English and spoken English grammar have many common features but along with that, they also differ in a number of aspects. The following features distinguish between the spoken and written English grammar −

Disfluencies and Repair

This striking feature makes spoken and written English grammar different from each other. It is individually known as phenomena of disfluencies and collectively as phenomena of repair. Disfluencies include the use of following −

- **Fillers words** − Sometimes in between the sentence, we use some filler words. They are called fillers of filler pause. Examples of such words are uh and um.
- **Reparandum and repair** − The repeated segment of words in between the sentence is called reparandum. In the same segment, the changed word is called repair. Consider the following example to understand this −

Does ABC airlines offer any one-way flights uh one-way fares for 5000 rupees?

In the above sentence, one-way flight is a reparandum and one-way flights is a repair.

Word Fragments

Sometimes we speak the sentences with smaller fragments of words. For example, **wwha-what is the time?** Here the words **w-wha** are word fragments.

NATURAL LANGUAGE GRAMMAR

For linguistics, language is a group of arbitrary vocal signs. We may say that language is creative, governed by rules, innate as well as universal at the same time. On the other hand, it is humanly too. The nature of the language is different for different people. There is a lot of misconception about the nature of the language. That is why it is very important to understand the meaning of the ambiguous term ***'grammar'***. In linguistics, the term grammar may be defined as the rules or principles with the help of which language works. In broad sense, we can divide grammar in two categories −

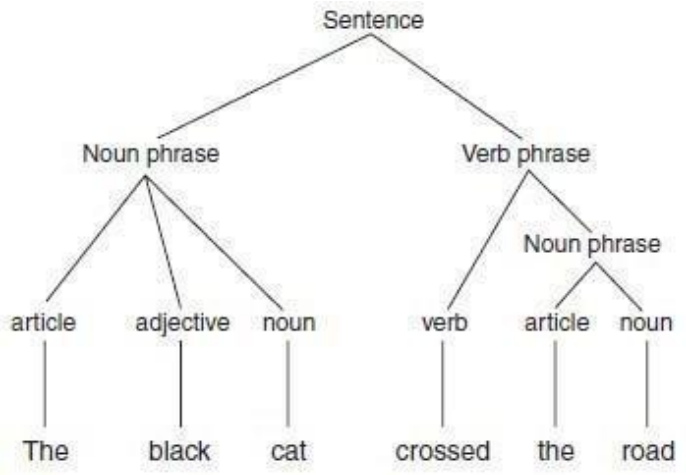
Descriptive Grammar

The set of rules, where linguistics and grammarians formulate the speaker's grammar is called descriptive grammar.

Perspective Grammar

It is a very different sense of grammar, which attempts to maintain a standard of correctness in the language. This category has little to do with the actual working of the language.

Grammatical Categories



A grammatical category may be defined as a class of units or features within the grammar of a language. These units are the building blocks of language and share a common set of characteristics. Grammatical categories are also called grammatical features.

In recent years, AI has evolved rapidly, and with that, NLP got more sophisticated, too. Many of us already use NLP daily without realizing it. You've probably used at least one of the following tools:

- Spell checker.
- Autocomplete.
- Spam filters.
- Voice text messaging.

GENERATIVE CAPACITY

Grammatical formalisms can be classified by their **generative capacity**: the set of languages they can represent. Chomsky (1957) describes four classes of grammatical formalisms that differ only in the form of the rewrite rules. Theclasses can be arranged in a hierarchy, where each class can be used to describe all the languages that can be describedby a less powerful class, as well as some additional languages. Here we list the hierarchy, most powerful class first:

Recursively enumerable grammars use unrestricted rules: both sides of the rewrite rules can have any number of terminal and nonterminal symbols, as in the rule A B C → D E. These grammars are equivalent to Turing machines in their expressive power.

Context-sensitive grammars are restricted only in that the right-hand side must contain at least as many symbols as the left-hand side. The name "contextsensitive" comes from the fact that a rule such as A X B → A Y B says that
an X can be rewritten as a Y in the context of a preceding A and a following B. Context-sensitive grammars can represent languages such as anbncn (a sequence of n copies of a followed by the same number of bs and then cs).

In **context-free grammars** (or **CFG**s), the left-hand side consists of a single nonterminal symbol. Thus, each rulelicenses rewriting the nonterminal as the right-hand side in *any* context. CFGs are popular for natural-language and programming-language grammars, although it is now widely accepted that at least some natural languages haveconstructions that are not context-free (Pullum, 1991). Context-free grammars can represent anbn, but not anbncn.

Regular grammars are the most restricted class. Every rule has a single nonterminal on the left-hand side and a terminal symbol optionally followed by a nonterminal on the right-hand side. Regular grammars are equivalent in power to finitestate machines. They are poorly suited for programming languages, because they cannot represent constructs such as balanced opening and closing parentheses (a variation of the anbn language). The closest they can come is representing a $*$ b $*$, a sequence of any number of as followed by any number of bs.

The grammars higher up in the hierarchy have more expressive power, but the algorithms for dealing with them areless efficient. Up to the 1980s, linguists focused on context-free and context-sensitive languages.

There have been many competing language models based on the idea of phrase structure; we will describe a popularmodel called the **probabilistic context-free grammar**, or PROBABILISTIC CONTEXT-FREE GRAMMAR PCFG.

1 A **grammar** is a collection of GRAMMAR rules that defines a **language** as a set of allowable LANGUAGE strings of words.
"Context-free" and "probabilistic" means that the grammar assigns a probability to every string. Here is a PCFG rule:

VP → Verb [0.70]

| VP NP [0.30] .

Here VP (*verb phrase*) and NP (*noun phrase*) are **non-terminal symbols**. The grammar also refers to actual words,which are called **terminal symbols**. This rule is saying that with probability 0.70 a verb phrase consists solely of a verb, and with probability 0.30 it is a VP followed by an NP. Appendix B describes non-probabilistic context-free grammars.

Five basic NLP tasks

As we mentioned before, human language is extremely complex and diverse. That's why natural language processing includes many techniques to interpret it, ranging from statistical and machine learning methodsto rules-based and algorithmic approaches.

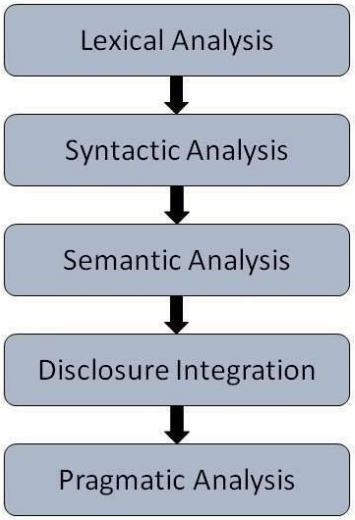
There are five basic NLP tasks that you might recognize from school.

Lexical Analysis − The first phase of NLP is the Lexical Analysis. This phase scans the source code as a stream of characters and converts it into meaningful lexemes. Lexical analysis divides the whole text into paragraphs, sentences, and words. It involves identifying and analyzing the structure of words. Lexicon of a language means the collection of words and phrases in a language.

Syntactic Analysis (Parsing) − Syntactic Analysis is used to check grammar, word arrangements, and shows the relationship among the words. It involves analysis of words in the sentence for grammar and arranging words in a manner that shows the relationship among the words. The sentence such as "The school goes toboy" is rejected by English syntactic analyzer.

Example: Agra goes to the Poonam

In the real world, Agra goes to the Poonam, does not make any sense, so this sentence is rejected by the Syntactic analyzer.



Semantic Analysis − Semantic analysis is concerned with the meaning representation. It mainly focuses on the literal meaning of words, phrases, and sentences.

It draws the exact meaning or the dictionary meaning from the text. The text is checked for meaningfulness. It is done by mapping syntactic structures and objects in the task domain. The semantic analyzer disregards sentence such as “hot ice-cream”.

Discourse Integration − Discourse Integration depends upon the sentences that proceeds it and alsoinvokes the meaning of the sentences that follow it.

- The meaning of any sentence depends upon the meaning of the sentence just before it. In addition,it also brings about the meaning of immediately succeeding sentence.

Pragmatic Analysis − Pragmatic is the fifth and last phase of NLP. It helps you to discover the intended effect by applying a set of rules that characterize cooperative dialogues.

For Example: "Open the door" is interpreted as a request instead of an order.

- During this, what was said is re-interpreted on what it actually meant. It involves deriving those aspects of language which require real world knowledge.

CONCEPT OF GRAMMAR

Grammar is very essential and important to describe the syntactic structure of well-formed programs. In the literary sense, they denote syntactical rules for conversation in natural languages. Linguistics have attempted to define grammars since the inception of natural languages like English, Hindi, etc.

The theory of formal languages is also applicable in the fields of Computer Science mainly in programming languages and data structure. For example, in ‘C’ language, the precise grammar rules state how functions are made from lists and statements.

A mathematical model of grammar was given by **Noam Chomsky** in 1956, which is effective for writing computer languages.

Mathematically, a grammar G can be formally written as a 4-tuple (N, T, S, P) where −

- **N** or **VN** = set of non-terminal symbols, i.e., variables.
- **T** or $\sum$ = set of terminal symbols.
- **S** = Start symbol where $S \in N$
- **P** denotes the Production rules for Terminals as well as Non-terminals. It has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \sum$ and least one symbol of α belongs to V_N

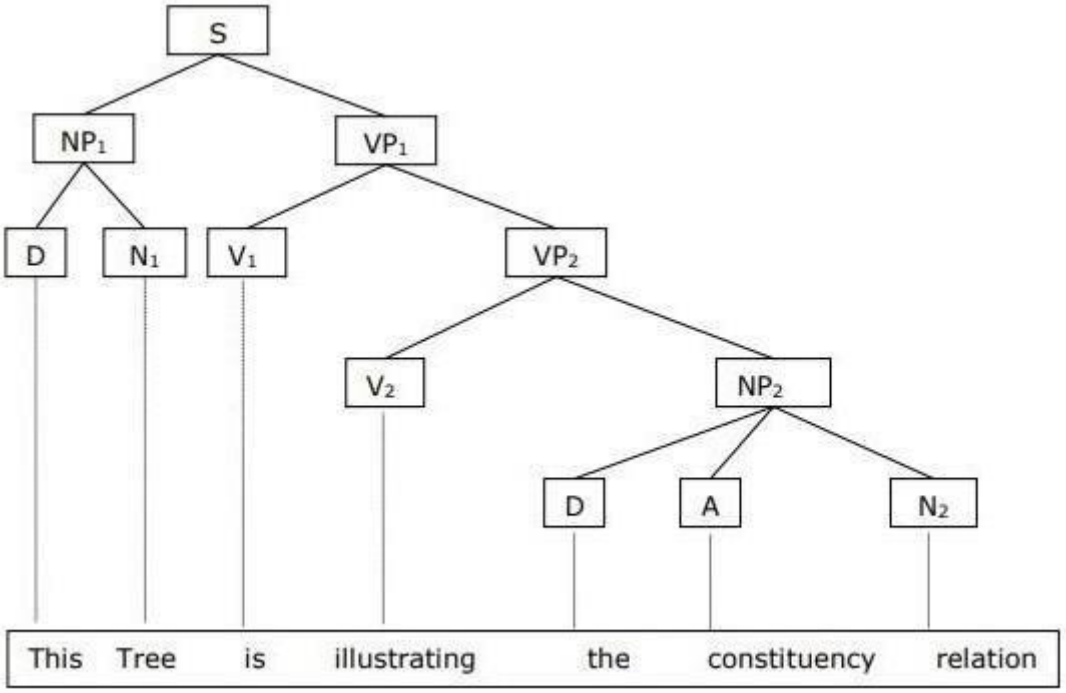
Phrase Structure or Constituency Grammar

Phrase structure grammar, introduced by Noam Chomsky, is based on the constituency relation. That is why it is also called constituency grammar. It is opposite to dependency grammar.

Example

Before giving an example of constituency grammar, we need to know the fundamental points about constituency grammar and constituency relation.

- All the related frameworks view the sentence structure in terms of constituency relation.
- The constituency relation is derived from the subject-predicate division of Latin as well as Greek grammar.
- The basic clause structure is understood in terms of **noun phrase NP** and **verb phrase VP**.We can write the sentence **"This tree is illustrating the constituency relation"** as follows −



Dependency Grammar

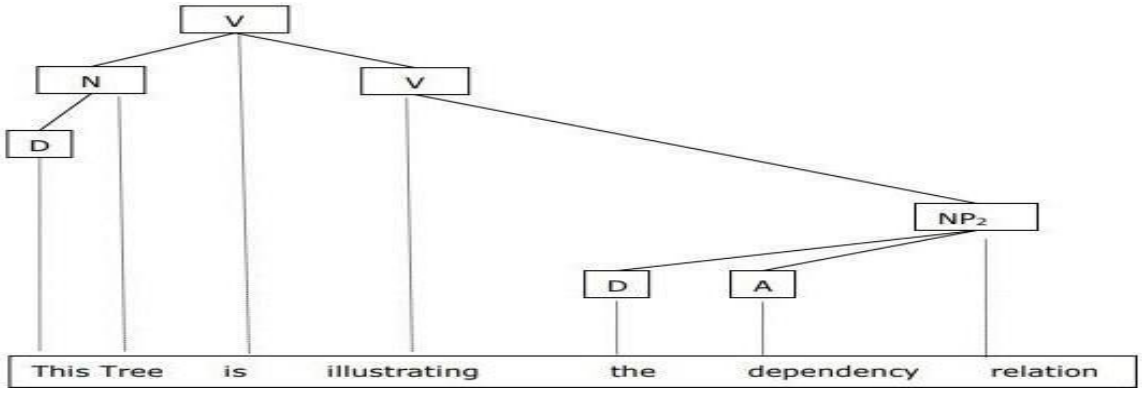
It is opposite to the constituency grammar and based on dependency relation. It was introduced by Lucien Tesniere. Dependency grammar (DG) is opposite to the constituency grammar because it lacks phrasal nodes.

Example

Before giving an example of Dependency grammar, we need to know the fundamental points about Dependency grammar and Dependency relation.

- In DG, the linguistic units, i.e., words are connected to each other by directed links.
- The verb becomes the center of the clause structure.
- Every other syntactic units are connected to the verb in terms of directed link. These syntactic units are called ***dependencies***.

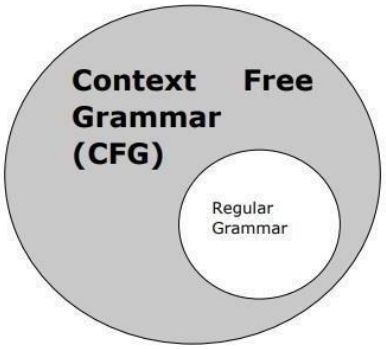
We can write the sentence **"This tree is illustrating the dependency relation"** as follows;



Parse tree that uses Constituency grammar is called constituency-based parse tree; and the parse trees that uses dependency grammar is called dependency-based parse tree.

Context Free Grammar

Context free grammar, also called CFG, is a notation for describing languages and a superset of Regular grammar. It can be seen in the following diagram −



Definition of CFG

CFG consists of finite set of grammar rules with the following four components −

Set of Non-terminals

It is denoted by V. The non-terminals are syntactic variables that denote the sets of strings, which further help defining the language, generated by the grammar.

Set of Terminals

It is also called tokens and defined by Σ. Strings are formed with the basic symbols of terminals.

Set of Productions

It is denoted by P. The set defines how the terminals and non-terminals can be combined. Every production(P) consists of non-terminals, an arrow, and terminals (the sequence of terminals). Non-terminals are called the left side of the production and terminals are called the right side of the production.

Start Symbol

The production begins from the start symbol. It is denoted by symbol S. Non-terminal symbol is always designated as start symbol.

IMPLEMENTATION ASPECTS OF SYNTACTIC ANALYSIS

Parsing is the process of analyzing a string of words to uncover its phrase structure, according to the rules of agrammar.

There are a number of algorithms researchers have developed for syntactic analysis, but we consider only the following simple methods –

- Context-Free Grammar
- Top-Down Parser

Context-Free Grammar

It is the grammar that consists rules with a single symbol on the left-hand side of the rewrite rules. Let us create grammar to parse a sentence −

"The bird pecks the grains"

Articles (DET) − a | an | the

Nouns − bird | birds | grain | grains

Noun Phrase (NP) − Article + Noun | Article + Adjective + Noun

= DET N | DET ADJ N

Verbs − pecks | pecking | pecked

Verb Phrase (VP) − NP V | V NP

Adjectives (ADJ) − beautiful | small | chirping

The parse tree breaks down the sentence into structured parts so that the computer can easily understand and process it. In order for the parsing algorithm to construct this parse tree, a set of rewrite rules, which describe what tree structures are legal, need to be constructed.

These rules say that a certain symbol may be expanded in the tree by a sequence of other symbols. According to first order logic rule, if there are two strings Noun Phrase (NP) and Verb Phrase (VP), then the string combined by NP followed by VP is a sentence. The rewrite rules for the sentence are as follows-

S → NP VP

NP → DET N | DET

ADJ NVP → V NP

Lexocon −

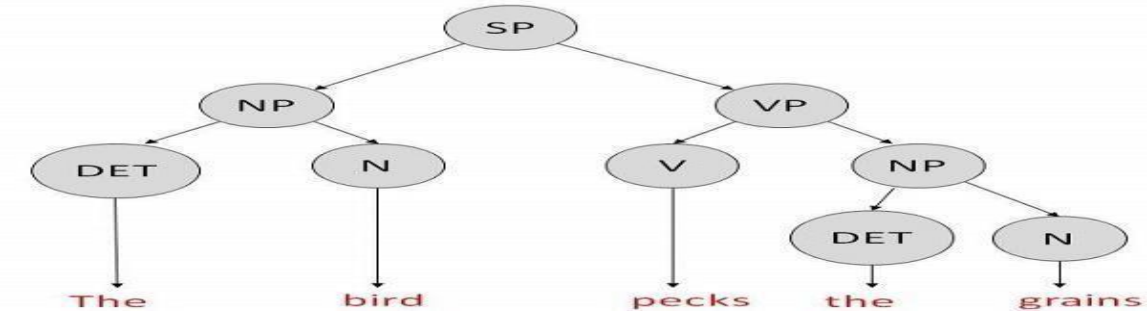
DET → a | the

ADJ → beautiful | perching

N → bird | birds | grain | grains

V → peck | pecks | pecking

The parse tree can be created as shown −



Now consider the above rewrite rules. Since V can be replaced by both, "peck" or "pecks", sentences such as "The bird peck the grains" can be wrongly permitted. i. e. the subject-verb agreement error is approvedas correct.

Merit − The simplest style of grammar, therefore widely used one.

Demerits −

- They are not highly precise. For example, “The grains peck the bird”, is a syntactically correct according to parser, but even if it makes no sense, parser takes it as a correct sentence.
- To bring out high precision, multiple sets of grammar need to be prepared. It may require a completely different sets of rules for parsing singular and plural variations, passive sentences, etc., which can lead to creation of huge set of rules that are unmanageable.

Top-Down Parser

Here, the parser starts with the S symbol and attempts to rewrite it into a sequence of *terminal symbols* that matches the classes of the words in the input sentence until it consists entirely of terminal symbols.

These are then checked with the input sentence to see if it matched. If not, the process is started over again with a different set of rules. This is repeated until a specific rule is found which describes the structure of the sentence.

Merit − It is simple to implement.

Demerits −

- It is inefficient, as the search process has to be repeated if an error occurs.
- Slow speed of working.

Natural Language Processing - Semantic Analysis

The purpose of semantic analysis is to draw exact meaning, or you can say dictionary meaning from the text. The work of semantic analyzer is to check the text for meaningfulness.

We already know that lexical analysis also deals with the meaning of the words, then how is semantic analysis different from lexical analysis? Lexical analysis is based on smaller token but on the other side semantic analysis focuses on larger chunks. That is why semantic analysis can be divided into the following two parts −

Studying meaning of individual word

It is the first part of the semantic analysis in which the study of the meaning of individual words is performed. This part is called lexical semantics.

Studying the combination of individual words

In the second part, the individual words will be combined to provide meaning in sentences.

The most important task of semantic analysis is to get the proper meaning of the sentence. For example, analyze the sentence **"Ram is great."** In this sentence, the speaker is talking either about Lord Ram or about a person whose name is Ram. That is why the job, to get the proper meaning of the sentence, of semantic analyzer is important.

Elements of Semantic Analysis

Followings are some important elements of semantic analysis −

Hyponymy

It may be defined as the relationship between a generic term and instances of that generic term. Here the generic term is called hypernym and its instances are called hyponyms. For example, the word color is hypernym and the color blue, yellow etc. are hyponyms.

Homonymy

It may be defined as the words having same spelling or same form but having different and unrelated meaning. For example, the word "Bat" is a homonymy word because bat can be an implement to hit a ball or bat is a nocturnal flying mammal also.

Polysemy

Polysemy is a Greek word, which means "many signs". It is a word or phrase with different but related sense. In other words, we can say that polysemy has the same spelling but different and related meaning. For example, the word "bank" is a polysemy word having the following meanings −

- A financial institution.
- The building in which such an institution is located.
- A synonym for "to rely on".

Difference between Polysemy and Homonymy

Both polysemy and homonymy words have the same syntax or spelling. The main difference between them is that in polysemy, the meanings of the words are related but in homonymy, the meanings of the words are not related. For example, if we talk about the same word "Bank", we can write the meaning 'a financial institution' or 'a river bank'. In that case it would be the example of homonym because the meanings are unrelated to each other.

Synonymy

It is the relation between two lexical items having different forms but expressing the same or a close meaning. Examples are 'author/writer', 'fate/destiny'.

Antonymy

It is the relation between two lexical items having symmetry between their semantic components relative to an axis. The scope of antonymy is as follows −

- **Application of property or not** − Example is 'life/death', 'certitude/incertitude'
- **Application of scalable property** − Example is 'rich/poor', 'hot/cold'
- **Application of a usage** − Example is 'father/son', 'moon/sun'.

Meaning Representation

Semantic analysis creates a representation of the meaning of a sentence. But before getting into the concept and approaches related to meaning representation, we need to understand the building blocks of semantic system.

Building Blocks of Semantic System

In word representation or representation of the meaning of the words, the following building blocks play an important role −

- **Entities** − It represents the individual such as a particular person, location etc. For example, Haryana. India, Ram all are entities.
- **Concepts** − It represents the general category of the individuals such as a person, city, etc.
- **Relations** − It represents the relationship between entities and concept. For example, Ram is a person.
- **Predicates** − It represents the verb structures. For example, semantic roles and case grammar are the examples of predicates.

Now, we can understand that meaning representation shows how to put together the building blocks of semantic systems. In other words, it shows how to put together entities, concepts, relation and predicatesto describe a situation. It also enables the reasoning about the semantic world.

Approaches to Meaning Representations

Semantic analysis uses the following approaches for the representation of meaning −

- First order predicate logic (FOPL)
- Semantic Nets
- Frames
- Conceptual dependency (CD)
- Rule-based architecture
- Case Grammar
- Conceptual Graphs

Need of Meaning Representations

A question that arises here is why do we need meaning representation? Followings are the reasons for the same −

Linking of linguistic elements to non-linguistic elements

The very first reason is that with the help of meaning representation the linking of linguistic elements to the non-linguistic elements can be done.

Representing variety at lexical level

With the help of meaning representation, unambiguous, canonical forms can be represented at the lexicallevel.

Can be used for reasoning

Meaning representation can be used to reason for verifying what is true in the world as well as to infer the knowledge from the semantic representation.

Lexical Semantics

The first part of semantic analysis, studying the meaning of individual words is called lexical semantics. It includes words, sub-words, affixes (sub-units), compound words and phrases also. All the words, sub-

words, etc. are collectively called lexical items. In other words, we can say that lexical semantics is the relationship between lexical items, meaning of sentences and syntax of sentence.

Following are the steps involved in lexical semantics −

- Classification of lexical items like words, sub-words, affixes, etc. is performed in lexical semantics.
- Decomposition of lexical items like words, sub-words, affixes, etc. is performed in lexical semantics.
- Differences as well as similarities between various lexical semantic structures is also analyzed.

MACHINE TRANSLATION

All translation systems must model the source and target languages, but systems vary in the type of models they use. Some systems attempt to analyze the source language text all the way into an interlingua knowledge representation and then generate sentences in the target language from that representation. This is difficult because it involves three unsolved problems: creating a complete knowledge representation of everything; parsing into that representation and generating sentences from that representation.

Other systems are based on a **transfer model**. They keep a database of translation rules (or examples), and whenever the rule (or example) matches, they translate directly. Transfer can occur at the lexical, syntactic, or semantic level.

What is a machine translation and how does it work?

Machine Translation or MT or robotized interpretation is simply a procedure when a computer software translates text from one language to another without human contribution. At its fundamental level, machine translation performs a straightforward replacement of atomic words in a single characteristic language for words in another.

Using corpus methods, more complicated translations can be conducted, taking into account better treatment of contrasts in phonetic typology, express acknowledgement, and translations of idioms, just as the seclusion of oddities. Currently, some systems are not able to perform just like a human translator, but in the coming future, it will also be possible.

In simple language, we can say that ***machine translation works by using computer software to translate the text from one source language to another target language.***

The term ‘machine translation’ (MT) refers to computerized systems responsible for producing translations with or without human assistance. It excludes computer-based translation tools that support translators by providing access to online dictionaries, remote terminology databanks, transmission and reception oftexts, etc.

Before the AI technology era, computer programs for the automatic translation of text from one language to another were developed. In recent years, AI has been tasked with making the automatic or machine translation of human languages’ fluidity and versatility of scripts, dialects, and variations. Machine translation is challenging given the inherent ambiguity and flexibility of human language.

Different types of machine translation in NLP

There are four types of machine translation:

1. **Statistical Machine Translation or SMT**

   It works by alluding to statistical models that depend on the investigation of huge volumes of bilingual content. It expects to decide the correspondence between a word from the source language

and a word from the objective language. ***A genuine illustration of this is Google Translate.***

Presently, SMT is extraordinary for basic translation, however its most noteworthy disadvantage is that it doesn't factor in context, which implies translation can regularly be wrong or you can say, don't expect great quality translation. There are several types of statistical-based machine translation models which are: ***Hierarchical phrase-based translation, Syntax-based translation, Phrase-based translation, Word-based translation.***

2. **Rule-based Machine Translation or RBMT**

   RBMT basically translates the basics of ***grammatical rules***. It directs a grammatical examination ofthe source language and the objective language to create the translated sentence. But, RBMT requires broad editing, and its substantial reliance on dictionaries implies that proficiency is accomplished after a significant period. (Also read: Top 10 Natural Processing Languages (NLP) Libraries with Python)

3. **Hybrid Machine Translation or HMT**

   HMT, as the term demonstrates, is a mix of RBMT and SMT. It uses a translation memory, making it unquestionably more successful regarding quality. Nevertheless, even HMT has a lot of downsides, the biggest of which is the requirement for enormous editing, and human translators will also be needed. There are several approaches to HMT like multi-engine, statistical rule generation, multi-pass, and confidence-based.

4. **Neural Machine Translation or NMT**

   NMT is a type of machine translation that relies upon neural network models (based on the human brain) to build statistical models with the end goal of translation. The essential advantage of NMT is that it gives a solitary system that can be prepared to unravel the source and target text. Subsequently, it doesn't rely upon specific systems that are regular to other machine translation systems, particularly SMT.

What are the benefits of machine translation?

One of the crucial benefits of machine translation is speed as you have noticed that computer programs ***can translate a huge amount of text rapidly.*** Yes, the human translator does their work more accurately but they cannot match the speed of the computer.

If you especially train the machine to your requirements, machine translation gives the ideal blend of brisk and cost-effective translations as ***it is less expensive*** than using a human translator. With a specially trained machine, MT can catch the setting of full sentences before translating them, which gives you high quality and human-sounding yield. Another benefit of machine translation is its ***capability to learn important words and reuse them wherever they might fit.***

Applications of machine translation

Machine translation technology and products have been used in numerous application situations, for example, business travel, the travel industry, etc. In terms of the object of translation, there are composed language-oriented content text translation and spoken language.

Machine Translation vs Human translation

- Machine translation hits that sweet spot of ***cost and speed***, offering a truly snappy path for brands to translate their records at scale without much overhead. Yet, that doesn't mean it's consistently relevant. On the other hand, human translation is incredible for those undertakings that require additional consideration and subtlety. Talented translators work on your image's substance to catch the first importance and pass on that feeling or message basically in another assortment of work.

- Leaning upon how much content should be translated, the machine translation can give translated content very quickly, though human translators will take additional time. Time spent finding, verifying, and dealing with a group of translators should likewise be considered.

- Numerous translation programming providers can give machine translations at practically zero cost, making it a reasonable answer for organizations who will be unable to manage the cost of expert translations.

- Machine Translation is the instant modification of text from one language to another utilizing artificial intelligence whereas a human translation, includes actual brainpower, in the form of one or more translators translating the text manually.

- **Text translation**

  Automated text translation is broadly used in an assortment of sentence-level and text-level translation applications. Sentence-level translation applications incorporate the translation of inquiry and recovery inputs and the translation of (OCR) outcomes of picture optical character acknowledgement. Text-level translation applications incorporate the translation of a wide range of unadulterated reports, and the translation of archives with organized data.

  Organized data mostly incorporates the presentation configuration of text content, object type activity, and other data, for example, textual styles, colours, tables, structures, hyperlinks, etc. Presently, the translation objects of machine translation systems are mostly founded on the sentence level.

  Most importantly, a sentence can completely communicate a subject substance, which normally frames an articulation unit, and the significance of each word in the sentence can be resolved to an enormous degree as per the restricted setting inside the sentence.

  Also, the methods and nature of getting data at the sentence level granularity from the preparation corpus are more effective than that dependent on other morphological levels, for example, words, expressions, and text passages. Finally, the translation depends on sentence-level can be normally reached out to help translation at other morphological levels.

- **Other applications**

Naturally, the task of machine translation is to change one source language word succession into another objective language word grouping which is semantically the same. Generally, it finishes a grouping transformation task, which changes over a succession object into another arrangement object as indicated by some information and rationale through model and algorithms.

All things considered, many undertaking situations total the change between grouping objects, and the language in the machine translation task is just one of the succession object types. In this manner, when the ideas of the source language and target language are stretched out from dialects to other arrangementobject types, machine translation strategies and techniques can be applied to settle numerous comparable change undertakings.

Speech translation : Speech recognition is the task of identifying a sequence of words uttered by a speaker, given the acoustic signal. It has become one of the mainstream applications of AI—millions of people interact with speech recognition systems every day to navigate voice mail systems, search the Web from mobile phones, and other applications. Speech is an attractive option when hands-free operation is necessary, as when operating machinery.

Speech recognition is difficult because the sounds made by a speaker are ambiguous and, well, noisy. As a well-known example, the phrase “recognize speech” sounds almost the same as “wreck a nice beach” when spoken quickly.

A speech understanding system must answer three questions:
1. What speech sounds did the speaker utter?
2. What words did the speaker intend to express with those speech sounds?
3. What meaning did the speaker intend to express with those words?

With the fast advancement of mobile applications, voice input has become an advantageous method of human-computer cooperation, and discourse translation has become a significant application situation. The fundamental cycle of discourse interpretation is "source language discourse source language text- target language text-target language discourse".

In this cycle, programmed text translation from source language text to target-language text is an important moderate module. What's more, the front end and back end likewise need programmed discourse recognition, ASR and text-to-speech, TTs.

The quality of a speech recognition system depends on the quality of all of its components the language model, the word-pronunciation models, the phone models, and the signal processing algorithms used to extract spectral featuresfrom the acoustic signal.

SPEECH RECOGNITION

Speech recognition refers to a computer interpreting the words spoken by a person and converting themto a format that is understandable by a machine. Depending on the end-goal, it is then converted to text orvoice or another required format.

For instance, Apple's Siri and Google's Alexa use AI-powered speech recognition to provide voice or text support whereas voice-to-text applications like Google Dictate transcribe your dictated words to text.Voice recognition is another form of speech recognition where a source sound is recognized and matched to a person's voice.

Speech recognition AI applications have seen significant growth in numbers in recent times as businesses are increasingly adopting digital assistants and automated support to streamline their services. Voice assistants, smart home devices, search engines, etc are a few examples where speech recognition has seen prominence. As per Research and Markets, the global market for speech recognition is estimated to growat a CAGR of 17.2% and reach $26.8 billion by 2025.

Speech recognition is fast overcoming the challenges of poor recording equipment and noise cancellation, variations in people's voices, accents, dialects, semantics, contexts, etc using artificial intelligence and machine learning. This also includes challenges of understanding human disposition, and the varyinghuman language elements like colloquialisms, acronyms, etc. The technology can provide a 95% accuracy now as compared to traditional models of speech recognition, which is at par with regular human communication.

Furthermore, it is now an acceptable format of communication given the large companies that endorse it and regularly employ speech recognition in their operations. It is estimated that a majority of search engines will adopt voice technology as an integral aspect of their search mechanism.

Speech recognition and AI play an integral role in NLP models in improving the accuracy and efficiency ofhuman language recognition.

Use Cases of Speech Recognition

Let's explore the uses of speech recognition applications in different fields:

1. Voice-based speech recognition software is now used to initiate purchases, send emails, transcribe meetings, doctor appointments, and court proceedings, etc.
2. Virtual assistants or digital assistants and smart home devices use voice recognition software to answer questions, provide weather news, play music, check traffic, place an order, and so on.
3. Companies like Venmo and PayPal allow customers to make transactions using voice assistants. Several banks in North America and Canada also provide online banking using voice-based software.
4. Ecommerce is significantly powered by voice-based assistants and allows users to make purchases quickly and seamlessly.
5. Speech recognition is poised to impact transportation services and streamline scheduling, routing, and navigating across cities.
6. Podcasts, meetings, and journalist interviews can be transcribed using voice recognition. It is also used to provide accurate subtitles to a video.
7. There has been a huge impact on security through voice biometry where the technology analyses the varying frequencies, tone and pitch of an individual's voice to create a voice profile. An example of this is Switzerland's telecom company Swisscom which has enabled voice authentication technology in its call centres to prevent security breaches.

8. Customer care services are being traced by AI-based voice assistants, and chatbots to automate repeatable tasks.

As speech recognition and AI impact both professional and personal lives at workplaces and homes respectively, the demand for skilled AI engineers and developers, Data Scientists, and Machine Learning Engineers, is expected to be at an all-time high.

There will be a requirement for skilled AI professionals to enhance the relationship between humans and digital devices. As job opportunities are created, they will result in increased perks and benefits for those in this field.

CHAPTER-2:

WHAT IS PERCEPTION IN AI?

Perception is a process to interpret, acquire, select and then organize the sensory information that is captured from the real world. Perception provides agents with information about the world they inhabit. Perception is initiated by **sensors.** A sensor is anything that can change the computational state of the agent in response to a change in the state of the world. It could be as simple as a one-bit sensor that detects whether a switch is on or off, or as complex as the retina of the human eye, which contains more than a hundred million photosensitive elements.

For example: Human beings have sensory receptors such as touch, taste, smell, sight and hearing. So, the information received from these receptors is transmitted to human brain to organize the received information.

Perception in Artificial Intelligence is the process of interpreting vision, sounds, smell, and touch. Perception helps to build machines or robots that react like humans. Perception is a process to interpret, acquire, select, and then organize the sensory information from the physical world to make actions like humans. The main difference between AI and robot is that the robot makes actions in the real world.

According to the received information, action is taken by interacting with the environment to manipulate and navigate the objects.

Perception provides agents with information about the world they inhabit by interpreting the response of **sensors**. A sensor measures some aspect of the environment in a form that can be used as input by an agent program. The sensor could be as simple as a switch, which gives one bit telling whether it is on or off, or as complex as the eye. A variety of sensory modalities are available to artificial agents.

Perception and action are very important concepts in the field of Robotics. The following figures show the complete **autonomous robot.**

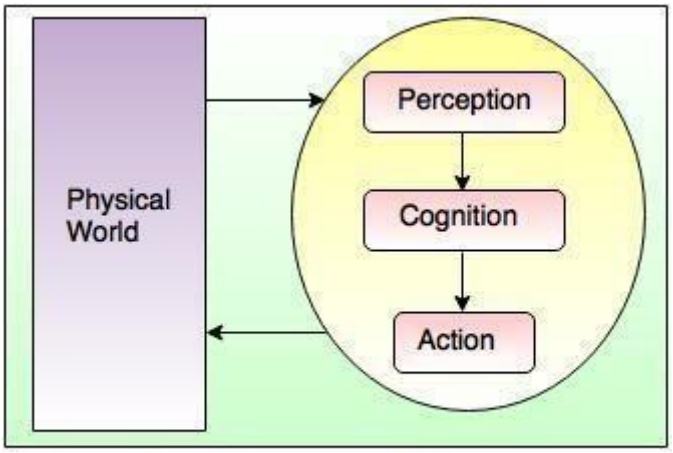


Fig: Autonomous Robot

There is one important difference between the artificial intelligence program and robot. The AI program performs in a computer stimulated environment, while the robot performs in the physical world.

For example:

In chess, an AI program can be able to make a move by searching different nodes and has no facility to touch or sense the physical world.

However, the chess playing robot can make a move and grasp the pieces by interacting with the physical world.

Image formation in digital camera

Image formation is a physical process that captures object in the scene through lens and creates a 2-D image.

Let's understand the geometry of a pinhole camera shown in the following diagram.

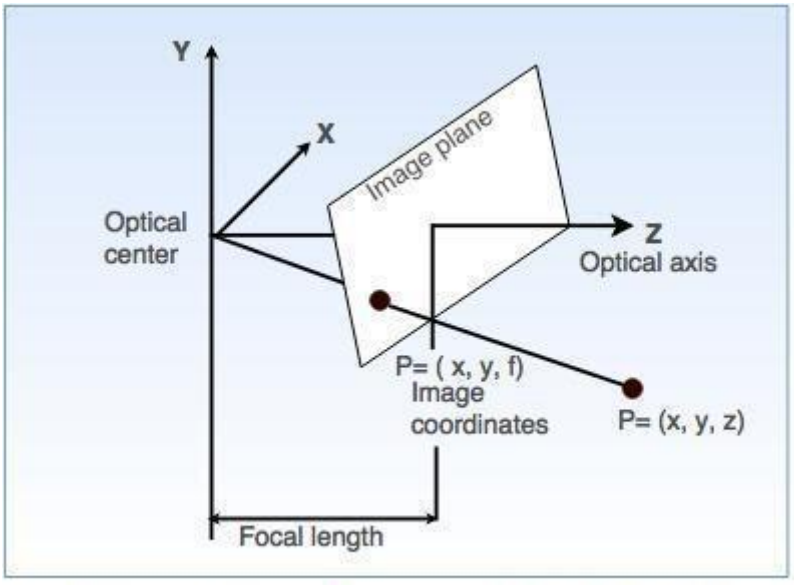


Fig: Geometry of Image Formation in the pinhole camera

In the above figure, an optical axis is perpendicular to the image plane and image planeis generally placed in front of the optical center.

So, let **P** be the point in the scene with coordinates (X,Y,Z) and **P'** be its image planewith coordinates (x, y, z).

If the focal length from the optical center is **f**, then by using properties of similartriangles, equation is derived as,

$-x/f = X/Z$ so $x = - fX/Z$ ..... equation (i)
$-y/f = -Y/Z$ so $y = - fY/Z$ ..... equation (ii)

These equations define an image formation process called as **perspective projection.**

EARLY IMAGE PROCESSING OPERATIONS

From the inception of AI, image processing to be incorporated in smart systems is a perennial project for people working on it. In its initial phase, it required a lot of manual input, providing instructions to computers, to get some output. These machines, or known as Expert Systems, were trained to recognize images.

There are 2 methods of image processing:

- – Analog image processing, which is used for processing photographs, printouts, and other image hardcopies.
- – Digital image processing, which is used for manipulating digital images with the help of complexalgorithms

Main Purpose of Image Processing

- – Representing processed data in a visual way one can understand, for instance, giving a visual form toinvisible objects.
- – To improve the processed image quality, image sharpening and restoration works well.
- – Image convalescence helps in searching images.
- – Helps to measure objects in the image.
- – With pattern recognition, it becomes easy to classify objects in the image, locate their position and getan overall understanding of the scene.

Image Processing Phases

There are 8 phases for image processing which goes step-wise:

- **Image acquisition**:
  Captures the image with a sensor and converts it into a manageable entity
- **Image enhancement**
  The input image quality is improved and also extracts details hidden in it
- **Image restoration**
  Any possible corruption like blur, noise, or camera misfocus is removed to get a cleaner vision on probabilistic and mathematical model basis
- **Color image processing**
  The colored images and varied color spaces are processed with pseudocolor or RGB processing way.
- **Image compression and decompression**

This allows for changes in image resolution and size, be it for reduction or restoring images depending onthe need.

- **Morphological processing**
  Defines the object structure and shape in the image.
- **Image recognition**
  For a particular object, the specific features are identified in the image and techniques like objectdetection are used for the same.
- **Representation and description**
  is all about visualizing the processed data.

IMAGE PROCESSING METHODS, TECHNIQUES, AND TOOLS

The images captured with regular sensors need preprocessing as some could contain too much noise or are misfocused. There are two detection techniques to be used for processing digital images as well as for preprocessing.

- **Filtering**
  Used to modify and enhance the input image. With various filters available, certain features in the imagecan be emphasized or removed, can also reduce the image noise and so on.
- **Edge detection**
  Used for data extraction and image segmentation, to find meaningful object edges in the images that are preprocessed.

To make things easier, there are specific libraries and frameworks that can be used to implement imageprocessing functionalities.

Object Recognition from Structural information,

Conventionally the recognition of objects in a scene is achieved by digital image processing, whosepurpose is directly related to artificial vision or computer vision. Artificial vision aims to detect, segment, locate, and recognize particular objects. Thus, the following methodology is proposed: (a) segmentation, (b) intelligent recognition, and, (c) extraction of characteristics.

A. Segmentation

The proposed image processing consists of converting the image to grayscale, obtaining a lighter image format. To obtain a lighter image format is needed to convert on grayscale images. Then, edgesof the images are detected by the derivative method; before the image is enlarged and eroded to closethe found edges. Finally, the borders are filled, achieving a mask that identifies the position of the object inside the image. Each step is described in detail below:

1) Greyscale Transformation. It consists of determining the equivalent of the luminance, which is defined as the light received on a surface being defined as the relation of the luminous flux on the illuminated area , luminance concept is associated with human eye perception of different light intensity. The luminance is calculated based on the weighted average of the color components of each pixel, as shown in Equation 1, where L corresponds to the luminance, R is the red component. Gthe green component and B the blue component.

$$L = 0.2989 * R + 0.5870 * G + 0.1140 * B \quad (1)$$

Equation 1 should be used for calculation grayscale to each pixel; this process is required for the imageedge detection.

2) Images Edges Detection. The image obtained in the previous step can be represented as a discrete function in 2D, which is defined by the coordinates of each pixel *m* and *n*. The discrete value ofthe function is evaluated at a specific point; its procedure is known as brightness or pixel intensity. An edge is defined as tone changes between pixels, in cases where changes exceed a threshold value, it isconsidered an edge. Different methods to identify edges have been proposed, one of them is the intensity gradient of each pixel, using a convolution mask, then magnitude is calculated finally, the threshold process is applied [14].

The most used edge detection techniques employ local operators [15], using discrete approximations ofthe first and second of grayscales images, hereunder it will be described the proposed operator, whichis based on the first derivative of the image.

What is the purpose of edge detection/image detection?

Edges are straight lines or curves in the image plane across which there is a "significant" change in image brightness.The goal of edge detection is to abstract away from the messy, multi megabyte image and toward a more compact, abstract representation

- Edge detection operation is used in an image processing.
- The main goal of edge detection is to construct the ideal outline of an image.
- Discontinuity in brightness of image is affected due to:

i) Depth discontinuities
ii) Surface orientation discontinuities
iii) Reflectance discontinuities
iv) Illumination.

B. Artificial Intelligence for Object Recognition

- The image is reconstructed using a Hopfield neural network; the network eliminates noise in edge image, improving accuracy and precision, and obtaining a proper segmented image.
- Artificial intelligence is known as "the science and engineering of making intelligent machines, especially intelligent computer programs", this definition was proposed by Professor JohnMccarthy, in 1956 . The target of artificial intelligence is to think, evaluate or act accordingto certain inputs to exercise some specific function, to achieve this, different processes could be performed: i.) Genetic algorithms, ii.) Artificial neural networks and iii.) Formal logic.
- For the specific problem, artificial neural networks are employed, it uses elements information processors, where local interactions depend on overall system performance. Networks consist of a large number of simple processing elements called nodes or neurons that are arranged in layers. Each neuron is connected to other neurons by communication links, which have a weight associated. Weights represent the information that will be used by the neural network to solve a given problem. Now, there are various types of neural networks, such as self-organization, recurrent, among others.

3D-Information extraction using vision

Why extraction of 3-D information is necessary?The 3-D information extraction process plays an important role to perform the tasks like manipulation, navigation and recognition. It deals with the following aspects:

1. Segmentation of the scene

The segmentation is used to arrange the array of image pixels into regions. This helps tomatch semantically meaningful entities in the scene.

- The goal of segmentation is to divide an image into regions which are homogeneous.
- The union of the neighboring regions should not be homogeneous.
- **Thresholding** is the simplest technique of **segmentation.** It is simply performed on theobject, which has an homogeneous intensity and a background with a different intensity level and the pixels are partitioned depending on their intensity values.

2. To determine the position and orientation of each object Determination of the position and orientation of each object relative to the observer is important for manipulation and navigation tasks. **For example:** Suppose a person goes to a store to buy something. While moving around he must know the locations and obstacles, so that he can make the plan and path to avoid them.

- The whole orientation of image should be specified in terms of a three dimensional rotation.

3. To determine the shape of each and every object When the camera moves around an object, the distance and orientation of that object will change but it is important to preserve the shape of that object.**For example:** If an object is cube, that fact does not change, but it is difficult to represent the global shape to deal with wide variety of objects present in the real world.

- If the shape of an object is same for some manipulating tasks, it becomes easy to decidehow to grasp that object from a particular place.
- The **object recognition** plays most significant role to identify and classify the objects asan example only when the geometric shapes are provided with color and texture. **However, a question arises that, how should we recover 3-D image from thepinhole camera?**

There are number of techniques available in the visual stimulus for 3D-image extraction such as **motion, binocular stereopsis, texture, shading, and contour**. Each of these techniques operates on the background assumptions about physical scene to provide interpretation.

EXTRACTING 3-D INFORMATION USING VISION

We need to extract 3-D information for performing certain tasks such as manipulation, navigation, and recognition. There are threte aspects of this:

1. Segmentation of the scene into distinct objects.
2. Determining the position and orientation of each object relative to the observer.
3. Determining the shape of each object.

Segmentation of the image is a key step towards organizing the array of image pixels into regions that would correspond to semantically meaningful entities in the scene. For recognition, we would like to know what features belong together so that one could compare them with stored models; to grasp an object, one needs to know what belongs together as an object.

Segmentation is best viewed as part of extraction of 3-D information about the scene.

One of the principal uses of vision is to provide information for manipulating objects—picking them up, grasping, twirling, and so on—as well as navigating in a scene while avoiding obstacles. The capability to use vision for these purposes is present in the most primitive of animal visual systems. Perhaps the evolutionary origin of the vision sense can be traced back to the presence of a photosensitive spot on one end of an organism that enabled it to orient itself toward (or away from) the light. Flies use vision based on optic flow to control their landing responses. Mobile robots moving around in an environment need to know where the obstacles are, where free space corridors are available, and so on.

Unit-V: Robotics: Introduction, Robot Hardware, Robotic Perception, Planning to move, planning uncertain movements, Moving, Robotic software architectures, application domains

Philosophical foundations: Weak AI, Strong AI, Ethics and Risks of AI, Agent Components, Agent Architectures, Arewe going in the right direction, What if AI does succeed.

Robotics is a branch of engineering and science that includes electronics engineering, mechanical engineering and computer science and so on. This branch deals with the design, construction, use to control robots, sensory feedback and information processing.

These are some technologies which will replace humans and human activities in coming years. These robots are designed to be used for any purpose but these are using in sensitive environments like bomb detection, deactivation of various bombs etc. Robots can take any form but many of them have given the human appearance. The robotswhich have taken the form of human appearance may likely to have the walk like humans, speech, cognition and most importantly all the things a human can do.

Most of the robots of today are inspired by nature and are known as bio-inspired robots. Robotics is that branch of engineering that deals with conception, design, operation, and manufacturing of robots. There was an author named Issac Asimov, he said that he was the first person to give robotics name in a short story composed in 1940's.

In that story, Issac suggested three principles about how to guide these types of robotic machines. Later on, these three principles were given the name of Issac's three laws of Robotics. These three laws state that:

- Robots will never harm human beings.
- Robots will follow instructions given by humans with breaking law one.
- Robots will protect themselves without breaking other rules.

Difference in Robot System and Other AI Program

Here is the difference between the two −

AI Programs	Robots
They usually operate in computer-stimulated worlds.	They operate in real physical world
The input to an AI program is in symbolsand rules.	Inputs to robots is analog signal in the form of speech waveform or images
They need general purpose computers to operate on.	They need special hardware with sensors and effectors.

Robot Locomotion

Locomotion is the mechanism that makes a robot capable of moving in its environment. There arevarious types of locomotion are

- Legged
- Wheeled
- Combination of Legged and Wheeled Locomotion
- Tracked slip/skid

Legged Locomotion

- This type of locomotion consumes more power while demonstrating walk, jump, trot, hop, climb up or down, etc.
- It requires more number of motors to accomplish a movement. It is suited for rough as well as smooth terrain where irregular or too smooth surface makes it consume more power fora wheeled locomotion. It is little difficult to implement because of stability issues.
- It comes with the variety of one, two, four, and six legs. If a robot has multiple legs then leg coordination is necessary for locomotion.

Wheeled Locomotion

It requires fewer number of motors to accomplish a movement. It is little easy to implement as there are less stability issues in case of more number of wheels. It is power efficient as compared to legged locomotion.

- **Standard wheel** − Rotates around the wheel axle and around the contact
- **Castor wheel** − Rotates around the wheel axle and the offset steering joint.
- **Swedish 45° and Swedish 90° wheels** − Omni-wheel, rotates around the contact point, around the wheel axle, and around the rollers.
- **Ball or spherical wheel** − Omnidirectional wheel, technically difficult to implement.

Slip/Skid Locomotion

In this type, the vehicles use tracks as in a tank. The robot is steered by moving the tracks with different speeds in the same or opposite direction. It offers stability because of large contact area of track and ground.

Characteristics

There are some characteristics of robots given below:

- **Appearance:** Robots have a physical body. They are held by the structure of their body and are moved by their mechanical parts. Without appearance, robots will be just a software program.
- **Brain:** Another name of brain in robots is On-board control unit. Using this robot receive information and sends commands as output. With this control unit robot knowswhat to do else it'll be just a remote-controlled machine.
- **Sensors:** The use of these sensors in robots is to gather info from the outside world and send it to Brain. Basically, these sensors have circuits in them that produces the voltage in them.

- **Actuators:** The robots move and the parts with the help of these robots move is called Actuators. Some examples of actuators are motors, pumps, and compressoretc. The brain tells these actuators when and how to respond or move.
- **Program:** Robots only works or responds to the instructions which are provided to them in the form of a program. These programs only tell the brain when to perform which operation like when to move, produce sounds etc. These programs only tell therobot how to use sensors data to make decisions.
- **Behaviour:** Robots behavior is decided by the program which has been built for it. Once the robot starts making the movement, one can easily tell which kind of programis being installed inside the robot.

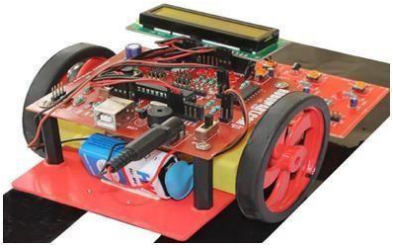
TYPES OF ROBOTS

- **Articulated:** The feature of this robot is its rotary joints and range of these are from 2to 10 or more joints. The arm is connected to the rotary joint and each joint is known as the axis which provides a range of movements.
- **Cartesian:** These are also known as gantry robots. These have three joints which usethe Cartesian coordinate system i.e x, y, z. These robots are provided with attached wrists to provide rotatory motion.
- **Cylindrical:** These types of robots have at least one rotatory joints and one prismaticjoint which are used to connect the links. The use of rotatory joints is to rotate along the axis and prismatic joint used to provide linear motion.
- **Polar:** These are also known as spherical robots. The arm is connected to base with atwisting joint and have a combination of 2 rotatory joint and one linear joint.
- **Scara:** These robots are mainly used in assembly applications. Its arm is in cylindricalin design. It has two parallel joints which are used to provide compliance in one selected plane.
- **Delta:** The structure of these robots are like spider-shaped. They are built by joint parallelograms that are connected to the common base. The parallelogram moves in adome-shaped work area. These are mainly used in food and electrical industries.

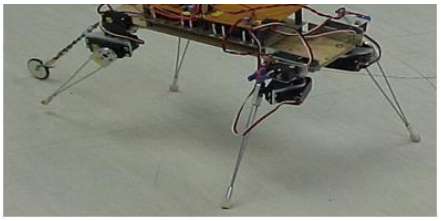
1) Mobile Robots

Mobile robots are able to move from one location to another location using locomotion. It is an automatic machine that is capable of navigating an uncontrolled environment without any requirement of physical and electromechanical guidance devices. Mobile Robots are of two types:

(a) Rolling robots - Rolling robots require wheels to move around. They can easily and quickly search. But they are only useful in flat areas.



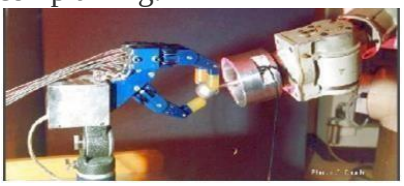
(b) Walking robots - Robots with legs are usually used in condition where the terrain is rocky. Most walking robots have at least 4 legs.

- 

2) Industrial Robots

- Industrial robots perform same tasks repeatedly without ever moving. These robots are working in industries in which there is requirement of performing dull and repeated taskssuitable for robot.

- An industrial robot never tired, it will perform their works day and night without ever complaining.

- 

3) Autonomous Robots

- Autonomous robots are self-supported. They use a program that provides them the opportunity to decide the action to perform depending on their surroundings.
- Using artificial intelligence these robots often learn new behavior. They start with a short routine and adapt this routine to be more successful in a task they perform. Hence, the most successful routine will be repeated.

- 

4) Remote Controlled Robots

- Remote controlled robot used for performing complicated and undetermined tasks that autonomous robot cannot perform due to uncertainty of operation.
- Complicated tasks are best performed by human beings with real brainpower. Therefore a person can guide a robot by using remote. Using remote controlled operation human can perform dangerous tasks without being at the spot where the tasks are performed.
- Let's see a NASA robot designed to explore volcanoes via remote control:

- 

Scope and limitations of robots: The advance version of machines are robots which are used to do advanced tasks and are programmed to make decisions on their own.When a robot is designed the most important thing to be kept in mind is that What the function is to be performed and what are the limitations of the robot.

Each robot has a basic level of complexity and each of the levels has the scope which limits the functions that are to be performed. For general basic robots, their complexity is decided by the number of limbs, actuators and the sensors that are used while for advanced robots the complexity is decided by the number of microprocessors and microcontroller used.

As increasing any component in the robot, it is increasing the scope of the robot and withevery joint added, the degree of the robot is enhanced.

Advantages: The advantages of using robots are given below:

- They can get information that a human can't get.
- They can perform tasks without any mistakes and very efficiently and fast.
- Maximum robots are automatic, so they can perform different tasks without needinghuman interaction.
- Robots are used in different factories to produce items like plane, car parts etc.
- They can be used for mining purposes and can be sent to earth's madrid.

Disadvantages: The disadvantages of using robots are given below:

- They need the power supply to keep going. People working in factories may lose theirjobs as robots can replace them.
- They need high maintenance to keep them working all day long. And the cost ofmaintaining the robots can be expensive.
- They can store huge amount of data but they are not as efficient as our human brains.
- As we know that robots work on the program that has been installed in them. So otherthan the program installed, robots can't do anything different.
- The most important disadvantage is that if the program of robots comes in wronghands they can cause the huge amount of destruction.

Applications: Different types of robots can performs different types of tasks. For example, many of the robots are made for assembly work which means that they are not relevant for any other work and these types of robots are called Assembly Robots.

Similarly, for seam welding many suppliers provide robots with their welding materials and these types of robots are known as Welding Robots. While on the other hand many robots are designed for heavy-duty work and are known as Heavy Duty Robots.

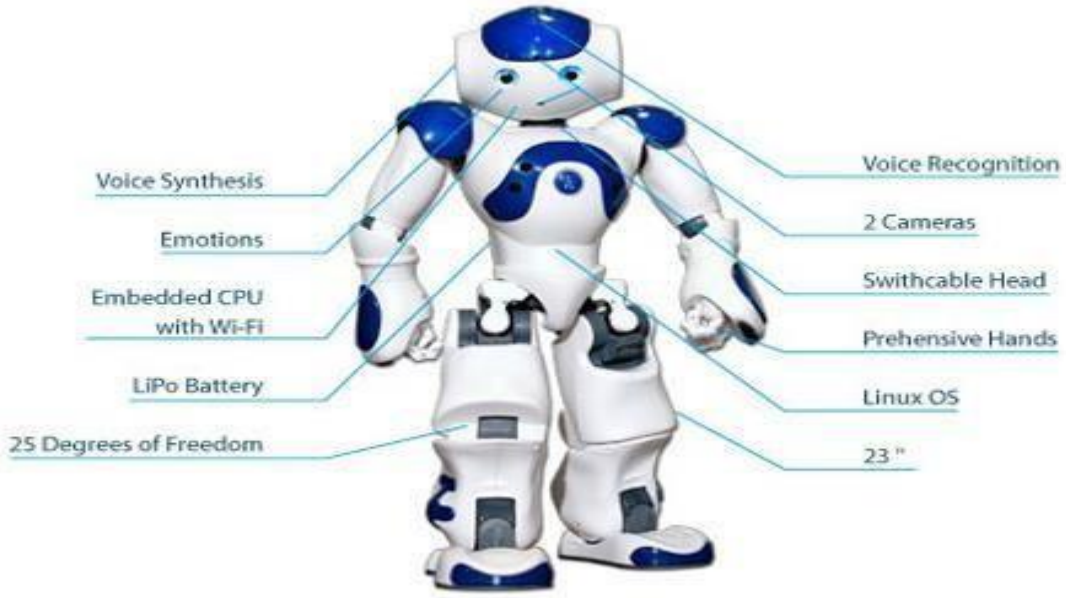
There are some applications given below:

- Caterpillar plans which is aiming to develop remote-controlled machines and areexpecting to develop heavy robots by 2021.
- A robot can also do Herding task.

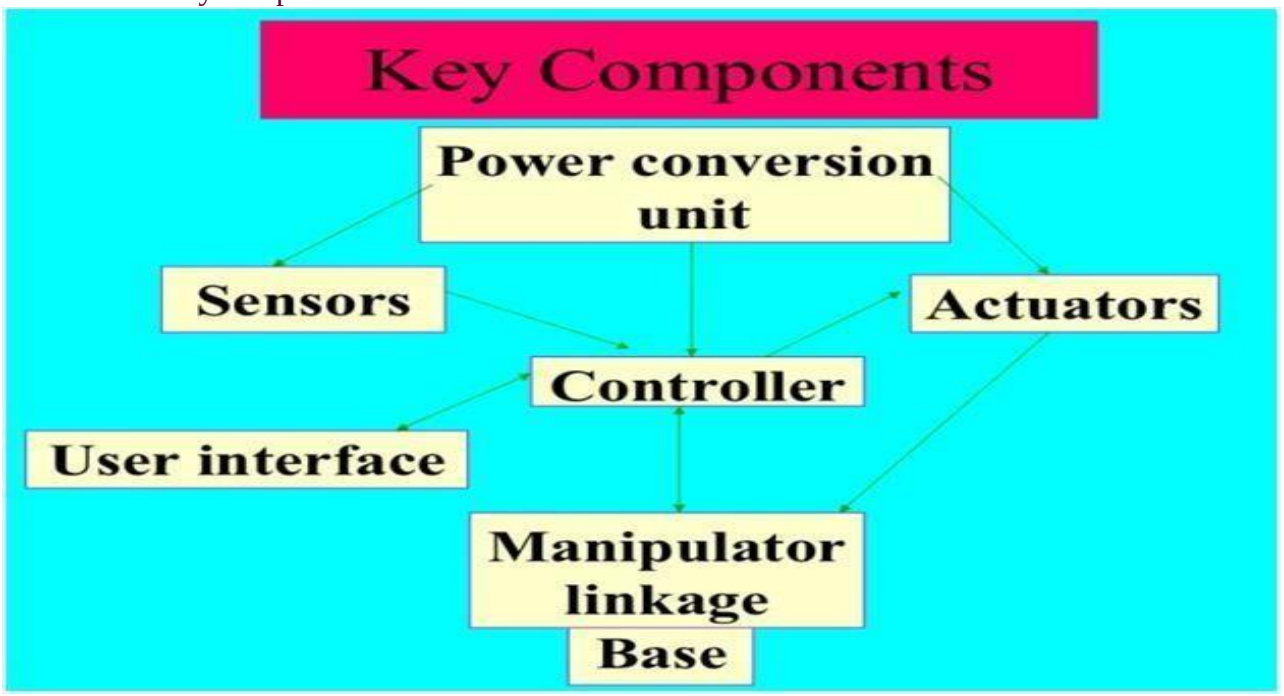
- Robots are increasingly been used more than humans in manufacturing while in auto-industry there are more than half of the labors are “Robots”.
- Many of the robots are used as Military Robots.
- Robots have been used in cleaning up of areas like toxic waste or industrial wastesetc.
- Some other types of Robots are Agricultural robots, Household robots. Domesticrobots. Nano robots. Swarm robots.

Components of Robot

Consider the robot structure showing different components of robots are:



Consider the key components of robotics are:-



Hardware of Computer Vision System

This involves −

- Power supply
- Image acquisition device such as camera
- A processor
- A software
- A display device for monitoring the system
- Accessories such as camera stands, cables, and connectors

- **Power Supply -** The working power to the robot is provided by batteries, hydraulic, solar power, or pneumatic power sources.
- **Actuators -** Actuators are the energy conversion device used inside a robot. The major function of actuators is to convert energy into movement.
- **Electric motors (DC/AC)-** Motors are electromechanical component used for converting electrical energy into its equivalent mechanical energy. In robots motors are used for providing rotational movement.
- **Sensors -** Sensors provide real time information on the task environment. Robots are equipped with tactile sensor it imitates the mechanical properties of touch receptors of human fingerprints and a vision sensor is used for computing the depth in the environment.
- **Controller -** Controller is a part of robot that coordinates all motion of the mechanical system. It also receives an input from immediate environment through various sensors. The heart of robot's controller is a microprocessor linked with the input/output and monitoring device. The command issued by the controller activates the motion control mechanism, consisting of various controller, actuators and amplifier.

Tasks of Computer Vision

- **OCR** − In the domain of computers, Optical Character Reader, a software to convert scanned documents into editable text, which accompanies a scanner.
- **Face Detection** − Many state-of-the-art cameras come with this feature, which enables to read the face and take the picture of that perfect expression. It is used to let a user access the software on correct match.
- **Object Recognition** − They are installed in supermarkets, cameras, high-end cars such asBMW, GM, and Volvo.
- **Estimating Position** − It is estimating position of an object with respect to camera as in position of tumor in human’s body.

Types of Robot Sensors

There are different type of sensors are available to choose from and the characteristics of sensors are used for determining the type of sensor to be used for particular application.

Passive sensors, such as cameras, are true observers of the environment: they capture signals that are generated by ACTIVE SENSOR other sources in the environment. **Active sensors**, such as sonar, sendenergy into the environment.

They rely on the fact that this energy is reflected back to the sensor. Active sensors tend to provide more information than passive sensors, but at the expense of increased power consumption and with a danger of interference when multiple active sensors are used at the same time. Whether active or passive, sensors can be divided into three types, depending on whether they sense the environment, the robot's location, or the robot's internal configuration.

RANGE FINDER **Range finders** are sensors that measure the distance to nearby objects. In the early SONAR SENSORS days of robotics, robots were commonly equipped with **sonar sensors**. Sonar sensors emitdirectional sound waves, which are reflected by objects, with some of the sound making it back into the sensor. The time and intensity of the returning signal indicates the distance to nearby objects. Sonar is the technology of choice for autonomous underwater vehicles. **Stereo vision** (see Section 24.4.2) relies STEREO VISION on multiple cameras to image the environment from slightly different viewpoints, analyzing the resulting parallax in these images to compute the range of surrounding objects.

1) Light Sensor

Light sensor is a transducer used for detecting light and creates a voltage difference equivalent to the light intensity fall on a light sensor.

2) Proximity Sensor

Proximity sensor can detect the presence of nearby object without any physical contact. The working of a proximity sensor is simple. In proximity sensor transmitter transmits an electromagnetic radiation and receiver receives and analyzes the return signal for interruptions. Therefore the amount of light receiver receives by surrounding can be used for detecting the presence of nearby object.

3) Sound Sensor

Sound sensors are generally a microphone used to detect sound and return a voltage equivalent to the sound level. Using sound sensor a simple robot can be designed to navigate based on the sound receives.

4) Temperature Sensor

Temperature sensors are used for sensing the change in temperature of the surrounding. It is based on the principle of change in voltage difference for a change in temperature this change in voltage will provide the equivalent temperature value of the surrounding.

Few generally used temperature sensors IC?s are TMP35, TMP37, LM34, LM35, etc.

5) Acceleration Sensor

Acceleration sensor is used for measuring acceleration and tilt. An accelerometer is a device usedfor measuring acceleration.

Effectors:

Effectors are the means by which robots move and change the shape of their bodies. To understand the design of effectors, it will help to talk about motion and shape in the abstract, using the concept of a **degree of freedom** (DOF) We count one degree of freedom for each independent direction in which a robot, or one of its effectors, can move. For example, a rigid mobile robot such as an AUV has six degreesof freedom, three for its (x, y, z) location in

space and three for its angular orientation, known as *yaw*, *roll*, and *pitch*. These six degrees define the **kinematic state**2 or **pose** of the robot. The **dynamic state** of a robot includes these Six plus an additional six dimensions for the rate of change of each kinematic dimension, that is, their velocities.

2) ROBOTIC PERCEPTION:

Perception is the process by which robots map sensor measurements into internal representations of the environment. Perception is difficult because sensors are noisy, and the environment is partially observable, unpredictable, and often dynamic. In other words, robots have all the problems of **state estimation** (or **filtering**). As a rule of thumb, good internal representations for robots have three properties: they contain enough information for the robot to make good decisions, they are structured so that they can be updated efficiently, and they are natural in the sense that internal variables correspond to natural state variables inthe physical world.

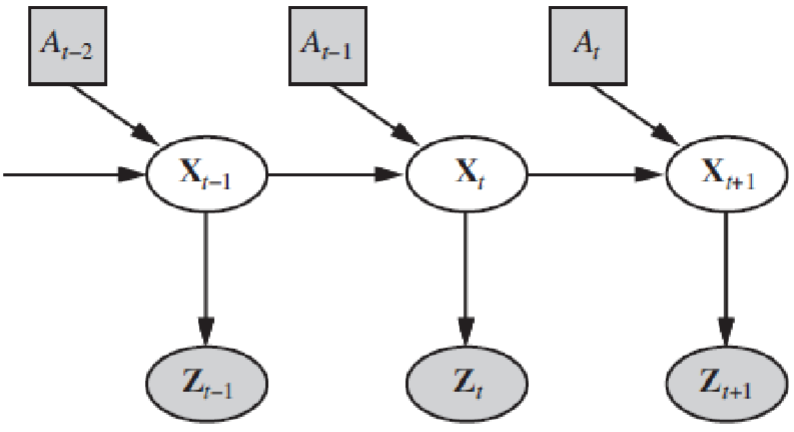


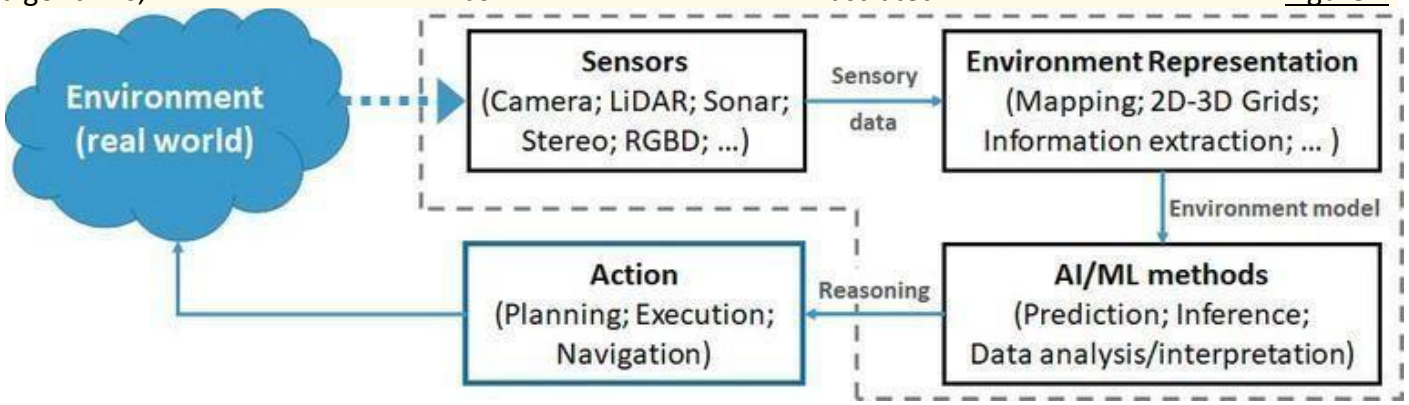
Figure 25.7 Robot perception can be viewed as temporal inference from sequences of actions and measurements, as illustrated by this dynamic Bayes network.

Machine learning plays an important role in robot perception. This is particularly the case when the best internal representation is not known. One common approach is to map high dimensional sensor streams into lower-dimensional spaces using unsupervised machine learning methods. Such an LOW-DIMENSIONAL approach is called **low-dimensional embedding**. EMBEDDING Machine learning makes it possible to learn sensor and motion models from data, while simultaneously discovering a suitable internal representations.

Robotic perception is related to many applications in robotics where sensory data and artificial intelligence/machine learning (AI/ML) techniques are involved. Examples of such applications are object detection, environment representation, scene understanding, human/pedestrian detection, activity recognition, semantic place classification, object modeling, among others.

Robotic perception, in the scope of this, encompasses the ML algorithms and techniques that empower robots to learn from sensory data and, based on learned models, to react and take decisions accordingly. The recent developments in machine learning, namely deep-learning approaches, are evident and, consequently, robotic perception systems are evolving in a way that new applications and tasks arebecoming a reality.

In robotics, perception is understood as a system that endows the robot with the ability to perceive, comprehend, and reason about the surrounding environment. The key components of a perception systemare essentially sensory data processing, data representation (environment modeling), and ML-based algorithms, as illustrated in Figure 1.



Key modules of a typical robotic perception system: sensory data processing (focusing here on visual and range perception); data representations specific for the tasks at hand; algorithms for data analysis and interpretation (using AI/ML methods); and planning and execution of actions for robot-environment interaction.

Robotic perception is crucial for a robot to make decisions, plan, and operate in real-world environments, by means of numerous functionalities and operations from occupancy grid mapping to object detection. Some examples of robotic perception subareas, including autonomous robot-vehicles, are obstacle detection, object recognition, semantic place classification, 3D environment representation, gesture and voice recognition, activity classification, terrain classification, road detection, vehicle detection, pedestrian detection, object tracking, human detection, and environment change detection.

Nowadays, most of robotic perception systems use machine learning (ML) techniques, ranging from classical to deep-learning approaches. Machine learning for robotic perception can be in the form of unsupervised learning, or supervised classifiers using handcrafted features, or deep-learning neural networks or even a combination of multiple methods.

Regardless of the ML approach considered, data from sensor(s) are the key ingredient in robotic perception. Data can come from a single or multiple sensors, usually mounted onboard the robot, but can also come from the infrastructure or from another robot (e.g., cameras mounted on UAVs flying nearby).

In multiple-sensors perception, either the same modality or multimodal, an efficient approach is usually necessary to combine and process data from the sensors before an ML method can be employed. Data alignment and calibration steps are necessary depending on the nature of the problem and the type of sensors used.

Sensor-based environment representation/mapping is a very important part of a robotic perception system. Mapping here encompasses both the acquisition of a metric model and its semantic interpretation, and is therefore a synonym of environment/scene representation.

Robot perception functions, like localization and navigation, are dependent on the environment where the robot operates. Essentially, a robot is designed to operate in two categories of environments: indoors or outdoors. Therefore, different assumptions can be incorporated in the mapping (representation) and perception systems considering indoor or outdoor environments.

Moreover, the sensors used are different depending on the environment, and therefore, the sensory data to be processed by a perception system will not be the same for indoors and outdoors scenarios.

3) PLANNING TO MOVE:

All of a robot's deliberations ultimately come down to deciding how to move effectors. The POINT-TO-POINT **point-to-point motion** problem is to deliver the robot or its end effectors to a designated target location. A greater challenge is the **compliant motion** problem, in which a robot moves while being in physical contact with an obstacle. An example of compliant motion is a robot manipulator that screws in a light bulb, or a robot that pushes a box across a table top.

The **path planning** problem is to find a path from one configuration to another in configuration space. There are two main approaches: **cell decomposition** and **skeletonization**. Each reduces the continuous path-planning problem to a discrete graph-search problem.

We will start with a simple representation for a simple robot motion problem. Consider the robot arm, It has two joints that move independently. Moving the joints alters the (x, y) coordinates of the elbow and the gripper. (The arm cannot move in the z direction.) This suggests that the robot's configuration can be described by a four dimensional coordinate: (xe, ye) for the location of the elbow relative to the environment and (xg, yg) for the location of the gripper. Clearly, these four coordinates characterize thefull state of the robot. They constitute what is known as **workspace representation.** Workspace representations are well-suited for collision checking, especially if the robot and all objects are represented by simple polygonal models.

CELL DECOMPOSITION METHODS

The first CELL approach to path planning uses **cell decomposition**—that is, it decomposes the free space into a finite number of contiguous regions, called cells. These regions have the important property that the path-planning problem within a single region can be solved by simple means (e.g., moving along a straight line). The path-planning problem then becomes a discrete graph-search problem, very much like the search problems.

The simplest cell decomposition consists of a regularly spaced grid. It shows a square grid decomposition of the space and a solution path that is optimal for this grid size. Grayscale shading indicates the *value* of each

free-space grid cell—i.e., the cost of the shortest path from that cell to the goal. (These values can be computed by a deterministic
form of the VALUE-ITERATION algorithm given shows the corresponding workspace trajectory for the arm. Of course, we can also use the A∗ algorithm to find a shortest path. A solution path that includes such a cell may not be a real solution, because there may be no way to cross the cell in the desired direction in a straight line.

Cell decomposition methods can be improved in a number of ways, to alleviate some of these problems. The first approach allows *further subdivision* of the mixed cells—perhaps using cells of half the original size. This can be continued recursively until a path is found that lies entirely within free cells. (Of course, the method only works if there is a way to decide if a given cell is a mixed cell, which is easy only if the configuration space boundaries have relatively simple mathematical descriptions.) This method is complete provided there is a bound on the smallest passageway through which a solution must pass.

A second way to obtain a complete algorithm is to insist on an **exact cell decomposition** of the free space. This EXACT CELL method must allow cells to be irregularly shaped DECOMPOSITION where they meet the boundaries of free space, but the shapes must still be "simple" in the sense that it should be easy to compute a traversal of any free cell. This technique requires some quite advanced geometric ideas, so we shall not pursue it further here.

SKELETONIZATION METHODS

SKELETONIZATION The second major family of path-planning algorithms is based on the idea of **skeletonization**. These algorithms reduce the robot's free space to a one-dimensional representation, for which the planning problem is easier. This lower-dimensional representation is called a **skeleton** of the configuration space.

Figure 25.18 shows an example skeletonization: it is a **Voronoi graph** of the free space—the set of all points that are equidistant to two or more obstacles. To do path planning with a Voronoi graph, the robot first changes its present configuration to a point on the Voronoi graph. It is easy to show that this can always be achieved by a straight-line motion in configuration space. Second, the robot follows the Voronoi graph until it reaches the point nearest to the target configuration. Finally, the robot leaves the Voronoi graph and moves to the target. Again, this final step involves straight-line motion in configuration space.

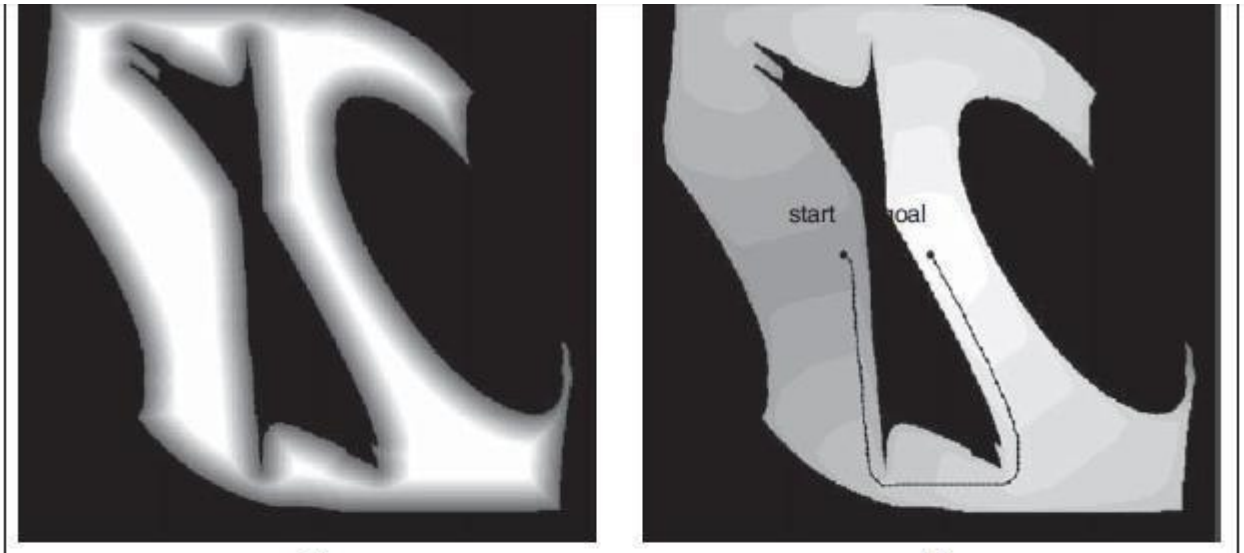


Figure 25.17 (a) A repelling potential field pushes the robot away from obstacles. (b) Path found by simultaneously minimizing path length and the potential.

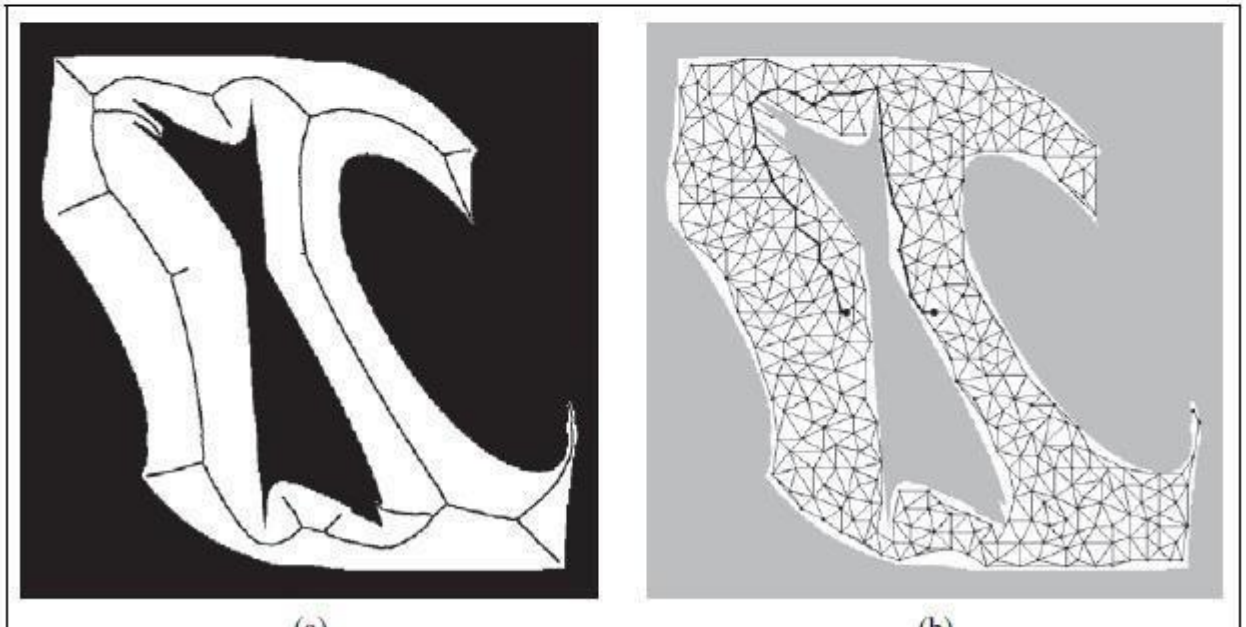


Figure 25.18 (a) The Voronoi graph is the set of points equidistant to two or more obstacles in configuration space. (b) A probabilistic roadmap, composed of 400 randomly chosen points in free space.

In this way, the original path-planning problem is reduced to finding a path on the Voronoi graph, which is generally one-dimensional (except in certain nongeneric cases) and has finitely many points where three or more one- dimensional curves intersect.

4) PLANNING UNCERTAIN MOVEMENTS:

None of the robot motion-planning algorithms discussed thus far addresses a key characteristic of robotics problems: *uncertainty*. In robotics, uncertainty arises from partial observability of the environment and

from the stochastic (or unmodeled) effects of the robot's actions. Errors can also arise from the use of approximation algorithms such as particle filtering, which does not provide the robot with an exact belief state even if the stochastic nature of the environment is modeled perfectly.

Planning paths through
configuration space is already a challenging problem; it would be worse if we had to work with a full probability distribution over states. Ignoring uncertainty in this way works when the uncertainty is small. In fact, when the environment model changes over time as the result of incorporating sensor measurements, many robots plan paths online during plan execution. ONLINE REPLANNING This is the **online replanning** technique

The field of robotics has adopted a range of techniques for accommodating uncertainty. Some are derived from the algorithms for decision making under uncertainty. For example, if it is uncertain about a critical state variable, it can rationally invoke an **information gathering action.**

Uncertainty can also be handled using so-called **robust control** methods rather than probabilistic methods. A robust method is one that assumes a *bounded* amount of uncertainty in each aspect of a problem, but does not assign probabilities to values within the allowed interval. A robust solution is one that works no matter what actual values occur, provided they are within the assumed interval. An extreme form of robust method is the **conformant planning** approach

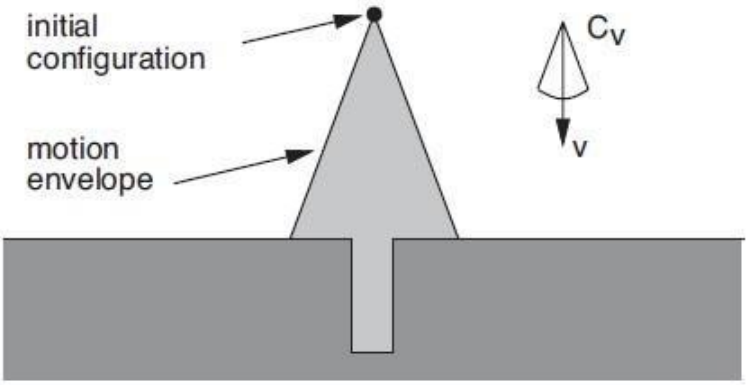


Figure 25.19 A two-dimensional environment, velocity uncertainty cone, and envelope of possible robot motions. The intended velocity is v, but with uncertainty the actual velocity could be anywhere in C_v, resulting in a final configuration somewhere in the motion envelope, which means we wouldn't know if we hit the hole or not.

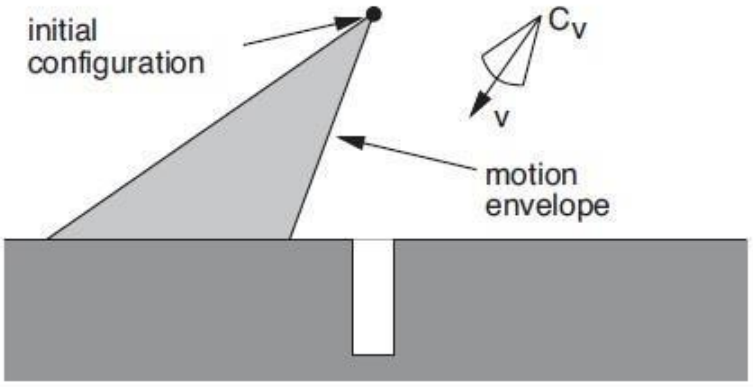


Figure 25.20 The first motion command and the resulting envelope of possible robot motions. No matter what the error, we know the final configuration will be to the left of the hole.

As one might imagine, the problem of *constructing* fine-motion plans is not trivial; in fact, it is a good deal harder than planning with exact motions. One can either choose a fixed number of discrete values for each motion or use the environment geometry to choose directions that give qualitatively different behavior. A fine-motion planner takes as input the configuration-space description, the angle of the velocity uncertainty cone, and a specification of what sensing is possible for termination (surface contact in this case). It should produce a multistep conditional plan or policy that is guaranteed to succeed, if such a plan exists.

5) ROBOTIC SOFTWARE ARCHITECTURES

A methodology for structuring algorithms is called a **software architecture**. An architecture includes languages and tools for writing programs, as well as an overall philosophy for how programs can be brought together.

Modern-day software architectures for robotics must decide how to combine reactive control and model-based deliberative planning. In many ways, reactive and deliberate techniques have orthogonal strengths and weaknesses. Reactive control is sensor-driven and appropriate for making low-level decisions in real time. However, it rarely yields a plausible solution at the global level, because global control decisions depend on information that cannot be sensed at the time of decision making. For such problems, deliberate planning is a more appropriate choice.

Consequently, **most robot architectures use reactive techniques at the lower levels of control and deliberative techniques at the higher levels.** Architectures that combine reactive and deliberate techniques are called **hybrid architectures**.

The **subsumption architecture** (Brooks, 1986) is a framework for assembling reactive controllers out of finite state machines. Nodes in these machines may contain tests for certain sensor variables, in which case the execution trace of a finite state machine is conditioned on the outcome of such a test. Arcs can be tagged with messages that will be generated when traversing them, and that are sent to the robot's motors or to other finite state machines. Additionally, finite state machines possess internal timers (clocks) that control the time it takes to traverse an arc. The resulting machines are refereed to as **augmented finite state machines**,or **AFSMs**, where the augmentation refers to the use of clocks.

An example of a simple AFSM is the four-state machine shown in Figure 25.24(b), which generates cyclic leg motion for a hexapod walker. This AFSM implements a cyclic controller, whose execution mostly does not rely on environmental feedback. The forward swing phase, however, does rely on sensor feedback. If the leg is stuck, meaning that it hasfailed to execute the forward swing, the robot retracts the leg, lifts it upa little higher, and attempts to execute the forward swing once again. Thus, the controller is able to *react* to contingencies arising from the interplay of the robot and its environment.

Unfortunately, the subsumption architecture has its own problems. First, the AFSMs are driven by raw sensor input, an arrangement that works if the sensor data is reliable and contains all necessary information for decision making, but fails if sensor data has to be integrated in nontrivial ways over time. Subsumption-style controllers have therefore mostly been applied to simple tasks.

THREE-LAYER ARCHITECTURE

Hybrid architectures combine reaction with deliberation. The most popular hybrid architecture is the **three-layer architecture**, which consists of a reactive layer, an executive layer, and a deliberative layer.
The **reactive layer** provides low-level control to the robot. It is characterized by a tight sensor–action loop. Its decision cycle is often on the order of milliseconds.

The **executive layer** (or sequencing layer) serves as the glue between the reactive layer and the deliberative layer. It accepts directives by the deliberative layer, and sequences them for the reactive layer.

The **deliberative layer** generates global solutions to complex tasks using planning. Because of the computational complexity involved in generating such solutions, its decision cycle is often in the order of

minutes. The deliberative layer (or planning layer) uses models for decision making. Those models might be either learned from data or supplied and may utilize state information gathered at the executive layer.
Variants of the three-layer architecture can be found in most modern-day robot software systems. The decomposition into three layers is not very strict. Some robot software systems possess additional layers, such as user interface layers that control the interaction with people, or a multiagent level for coordinating a robot's actions with that of other robots operating in the same environment.

PIPELINE ARCHITECTURE
Another architecture for robots is known as the **pipeline architecture**. Just like the subsumption architecture, the pipeline architecture executes multiple process in parallel. However, the specific modules in this architecture resemble those in the three-layer architecture.

The key to the pipeline architecture is that this all happens in parallel. While the perception layer processes the most recent sensor data, the control layer bases its choices on slightly older data. In this way, the pipeline architecture is similar to the human brain. We don't switch off our motion controllers when we digest new sensor data. Instead, we perceive, plan, and act all at the same time. Processes in the pipeline architecture run asynchronously, and all computation is data-driven. The resulting system is robust, and itis fast.

6) APPLICATION DOMAINS:
Here are some of the prime application domains for robotic technology. **Industry and Agriculture.** Traditionally, robots have been fielded in areas that require difficult human labor, yet are structured enough to be amenable to robotic automation. The best example is the assembly line, where manipulators routinely perform tasks such as assembly, part placement, material handling, welding, and painting. In many of these tasks, robots have become more cost-effective than human workers. Outdoors, many of the heavy machines that we use to harvest, mine, or excavate earth have been turned into robots.

Transportation. Robotic transportation has many facets: from autonomous helicopters that deliver payloads to hard-to-reach locations, to automatic wheelchairs that transport people who are unable to control wheelchairs by themselves, to autonomous straddle carriers that outperform skilled human drivers when transporting containers from ships to trucks on loading docks. A prime example of indoor transportation robots, or gofers, is the Helpmate robot.

Robotic cars. Most of use cars every day. Many of us make cell phone calls while driving. Some of us even text. The sad result: more than a million people die every year in traffic accidents. Robotic cars like BOSS and STANLEY offer hope: Not only will they make driving much safer, but they will also free us from the need to pay attention to the road during our daily commute.

Health care. Robots are increasingly used to assist surgeons with instrument placement when operating on organs as intricate as brains, eyes, and hearts. Figure 25.28(b) shows such a system. Robots have become indispensable tools in a range of surgical procedures, such as hip replacements, thanks to their high precision. In pilot studies, robotic devices have been found to reduce the danger of lesions when performing colonoscopy. Outside the operating room, researchers have begun to develop robotic aides for elderly and handicapped people, such as intelligent robotic walkers and intelligent toys that provide reminders to take medication and provide comfort. Researchers are also working on robotic devices for rehabilitation that aid people in performing certain exercises.

Hazardous environments. Robots have assisted people in cleaning up nuclear waste, most notably in Chernobyl and Three Mile Island. Robots were present after the collapse of the World Trade Center, where they entered structures deemed too dangerous for human search and rescue crews.

Some countries have used robots to transport ammunition and to defuse bombs—a notoriously dangerous task. A number of research projects are presently developing prototype robots for clearing minefields, on land and at sea.

Personal Services. Service is an up-and-coming application domain of robotics. Service robots assist individuals in performing daily tasks. Commercially available domestic service robots include autonomous vacuum cleaners, lawn mowers, and golf caddies. The world's most popular mobile robot is a personal service robot: the robotic vacuum cleaner ROOMBA **Roomba**,

Human augmentation. A final application domain of robotic technology is that of human augmentation. Researchers have developed legged walking machines that can carry people around, very much like a wheelchair. Several research efforts presently focus on the development of devices that make it easier for people to walk or move their arms by providing additional forces through extra skeletal attachments.

PHILOSOPHICAL FOUNDATIONS:

Weak AI, Strong AI:

Artificial intelligence (AI) is already having a huge impact on our lives, making it critical to remain ahead of the game. When AI continues to take on more human-like elements of perception, the forms of artificial intelligence are a way to envision what the future of AI could look like. It's crucial to consider the different forms of AI in order to predict how they'll grow in the future. Keep reading to learn more about the various types of artificial intelligence.

Weak AI

The most common type of AI today is weak AI. Weak AI is sweeping the globe, from smartphone apps to the Internet to big data analytics.

The name comes from the fact that these artificial intelligence systems are specifically designed toperform one mission. They are also known as 'Narrow AI' because of their limited approach and inability to perform tasks other than those assigned to them. This narrows their intelligence to a single task or group of tasks, enabling for more tweaking and optimization.

Strong AI

Artificial general intelligence (AGI), also known as strong AI or deep AI, refers to the idea of a machine with general intelligence that can learn and apply its intelligence to solve every problem.

In any given case, AGI can think, comprehend, and behave in a manner that is distinct from that ofa person.

Strong AI employs a philosophy of mind AI framework, which is defined as the ability to perceive other intelligent entities' desires, feelings, values, and thought processes. Even so, due to recent advances in nanotechnology, Strong AI, which is still in its infancy, holds a lot of promise. Moreover, Strong AI is being considered as a potential application for the development of an artificial neural network that can act as a proper human being.

Super AI

As per a report, according to University of Oxford scholar and AI expert Nick Bostrom, when AI becomes much smarter than the best human brains in practically every field, including scientific creativity, general wisdom and social skills, we've achieved Artificial Super Intelligence.

Artificial Super Intelligence (ASI) can outperform human intelligence in all areas, including innovation, general intelligence, and problem-solving. Machines would be able to demonstrate knowledge previously only seen in the best of us. This is the kind of AI that many people are concerned about, and the kind of AI that Elon Musk believes will lead to humanity's extinction.

2) ETHICS AND RISKS OF AI:

Artificial Intelligence ethics, or AI ethics, comprise a set of values, principles, and techniques whichemploy widely accepted standards of right and wrong to guide moral conduct in the development and deployment of Artificial Intelligence technologies.

Robot ethics, also known as roboethics or machine ethics, is concerned with what rules should be applied to ensure the ethical behavior of robots as well as how to design ethical robots. Roboethics deals with concerns and moral dilemmas such as whether robots will pose a threat to humans in the long run, or whether using some robots, such as killer robots in wars, can become problematic for humanity.

Roboticists must guarantee that autonomous systems can exhibit ethically acceptable behavior in situations where robots, AI systems, and other autonomous systems such as self-driving vehicles interact with humans.

Artificial Intelligence (AI) and automation are dramatically changing and influencing our society. Applying principles of AI ethics to the design and implementation of algorithmic or intelligent systems and AI projects in the public sector is paramount. AI ethics will guarantee that the development and deployment of Artificial Intelligence are ethical, safe, and uttermost responsible.

The new interconnected digital world powered by 5G technology is delivering great potential and rapid gains in the power of Artificial Intelligence to better society. Innovation and implementation of AI are already making an impact on improving services from healthcare, education, and

transportation to the food supply chain, energy, and environmental management plans, to mention just a few.

With the rapid advancements in computing power and access to vast amounts of big data, Artificial Intelligence and Machine Learning systems will continue to improve and evolve. In just a few years into the future, AI systems will be able to process and use data not only at even more speed but also with more accuracy.

As always, with power comes great responsibility. Despite the advantages and benefits that technologies such as Artificial Intelligence bring to the world, they may potentially cause irreparable harm to humans and society if they are misused or poorly designed. The development of AI systems must always be responsible and developed toward optimal sustainability for public benefit.

Artificial Intelligence ethics and potential harms caused by AI systems

AI ethics and safety must be a priority in the design and implementation of AI systems. AI Ethics emerges to avoid individual and societal harms caused by the misuse, abuse, poor design, or unintended negative consequences of AI systems.

According to Dr. David Leslie, Ethics Theme Lead within the public policy program and Ethics Fellow at The Alan Turing Institute in London, England, potential harms caused by AI systems include the following:

- **AI systems: Bias and discrimination**

AI systems designers choose the features, metrics, and analytics structures of the models that enabledata mining. Thus, data-driven technologies, such as Artificial Intelligence, can potentially replicate the preconceptions and biases of their designer.

- **AI systems: Denial of individual autonomy, recourse, and rights**

In the past, AI systems that automate cognitive functions were attributable exclusively to accountable human agents. Today, AI systems make decisions, predictions, and classifications that affect citizens.

Certain situations may arise where such individuals are unable to hold accountable the parties responsible for the outcomes. One of the most common responses from humans to justify negative results is to blame the AI system, adding that there is nothing they can do to change the outcome. Something which is not real.

Such a response is utterly ridiculous since AI systems are designed and programmed by a human designer. Therefore, a human is who can correct and change an outcome that is not satisfactory. Take as an example a case of injuries, or a negative consequence such an accountability gap, which may harm the autonomy and violate the rights of the affected individuals.

- **AI systems: Non-transparent, unexplainable, or unjustifiable outcomes**

In some cases, machine learning models may generate their results by operating on high- dimensional correlations that are beyond the interpretive capabilities of human reasoning.

These are cases in which the rationale of algorithmically produced outcomes that directly affect decision subjects may remain opaque to those subjects. In some use cases, this lack of explainability may not be a cause of too much trouble.

However, in applications where the processed data could harbor traces of discrimination, bias, inequity, or unfairness, the lack of clarity of the model may be deeply problematic.

- **AI systems: Invasions of privacy**

AI systems pose threats to privacy in two ways:

- As a result of their design and development processes
- As a result of their deployment

AI projects lay on the grounds of the structuring and processing of big data. Massive amounts of personal data are collected, processed, and utilized to develop AI technologies. More often than not, big data is captured and extracted without gaining the proper consent of the data owner subject. Quite often, some use of big data reveals —or places under risk— personal information, compromising the privacy of the individual.

The deployment of AI systems can target, profile, or nudge data owner subjects without their knowledge or consent. It means that such AI systems are infringing upon the ability of the individuals to lead a private life. Privacy invasion can consequently harm the right to pursue goals or life plans free from unchosen influence.

- **AI systems: Isolation and disintegration of social connection**

The capacity of AI systems to curate individual experiences and to personalize digital services has the potential of improving consumer life and service delivery. This, which is a benefit if done right, yet it comes with potential risks.

Such risks may not be visible or show as risks at the start. However, excessive automation may potentially lead to the reduction of human-to-human interaction, and with it, solving problematic situations at an individual level could not be possible any longer.

Algorithmically enabled hyper-personalization might improve customer satisfaction, but limits our exposure to worldviews different from ours, and this might polarize social relationships.

Since the times of Greek philosopher Plato, well-ordered and cohesive societies have built on relations of human trust, empathy, and mutual understanding. As Artificial Intelligence technologies become more prevalent, it is paramount that these relations of human trust, or empathy, or mutual understanding remain intact.

- **AI systems: Unreliable, unsafe, or poor-quality outcomes**

The implementation and distribution of AI systems that produce unreliable, unsafe, or poor-quality outcomes may be the result of irresponsible data management, negligent design production processes, or questionable deployment practices. Consequently, this can directly lead to damaging the wellbeing of individuals as well as damaging the public welfare.

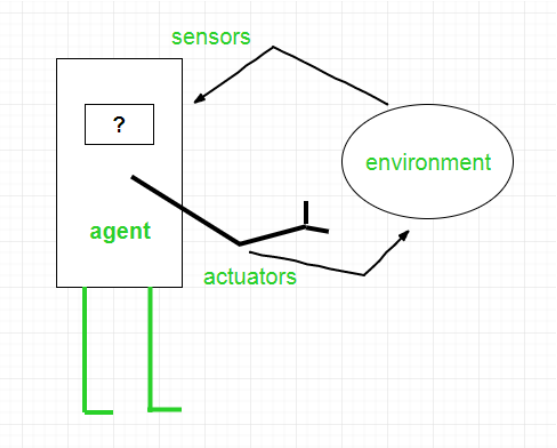
Agents in Artificial Intelligence

Artificial intelligence is defined as the study of rational agents. A rational agent could be anything that makes decisions, as a person, firm, machine, or software. It carries out an action with the best outcome after considering past and current percepts(agent's perceptual inputs at a given instance). An AI system is composed of an **agent and its environment**.

The agents act in their environment. The environment may contain other agents.
An agent is anything that can be viewed as :

- perceiving its environment through **sensors** and
- acting upon that environment through **actuators**

Note: Every agent can perceive its own actions (but not always the effects)

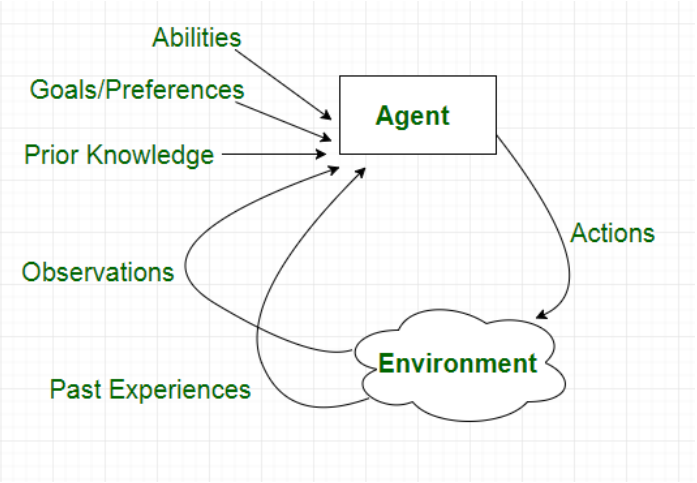


To understand the structure of Intelligent Agents, we should be familiar with *Architecture* and *Agent* programs. **Architecture** is the machinery that the agent executes on. It is a device with sensors and actuators, for example, a robotic car, a camera, a PC. **Agent program** is an implementation of an agent function. An **agent function** is a map from the percept sequence(history of all that an agent has perceived to date) to an action.

Agent = Architecture + Agent Program

Examples of Agent:

- A **software agent** has Keystrokes, file contents, received network packages which act as sensorsand displays on the screen, files, sent network packets acting as actuators.
- A Human-agent has eyes, ears, and other organs which act as sensors, and hands, legs, mouth,and other body parts acting as actuators.
- A **Robotic agent** has Cameras and infrared range finders which act as sensors and variousmotors acting as actuators.



An agent can be anything that perceive its environment through sensors and act upon that environment through actuators. An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**.An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensorsand hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, NLP for sensorsand various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and acton those inputs and display output on the screen.

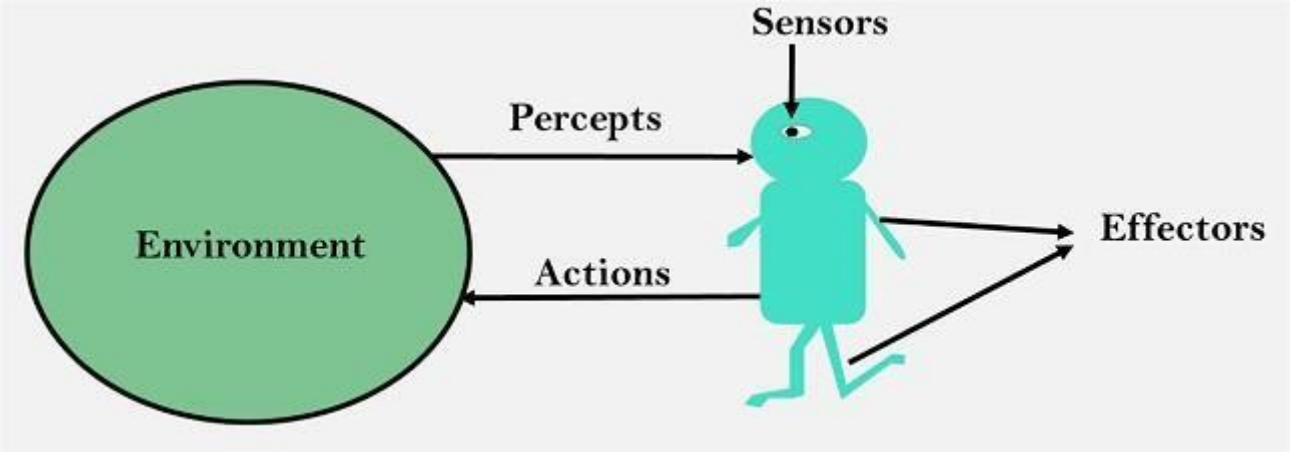
Hence the world around us is full of agents such as thermostat, cellphone, camera, and even weare also agents.

Before moving forward, we should first know about sensors, effectors, and actuators.

Sensor: Sensor is a device which detects the change in the environment and sends the informationto other electronic devices. An agent observes its environment through sensors.

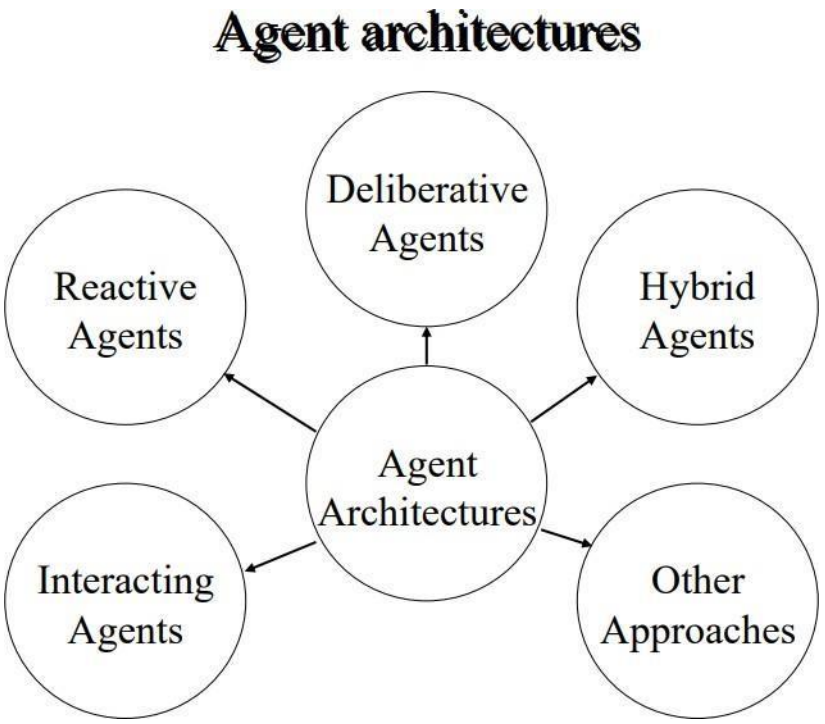
Actuators: Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.

Effectors: Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.



Types of Agents
Agents can be grouped into four classes based on their degree of perceived intelligence and capability :

- Simple Reflex Agents
- Model-Based Reflex Agents
- Goal-Based Agents
- Utility-Based Agents
- Learning Agent



ARE WE GOING IN THE RIGHT DIRECTION, WHAT IF AI DOES SUCCEED.

For decades, AI scientists and researchers have been trying to recreate the logic and functionalities of the human brain. And for decades, they have dismayed themselves and the general public. Today, we've reached a point where artificial intelligence algorithms can solve very complicated problems, and in many cases with speed and accuracy that is far superior to those of humans. But whether contemporary AI works likes the human mind is up for debate.

"Most AI research has been conducted as if the computation available to the agent were constant (in which case leveraging human knowledge would be one of the only ways to improve performance) but, over a slightly longer time than a typical research project, massively more computation inevitably becomes available. Seeking an improvement that makes a difference in the shorter term, researchers seek toleverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation," Sutton says, adding that "the human-knowledge approach tends to complicate methods in ways that make them less suited to taking advantage of general methods leveraging computation."

Contrary to human intelligence, artificial intelligence benefits from Moore's law, which maintains that computing power doubles in power and halves in price every two years. Sutton generalizes the idea to "continued exponentially falling cost per unit of computation."

Moore's law has held for decades after Intel co-founder Gordon Moore made the prediction in the 1960s. While the speed of advances in computing power has slowed down in the past decade, it still beats the evolution of the human brain by a factor of several million (or maybe more).

Basically, what this means is that we must acknowledge the fundamental differences between the evolution of the human brain and AI. The human brain has been optimized to find techniques to use its unchanging infrastructure in new and innovative ways. AI on the other hand can be optimized to apply techniques that can scale with new and more powerful hardware.

"Too many AI researchers think real-world problems are not relevant. The community's hyperfocus on novel methods ignores what's really important."

The AI community largely ignores the obvious: deep learning is an experimental science. Despite their formal roots, neural networks are massive non-linear systems that defy explanation. Although the growing body of work dedicated to their interpretation, neural networks remain as mysterious as ever. The only reliable tool we have to understand them is the scientific method — which is deeply rooted in experimentation.

HOW CAN AI BE DANGEROUS?

Most researchers agree that a superintelligent AI is unlikely to exhibit human emotions like love or hate, and that there is no reason to expect AI to become intentionally benevolent or malevolent. Instead, when considering how AI might become a risk, experts think two scenarios most likely:

1. **The AI is programmed to do something devastating:** Autonomous weapons are artificial intelligence systems that are programmed to kill. In the hands of the wrong person, these weapons could easily cause mass casualties. Moreover, an AI arms race could inadvertently lead to an AI war that also results in mass casualties. To avoid being thwarted by the enemy, these weapons would be designed to be extremely difficult to simply "turn off," so humans could plausibly lose control of such a situation. This risk is one that's present even with narrow AI, but grows as levels of AI intelligence and autonomy increase.
2. **The AI is programmed to do something beneficial, but it develops a destructive method for achieving its goal:** This can happen whenever we fail to fully align the AI's goals with ours, which is strikingly difficult. If you ask an obedient intelligent car to take you to the airport as fast as possible, it might get you there chased by helicopters and covered in vomit, doing not what you wanted but literally what you asked for. If a superintelligent system is tasked with a ambitious geoengineering project, it might wreak havoc with our ecosystem as a side effect, and view human attempts to stop it as a threat to be met.